

药材烘干问题的数学推导与代码解释

写给完全没有基础的读者

2026 年 9 月 13 日

这份文档的目标是让一个没有学过偏微分方程、没有写过科学计算程序的人，也能完整看懂 2026 年 A 题药材烘干问题的建模过程与程序实现。全文分成五个部分，先后关系如下。

部分	标题	内容
第一部分	数学与物理基础	偏导数、守恒、微分方程、边界条件，以及从物理规律推出控制方程的全过程
第二部分	数值方法原理	为什么算不出来、坐标变换、谱方法、时间推进、烘干时间的判定、收缩的处理
第三部分	代码逐行解释	主求解器、参数文件、闭式解、驱动脚本，每一段代码在做什么
第四部分	如何确认结果正确	用解析解、网格加密、多种方法互相验证
第五部分	其余程序逐行解释	两套对照格式、结果表格的生成、插图的生成，以及整条产出链
第六部分	问题一闭式解完整推导	齐次化、准静态分解、特征值问题、正交展开，从头推到尾

阅读建议：如果时间紧，可以只读第一部分第一到第三章、第二部分第五章和第六章，再读第三部分，这样能抓住主干。如果每一个符号都想弄懂，就按顺序读。

文档里有七张插图，全部由配套程序的同一套数据算出，可以用 `python figs.py` 重新生成。插图分别是：控制体几何、两个场的剖面演化、坐标变换的几何意义、切比雪夫节点的来源、干燥曲线与终点放大、通量形式的谱配置、两种离散方式的收敛对比。

文档里出现的每一个数值都来自配套代码的实际运行结果，原始输出保存在代码/复跑报告.txt。如果某个数字与你自己运行的结果不一致，请以你的运行为准，并检查是否改

动过参数。

七个关键答案一览。下面这张表把全题的关键答案集中列出。第三列是保留四位小数之前的完整数值，第四列是按题目要求保留四位小数的结果，第五列说明这个数在正文哪一部分被推导出来。

题目	量	完整数值	四位小数	推导出处
问题 1	中心温度 $T(0, 1800)$ / °C	34.0424989153	34.0425	第六部分
问题 1	表面温度 $T(R_0, 1800)$ / °C	37.1989219992	37.1989	第六部分
问题 1	表面含水 $C(R_0, 1800)$ / (kg/kg)	1.51091810	1.5109	第二部分
问题 2	中心温度 $T(0, 3\text{ h})$ / °C	49.905089	49.9051	第二部分
问题 2	中心含水 $C(0, 3\text{ h})$ / (kg/kg)	1.767275	1.7673	第二部分
问题 2	表面含水 $C(R_0, 3\text{ h})$ / (kg/kg)	1.008288	1.0083	第二部分
问题 3	烘干时长 / s	205574.519	205574.5	第二部分
问题 4	烘干时长 / s	182778.281	182778.3	第二部分

问题 3 的 205574.5 秒等于 57.1040 小时，问题 4 的 182778.3 秒等于 50.7717 小时。问题 4 的答案依赖于附件 2 给出的半径收缩数据，收缩使扩散长度平方降到 $(R/R_0)^2 = 0.36$ ，这是它比问题 3 明显更快的主要原因。

目录

I	数学与物理基础	12
1	数学预备知识	12
1.1	一个量可以同时依赖两个变量	12
1.2	偏导数：把一个变量当常数	12
1.3	二阶偏导数	13
1.4	圆柱坐标与那一项多出来的东西	13
1.5	微分方程、阶、线性与非线性	15
1.6	初值条件与边界条件	15
1.7	三类边界条件	15
2	从守恒律推出控制方程	16
2.1	微元法的总思路	16
2.2	傅里叶定律	16
2.3	圆柱薄壳的几何量	17
2.4	能量守恒，一步不跳	18
2.5	从差分到微分	18
2.6	菲克定律与水分守恒	20
2.7	干基含水率是什么	20
2.8	干物质不迁移这个事实	21
2.9	水分守恒方程	21
2.10	无量纲分析：先判断哪边快	22
3	边界条件与初始条件	24
3.1	圆心处的条件：对称性	24
3.2	表面处的条件：牛顿冷却定律	24
3.3	初始条件	25
3.4	烘房环境数据的处理	25
3.5	最小二乘拟合是怎么做的	26

3.6	拟合结果	27
3.7	工艺分段	28
3.8	第一部分小结	29
II	数值方法原理	29
4	为什么必须用数值方法	30
4.1	障碍一：扩散系数随含水率变化	30
4.2	障碍二：物性同时依赖两个未知量	30
4.3	障碍三：边界条件是耦合的	31
4.4	障碍四：求解区域会变形	31
5	坐标变换：把麻烦的 $1/r$ 去掉	31
5.1	麻烦在哪里	31
5.2	换元：用 u 代替 r	32
5.3	第一步：求出两个导数之间的换算关系	33
5.4	第二步：逐项改造算子	33
5.5	第三步：检查圆心处的表现	34
5.6	第四步：把方程改写成新坐标下的样子	35
5.7	第五步：边界条件也要改写	35
5.8	第六步：圆心处还需要加条件吗	36
6	谱方法：用很少的点算得很准	37
6.1	基本想法：用多项式代替未知函数	37
6.2	节点怎么选：一个反面教材	37
6.3	切比雪夫节点	37
6.4	插值多项式的唯一性	39
6.5	拉格朗日插值公式	39
6.6	从插值多项式得到导数	39
6.7	微分矩阵的元素公式	40
6.8	记号的换算	41

6.9	一个三节点的手算例子	41
6.10	为什么谱方法精度高	45
7	时间推进：从当前时刻推到下一时刻	45
7.1	半离散化	46
7.2	最简单的方法：显式欧拉法	46
7.3	显式欧拉法的问题：稳定性	46
7.4	隐式方法	47
7.5	更高阶的方法：BDF	48
7.6	自适应步长	48
7.7	一个必须注意的细节：系数用什么时刻的值	49
8	烘干时间怎么确定	50
8.1	判据的数学表述	50
8.2	哪一点最后干	51
8.3	怎么求这个时刻	51
8.4	第二部分小结	52
9	问题四的收缩怎么处理	53
9.1	第一种想法为什么是错的	53
9.2	用一个极端例子把这个错误暴露出来	53
9.3	正确的做法：跟着干物质走	54
9.4	干物质守恒给出的一个重要关系	55
9.5	水分守恒的完整推导	56
9.6	从积分式得到微分方程	57
9.7	换成 u 坐标	58
9.8	一个重要的结论：对流项为零	59
9.9	输出表格时的坐标换算	60
III	代码逐行解释	61
10	代码总览	61

10.1 文件分工	61
10.2 为什么把参数单独放一个文件	63
10.3 数据是怎样从附件走到论文表格的	64
11 主求解器 spectral.py 逐段解释	64
11.1 文件头部与导入	64
11.2 函数 cheb_matrices: 构造微分矩阵	65
11.3 类 HerbSolver 的初始化	68
11.4 几何与环境: 三个小方法	69
11.5 右端项函数: 核心中的核心	70
11.6 初值与求解	74
11.7 采样之一: 物理半径节点与重心插值	75
11.8 采样之二: 面积平均权重	78
12 参数文件 common.py 逐段解释	80
12.1 文件定位: 让程序可以搬动	80
12.2 烘房环境的标定	81
12.3 两种 RMSE 口径	83
12.4 物性公式与浓度下限	84
12.5 附件 2 与半径插值	85
12.6 Bessel 特征根	86
13 闭式解文件 analytic_p1.py 逐段解释	87
13.1 它解决什么问题	87
13.2 准静态分解的思路	88
13.3 代码实现: T_exact	89
13.4 自检函数: 让解析解自己证明自己	91
14 驱动脚本逐段解释	93
14.1 run_all.py: 一键复跑	93
14.2 sensitivity.py: 参数扰动与环境口径	94
14.3 verify_data.py: 四类扫描与一致性校验	97

14.4	console_utf8.py: 一个三十行的小文件	98
14.5	这一部分与第五部分的分工	99
IV	如何确认结果正确	99
15	为什么必须验证	100
15.1	程序不会告诉你它算错了	100
15.2	验证的层次	100
16	第一层验证: 与精确解比较	101
16.1	精确解从哪里来	101
16.2	比较的方法	101
16.3	怎么读这张表	101
16.4	这个 10 的负 10 次方意味着什么	102
17	第二层验证: 网格加密看收敛	102
17.1	收敛是什么意思	102
17.2	空间方向的加密	103
17.3	怎么读这张表	103
17.4	时间方向的加密	104
18	第三层验证: 多种方法互证	105
18.1	为什么要用不同方法	105
18.2	比较结果	105
18.3	怎么读这张表	105
18.4	一次特别的比较	106
19	灵敏度分析结果怎么读	107
19.1	做了什么事	107
19.2	结果	107
19.3	三个要读懂的地方	107
19.4	一个反直觉的结论	108

20 常见疑问解答	108
20.1 关于建模	108
20.2 关于数值方法	109
20.3 关于结果	109
20.4 关于写作	110
21 全文回顾	110
V 其余程序逐行解释	111
22 对照求解器之一：节点中心有限体积 fv_theta.py	111
22.1 这个文件的两个用途	111
22.2 有限体积法的三个概念与网格记号	111
22.3 体积测度的完整推导	112
22.4 面通量与离散算子	113
22.5 清单：时间推进与 Rannacher 启动	116
22.6 冻结系数为什么使时间方向只有一阶精度	120
22.7 find_tdry：线性求交	121
22.8 时间离散口径的三处不同	123
23 对照求解器之二：均匀网格守恒型有限体积 fv_uniform.py	124
23.1 与主求解器的三处刻意差别	124
23.2 网格、测度与输出索引	124
23.3 后向欧拉离散：逐个节点写出方程	126
23.4 Picard 迭代为什么能处理非线性	127
23.5 追赶法的完整推导	127
23.6 追赶法为什么不需要选主元	133
23.7 顺带看清误差是怎么往回传的	136
23.8 采样与报告	136
24 结果文件：make_results.py 逐段解释	137
24.1 题目要求的表格与文件格式	137

24.2 写表函数逐行说明	138
24.3 采样函数与坐标换算	140
24.4 result1 与 result2	141
24.5 烘干时间与事件函数	142
24.6 result3 与 result4	143
24.7 收尾统计与回读校验	145
25 插图体系: fig_core.py 与 make_figures.py	147
25.1 为什么要分成两个文件	147
25.2 中文字体与出图分辨率	147
25.3 解缓存与源码指纹	149
25.4 画图前的强制对账	151
25.5 25 张图画的是什么	152
25.6 采样细节: 域外的点必须留空	153
25.7 求交而不是取点	154
26 受控对照: make_controlled_comparison.py	155
26.1 这个脚本要解决什么问题	155
26.2 受控算例是怎么算的	155
26.3 两档网格的自查	157
27 整条产出链	158
VI 问题一闭式解的完整推导	160
28 这一部分要解决什么	160
29 第一步: 把边界条件化成齐次	160
30 第二步: 把解拆成两部分	163
31 第三步: 解出准静态部分 W	165
31.1 把方程化成标准形式	165

31.2 把拉普拉斯算子写开	165
31.3 认识贝塞尔方程	166
31.4 写出通解, 去掉发散的那一项	166
31.5 用表面条件定出 B	166
31.6 代入数值	167
32 第四步: 解瞬态部分 $\hat{w}$ 的特征值问题	168
32.1 分离变量	168
32.2 解时间部分	168
32.3 解空间部分	169
32.4 用表面条件定出 β	170
33 第五步: 正交性与展开系数	171
33.1 把 $\hat{w}$ 写成级数	171
33.2 内积与权	171
33.3 证明正交性	172
33.4 模长	173
33.5 展开系数	173
34 第六步: 把三部分拼起来, 并做三个自检	175
34.1 最终的公式	175
34.2 自检一: 初始时刻	175
34.3 自检二: 无穷时刻	176
34.4 自检三: 表面热流	176
34.5 数值结果	176
35 第七步: 级数收敛有多快	176
35.1 衰减因子决定的收敛速度	176
35.2 取几项就够	177
36 第八步: 这些公式在代码里长什么样	177
36.1 导入时算好的量	177
36.2 函数里的求和	178

36.3 特征方程怎么求根	179
36.4 自检	181
37 第九步：用这个闭式解检验数值解	181
38 第十步：用这个闭式解做的敏感性检验	182
38.1 一个必须回答的问题	182
38.2 怎么读这张表	183
39 这一部分的小结	183

Part I

数学与物理基础

这一部分从最简单的概念开始，一直推到两个控制方程。如果你已经会偏导数，可以跳过第一章，直接从第二章读起。

1 数学预备知识

1.1 一个量可以同时依赖两个变量

先说一个日常例子。房间里某一点的温度，会随着时间变化，也会随着位置变化。房间中央和靠窗的位置温度不一样，同一个位置早上和中午也不一样。

为了把这件事写清楚，我们用两个符号：

- t 表示时间，单位是秒，符号 s。
- r 表示某个位置到中心的距离，单位是米，符号 m。

于是温度就写成一个函数

$$T = T(r, t). \quad (1)$$

这个式子的读法是：温度 T 由两个自变量 r 和 t 共同决定。括号里先写位置，后写时间，这个顺序一旦定下来就不能改。

同样地，水分浓度也是两个自变量的函数，写作

$$C = C(r, t). \quad (2)$$

水分浓度的单位是千克每千克，符号 kg/kg，它的确切含义我们在第二章讲。

1.2 偏导数：把一个变量当常数

现在的问题是：温度变化有多快？

如果只问时间方向的变化快慢，那就把位置 r 固定住不动，只看 t 变化时 T 怎么变。这种在一个变量变化、其余变量固定的情况下求出的变化率，叫做偏导数，写作

$$\frac{\partial T}{\partial t}. \quad (3)$$

符号 ∂ 读作偏，它和普通导数符号 d 的区别只有一点：用 ∂ 的时候，暗示还有别的自变量存在，只是暂时被固定住了。

计算规则很简单：把另一个变量真的当成一个普通常数去求导就行。举例说明。

假设有一个函数

$$f(x, y) = 3x^2y + 5y + 7. \quad (4)$$

要求 $\frac{\partial f}{\partial x}$ ，就把 y 当成常数。此时 $3y$ 是常数， $5y + 7$ 也是常数，常数求导为零，于是

$$\frac{\partial f}{\partial x} = 3y \cdot 2x = 6xy. \quad (5)$$

要求 $\frac{\partial f}{\partial y}$ ，就把 x 当成常数，于是 $3x^2$ 是常数，

$$\frac{\partial f}{\partial y} = 3x^2 \cdot 1 + 5 = 3x^2 + 5. \quad (6)$$

回到温度。 $\frac{\partial T}{\partial t}$ 的含义是：站在药材的某一个固定位置上，看温度随时间涨得多快，单位是摄氏度每秒。

$\frac{\partial T}{\partial r}$ 的含义是：在某个固定时刻，沿着半径方向走一小段，看温度变化多少，单位是摄氏度每米。这个量叫温度梯度，它是热量流动的直接原因。

1.3 二阶偏导数

偏导数本身还是一个函数，所以可以再对它求一次偏导数。对同一个变量求两次，得到

$$\frac{\partial^2 T}{\partial r^2} = \frac{\partial}{\partial r} \left(\frac{\partial T}{\partial r} \right). \quad (7)$$

这个量的物理含义是温度梯度沿半径方向的变化快慢。如果 $\frac{\partial^2 T}{\partial r^2}$ 大于零，说明温度曲线是向上弯的，梯度在往外走的过程中越来越大。

1.4 圆柱坐标与那一项多出来的东西

平面上，一个点到原点的距离记作 r ，它和直角坐标 x 、 y 的关系是

$$r = \sqrt{x^2 + y^2}. \quad (8)$$

药材是圆柱形的，它的截面是一个圆，温度只和到圆心的距离 r 有关，和转过的角度无关，这种情形叫做轴对称。

在直角坐标里，温度对位置的全部二阶变化写成一个算子，叫拉普拉斯算子，

$$\nabla^2 T = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}. \quad (9)$$

换到圆柱坐标以后，这个算子变成

$$\nabla^2 T = \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right). \quad (10)$$

这个式子里的 $1/r$ 和那个多出来的 r 是从哪里来的，值得说清楚，因为它正是后面第五章要大动干戈的原因。

推导过程如下。圆柱坐标下，面积微元是

$$dA = r dr d\theta. \quad (11)$$

径向的热流密度要乘以侧面积 $2\pi r L$ 才是总热流，所以算式里带一个 r 。这里的 $2\pi r dr$ 是单位长度下的体积微元，不是面积。侧面积是 $2\pi r L$ ，体积是 $2\pi r L dr$ 。把通量沿径向的净变化除以体积微元时，因子 $2\pi L$ 和 dr 都能约掉，但是导数号里面的 r 不能提到导数号外面去，所以分式前面留下一个 $1/r$ 。这个 $1/r$ 就是式 (10) 里多出来的那一项。第二章会把这个过程完整写出来。

把式 (10) 展开，得到另一种写法：

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right) = \frac{1}{r} \left[\frac{\partial T}{\partial r} + r \frac{\partial^2 T}{\partial r^2} \right] = \frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r}. \quad (12)$$

中间那一步用到了乘积求导法则。设 $u = r$ ， $v = \frac{\partial T}{\partial r}$ ，则 $(uv)' = u'v + uv'$ ，也就是

$$\frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right) = 1 \cdot \frac{\partial T}{\partial r} + r \cdot \frac{\partial^2 T}{\partial r^2}. \quad (13)$$

再除以 r 就得到式 (12)。

这一节要记住的 式 (12) 里出现了 $1/r$ 。当 r 趋向于零的时候，这一项看起来会变成无穷大。这是圆柱坐标带来的麻烦，第五章会用一个巧妙的换元把它消掉。

1.5 微分方程、阶、线性与非线性

一个方程里如果含有未知函数的导数，就叫微分方程。我们要求解的 T 和 C 都是未知函数，所以下面出现的都是微分方程。

按最高阶导数的阶数命名。只出现一阶导数叫一阶方程，出现二阶导数叫二阶方程。本题的两个方程都是二阶的。

按未知函数和它的导数是否以一次方的形式出现，分为线性与非线性。

如果方程里的系数只依赖自变量，比如 r 和 t ，那么方程是线性的。线性方程有一个很好的性质：两个解加起来还是解。如果系数依赖未知函数本身，比如导热系数依赖温度，那就成了非线性方程，线性方程的许多便利就不能用了。

本题中，问题一的所有物性都是常数，所以方程是线性的，而且是两个互不影响的线性方程。问题二到问题四的物性依赖 C 和 T ，方程变成非线性，两个方程还互相耦合，必须一起求解。

1.6 初值条件与边界条件

一个二阶微分方程要有唯一确定的解，需要补充条件。

对时间方向，因为方程里出现 $\frac{\partial T}{\partial t}$ ，需要一个起始时刻的状态，叫做初值条件，也叫初始条件：

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}, \quad 0 \leq r \leq R_0. \quad (14)$$

读法是：在时间等于零的时刻，从圆心到表面每一个位置的温度和水分浓度都已知。题目给出的是这两组数。

对空间方向，因为方程里出现 $\frac{\partial^2 T}{\partial r^2}$ ，需要在空间区域的两个端点各给一个条件。本题的空间区域是从圆心 $r = 0$ 到表面 $r = R_0$ ，所以要在 $r = 0$ 和 $r = R_0$ 各给一个条件，这叫边界条件。

1.7 三类边界条件

边界条件通常分三类，理解这三类对读懂本题很关键。

第一类，给定边界上的值。例如直接规定表面温度恒为 80 摄氏度，写作 $T(R_0, t) = 80$ 。这类条件叫狄利克雷条件。

第二类，给定边界上的通量。例如规定表面每平方米每秒流入多少焦耳的热量，写作 $-k \frac{\partial T}{\partial r} \Big|_{r=R_0} = q_0$ 。这里出现的是导数而不是函数值。这类条件叫诺伊曼条件。

第三类，给定边界值与外界的关系。例如表面通过空气对流换热，换热快慢取决于表面和空气的温差，写作

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h(T(R_0, t) - T_{\text{air}}(t)). \quad (15)$$

这类条件叫罗宾条件，本题的表面条件正是这一种，因为热风烘干是靠空气流动把热量带到药材表面的。

这一节要记住的 初值条件管时间方向的起点，边界条件管空间方向的两端。两类条件都齐全，方程的解才唯一。

2 从守恒律推出控制方程

这一章是整份文档的核心。我们从一个薄圆筒壳出发，用中学物理里的能量守恒和质量守恒，一步步推出两个控制方程。每一步的代数运算都会写全。

2.1 微元法的总思路

直接对整个圆柱列方程很难，因为圆柱内部各处的温度不一样。办法是取一个很小的区域，让这个小区域内部可以近似认为温度是均匀的，对这个小区域列方程，然后再让它缩小到一点，得到只描述该点的方程。

这个小区域叫微元，也叫控制体。取法有很多种，本题取一个薄圆筒壳，因为它正好配合圆柱的轴对称性质。

2.2 傅里叶定律

热量总是从高温处流向低温处，流动的快慢与温度梯度成正比。这个经验规律叫傅里叶定律，一维形式写作

$$q_r = -k \frac{\partial T}{\partial r}. \quad (16)$$

其中 q_r 是沿半径方向的热流密度，单位是瓦特每平方米，符号 W/m^2 ，它表示单位时间穿过单位面积的热量。 k 是导热系数，单位是瓦特每米每开尔文，符号 $\text{W}/(\text{m} \cdot \text{K})$ ，它衡量材料导热能力的强弱。

式 (16) 里的负号表示方向：如果温度沿半径向外降低，也就是 $\frac{\partial T}{\partial r} < 0$ ，那么 $q_r > 0$ ，热量朝外流，方向和温度降低的方向一致。如果温度沿半径向外升高，热量就朝内流。

2.3 圆柱薄壳的几何量

现在取微元。半径从 r 到 $r + dr$ 之间的一层薄壳，高度取药材的长度 L 。

这个薄壳的体积要算清楚，因为后面要用它表示内能。

把薄壳看成一张卷起来的纸，展开以后它的长度是圆周长 $2\pi r$ ，高度是 L ，厚度是 dr 。所以体积是

$$dV = 2\pi r L dr. \quad (17)$$

这里出现的 $2\pi r$ 就是圆周长，那个 r 来自周长公式。请特别记住这一点，它是圆柱坐标里所有 r 因子的来源。

薄壳的质量等于体积乘密度，

$$dm = \rho dV = \rho 2\pi r L dr, \quad (18)$$

其中 ρ 是密度，单位是千克每立方米，符号 kg/m^3 。

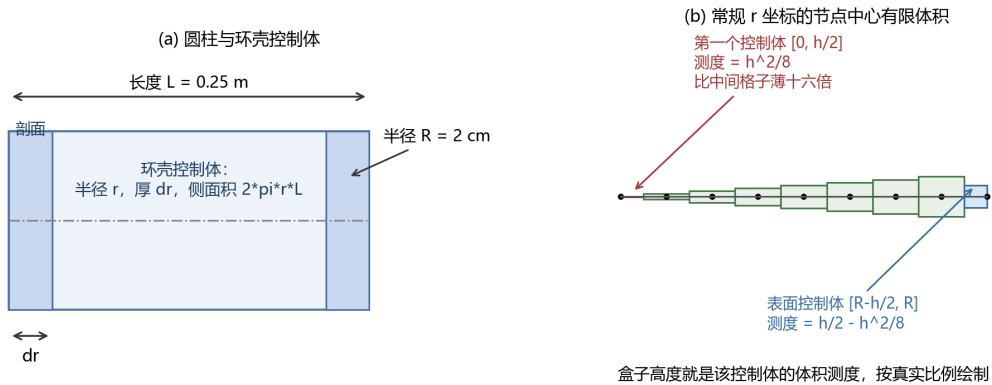


图 1：圆柱环壳控制体与两种径向离散的对照。(a) 取半径 r 到 $r+dr$ 的一层薄壳作控制体，侧面积是 $2\pi r L$ ；(b) 常规 r 坐标下的节点中心有限体积，第一个控制体只有半格宽，其体积测度是 h 平方除以 8。

2.4 能量守恒，一步不跳

控制体是半径 r 到 $r + dr$ 、高度 L 的环壳。体积是 $2\pi r L dr$ ，内能的变化率等于净流入的热量速率。

对这个薄壳列能量守恒。表述是：

$$\text{薄壳内能的增加速率} = \text{单位时间净流入薄壳的热量}. \quad (19)$$

先算左边。内能可以用比热容表示。比热容 c_p 的含义是：让一千克物质升高一摄氏度需要吸收多少焦耳的热量，单位是焦耳每千克每开尔文，符号 $\text{J}/(\text{kg} \cdot \text{K})$ 。左边等于 ρc_p 乘以体积再乘以温度的时间导数：

$$\text{左边} = \rho c_p 2\pi r L dr \frac{\partial T}{\partial t}. \quad (20)$$

再算右边。规定 $q_r(r, t)$ 是沿正的 r 方向的热流密度，单位是瓦特每平方米。

在半径 r 的柱面上，这个面是控制体的内表面。热流沿正 r 方向穿过它，就是从外面进入控制体，所以单位时间从内表面进入的热量是正的 $q_r(r)$ 乘以面积 $2\pi r L$ 。

在半径 $r + dr$ 的柱面上，热流沿正 r 方向穿过它，是从控制体流出去，所以单位时间从外表面流出的热量是 $q_r(r + dr)$ 乘以面积 $2\pi(r + dr)L$ 。

净流入等于流入减去流出：

$$\text{净流入} = q_r(r) 2\pi r L - q_r(r + dr) 2\pi(r + dr)L. \quad (21)$$

记 $G(r) = r q_r(r)$ ，则上式等于 $-2\pi L [G(r + dr) - G(r)]$ 。方括号里的量是一阶差分。

2.5 从差分到微分

把 G 在 r 处做泰勒展开：

$$G(r + dr) = G(r) + \frac{\partial G}{\partial r} dr + \text{高阶项}. \quad (22)$$

所以净流入等于 $-2\pi L \frac{\partial G}{\partial r} dr$ 再加上高阶项。这里用到的事实是：导数就是差商的极限，

$$\lim_{dr \rightarrow 0} \frac{G(r + dr) - G(r)}{dr} = \frac{\partial G}{\partial r}. \quad (23)$$

让 dr 趋于零，高阶项消失，得到

$$\text{净流入} = -2\pi L \frac{\partial}{\partial r} (r q_r) dr. \quad (24)$$

左边是内能变化率，也就是式 (20) 给出的量。两边相等，再约去共同的 $2\pi L dr$ ，得到

$$\rho c_p r \frac{\partial T}{\partial t} = -\frac{\partial}{\partial r} (r q_r). \quad (25)$$

到这里负号还在。代入傅里叶定律 (16)，也就是 $q_r = -k \frac{\partial T}{\partial r}$ ：

$$-\frac{\partial}{\partial r} \left(r \left(-k \frac{\partial T}{\partial r} \right) \right) = \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right). \quad (26)$$

这一步用到的是负号乘负号得正号。于是

$$\rho c_p r \frac{\partial T}{\partial t} = \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right). \quad (27)$$

两边同除以 r ，就得到标准的圆柱非稳态导热方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right). \quad (28)$$

最后把系数的自变量写明，写成标准形式

$$\boxed{\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right).} \quad (29)$$

这就是第一个控制方程。

我们把系数写成 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 是为了提醒：在问题二到问题四里，它们都随水分浓度 C 变化，不是常数。在问题一里它们都是常数，方程是线性的。

为什么要这样做：这里关于 r 的运算，哪些可以做，哪些不可以做

可以做的：两边同除以 r ，因为 r 大于零，除完以后左边变成 $\rho c_p \frac{\partial T}{\partial t}$ 。不可以做的：把导数号里面的 r 提到导数号外面，

$$\frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \neq r \frac{\partial}{\partial r} \left(k \frac{\partial T}{\partial r} \right). \quad (30)$$

原因是这个 r 随位置变化，不是常数因子，它要按乘积求导法则参与求导。左边的 r 来自体积公式里的圆周长，右边的 r 在导数符号里面，两者形式相同，来源不同。

2.6 菲克定律与水分守恒

水分的迁移和热量类似，也是由浓度差驱动，规律叫菲克定律。按干基含水率写出来的通量形式是

$$j_r = -\rho_d D \frac{\partial C}{\partial r}, \quad (31)$$

其中 D 是水分扩散系数，单位是平方米每秒，符号 m^2/s ， ρ_d 是单位湿体积内的干物质质量，下一小节会给出它的定义。 j_r 是相对于干物质骨架的水分通量，单位是千克每平方米每秒，也就是单位时间穿过单位面积的水的质量。这里要带上 ρ_d ，原因是干基含水率 C 是一个比值，单位是千克水每千克干物质，它本身不带质量密度的信息。负号的含义和傅里叶定律一样：水分从浓度高处流向浓度低处。

但是这里有一个容易出错的地方。水分是存在药材内部的，药材本身在干燥过程中还会收缩，所以不能简单地把水分通量除以面积就完事。必须盯住干物质，因为干物质是不迁移的。

正确的做法是，用干基含水率 C 作为未知量，并且注意到控制体里的干物质质量不随时间变化。第二章下一小节详细说这件事。

2.7 干基含水率是什么

日常生活中说含水率，通常指湿基含水率，定义是

$$\text{湿基含水率} = \frac{\text{水的质量}}{\text{水的质量} + \text{干物质的质量}}. \quad (32)$$

题目用的是干基含水率，定义是

$$C = \frac{\text{水的质量}}{\text{干物质的质量}}. \quad (33)$$

例如题目给初始值 $C_0 = 2.55 \text{ kg/kg}$ ，意思是每 1 千克干物质里含有 2.55 千克水。

为什么要这样做：为什么用干基而不是湿基

因为干燥过程中水分不断减少，用湿基的话分母一直在变，写守恒方程时会出现额外的项。用干基的话，分母是干物质质量，而干物质不迁移也不增减，是一个固定的参考量。这样写出来的方程最干净，处理收缩问题也最方便。这是一个非常重要但常被忽略的建模技巧。

2.8 干物质不迁移这个事实

现在把控制体从按空间位置取，改成按干物质取。具体说，我们跟踪同一批干物质，看它内部的水分怎么变。

这样取控制体的好处是：控制体自身的质量不变，水分的进出只能通过控制体的两个侧面，不会因为控制体自己收缩而凭空多出或少掉水。

单位湿体积内的干物质质量记作

$$\rho_d = \frac{\rho}{1+C}. \quad (34)$$

这个式子怎么来的？体积内总质量是 ρ ，其中干物质占的比例是 $\frac{1}{1+C}$ 。因为按式 (33)，每 1 千克干物质对应 C 千克水，所以总质量是干物质的 $1+C$ 倍。

2.9 水分守恒方程

以干基含水率 C 为变量，它的定义是每千克干物质里含有多少千克水。在半径 r 到 $r+dr$ 的环壳里，干物质的质量是

$$dm_d = \rho_d 2\pi r L dr, \quad (35)$$

其中 ρ_d 是干物质在湿体积中的密度，它的定义在上一个小节里给出。干物质不迁移，所以 dm_d 不随时间变化。壳层里的水质量等于 C 乘以 dm_d 。

规定 j_r 是相对于干物质骨架的水分通量，单位是每秒每平方米通过多少千克水。菲克定律给出

$$j_r = -\rho_d D \frac{\partial C}{\partial r}. \quad (36)$$

穿越半径 r 的柱面进入壳层的水分速率是 $j_r(r) 2\pi r L$ ，穿越半径 $r+dr$ 的柱面流出的速率是 $j_r(r+dr) 2\pi(r+dr)L$ 。与热量的情形完全相同，净流入等于流入减去流出，记 $H(r) = r j_r(r)$ ，再做一次泰勒展开并取极限，得到

$$\text{净流入} = -2\pi L \frac{\partial}{\partial r} (r j_r) dr. \quad (37)$$

左边是水质量的变化率。因为 dm_d 是常数，水的质量 $C dm_d$ 的变化率等于 $dm_d \frac{\partial C}{\partial t}$ ：

$$\rho_d 2\pi r L dr \frac{\partial C}{\partial t} = -2\pi L \frac{\partial}{\partial r} (r j_r) dr. \quad (38)$$

约去 $2\pi L dr$:

$$\rho_d r \frac{\partial C}{\partial t} = -\frac{\partial}{\partial r} (r j_r). \quad (39)$$

代入菲克定律 (36), 也就是 $j_r = -\rho_d D \frac{\partial C}{\partial r}$:

$$\rho_d r \frac{\partial C}{\partial t} = \frac{\partial}{\partial r} \left(r \rho_d D \frac{\partial C}{\partial r} \right). \quad (40)$$

两边同除以 ρ_d 并同除以 r , 得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (41)$$

最后这一步把 ρ_d 从导数号里面提出来约掉, 用到了干物质密度在同一个壳层内不随位置变化这个常用近似。题目把 D 定义为以干物质为基准的有效扩散系数, 本文与题目保持同一口径。

把系数的自变量写明, 写成标准形式

$$\boxed{\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right)}. \quad (42)$$

这就是第二个控制方程。

为什么要这样做: 两个方程的左边为什么不同

传热方程的左边是 $\rho c_p \frac{\partial T}{\partial t}$, 水分方程的左边只有 $\frac{\partial C}{\partial t}$ 。原因在于未知量的选取方式不同。温度是单位质量的能量指标, 要乘上密度和比热容才变成单位体积的能量。而 C 已经是每千克干物质的水量, 本身就是一个比值, 控制体又跟着干物质走, 所以左边干净。这两点如果搞混, 方程就会写错。

2.10 无量纲分析: 先判断哪边快

在动手算之前, 先做一件事: 估算两个过程的快慢, 看哪个先完成。

定义一个时间尺度, 表示某个过程走完全程大概需要多久。传热的特征时间取

$$\tau_T = \frac{R_0^2}{\alpha}, \quad \alpha = \frac{k}{\rho c_p}, \quad (43)$$

其中 α 叫热扩散率, 单位是平方米每秒。传质的特征时间取

$$\tau_D = \frac{R_0^2}{D}. \quad (44)$$

代入题目数据，问题一里

$$\alpha = \frac{0.36}{820 \times 2600} = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}, \quad (45)$$

$$\tau_T = \frac{0.02^2}{1.6886 \times 10^{-7}} = 2369 \text{ s}, \quad (46)$$

而水分扩散系数在 $C = 2.55$ 时等于

$$D = 7 \times 10^{-9} \times e^{-0.89/2.55} = 7 \times 10^{-9} \times 0.7053 = 4.9377 \times 10^{-9} \text{ m}^2/\text{s}, \quad (47)$$

$$\tau_D = \frac{0.02^2}{4.9377 \times 10^{-9}} = 8.10 \times 10^4 \text{ s}. \quad (48)$$

两者相差三十多倍。结论很清楚：温度很快就能平衡，水分则慢得多。这个判断在第四部分解释结果时会反复用到。

这一节要记住的 传热特征时间约 2369 秒，传质特征时间约 81000 秒。所以题目里说的前 30 分钟，也就是 1800 秒，温度明显在上升，表面含水率也已经在下降：1800 秒时表面含水率由 2.55 降到 1.5109，降了约百分之四十一。只有中心几乎没动。

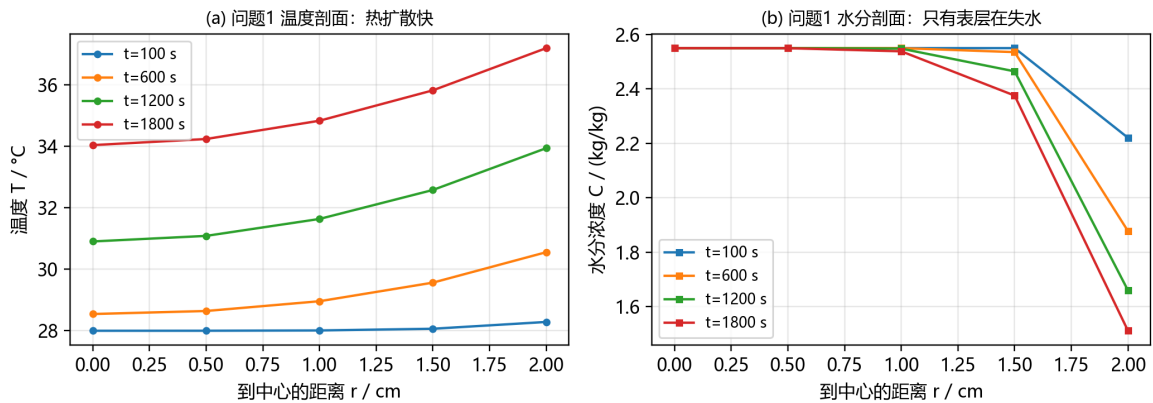


图 2：问题 1 的温度剖面与水分剖面演化。(a) 温度在 30 分钟内已经拉开明显梯度；(b) 水分在同样时间内几乎只动表层。两张图的差别来自热扩散率与水分扩散系数的量级差别。

3 边界条件与初始条件

方程本身只描述了内部各点之间的相互关系，还需要补充端点与起始时刻的信息，才能得到唯一的解。这一章把四个条件一个一个推导出来。

3.1 圆心处的条件：对称性

在圆心 $r = 0$ 处，温度不可能是无穷大，也不能有尖角。这件事要用轴对称的定义说清楚。

轴对称的意思是：把整个圆柱绕轴旋转任意角度，温度分布不变。也就是说，温度其实是横坐标 x 的光滑函数，其中 x 是到轴的有符号距离，轴的一侧取正，另一侧取负。中心线上 $x = 0$ ，轴两侧对称的两个位置是 x 和 $-x$ ，它们的温度必须相同，所以这个函数是 x 的偶函数。

偶函数在 $x = 0$ 处的导数为零，也就是

$$\left. \frac{dT}{dx} \right|_{x=0} = 0. \quad (49)$$

在 $r \geq 0$ 的半轴上， r 就是 x 取正的那一半，两者在 $x = 0$ 附近的导数相同，所以在 $r = 0$ 处温度对 r 的一阶导数为零：

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (50)$$

为什么要这样做：这个条件为什么不能省

如果不用它，数学上会出现无穷多个解，其中大多数在物理上荒谬，比如温度在圆心附近无限升高。另外，它还保证了第一章里提到的 $1/r$ 那一项在原点不会真的发散，因为分子 $\frac{\partial T}{\partial r}$ 同时趋向于零。

3.2 表面处的条件：牛顿冷却定律

药材表面与热空气接触，热量通过空气流动带过来。这个过程叫对流换热。牛顿冷却定律说，单位面积上单位时间传入的热量与温差成正比，

$$q_{\text{传入}} = h(T_{\text{air}}(t) - T(R_0, t)), \quad (51)$$

其中 h 叫对流换热系数，单位是瓦特每平方米每开尔文，符号 $\text{W}/(\text{m}^2 \cdot \text{K})$ 。题目给出 $h = 25$ 。

把式 (51) 转化成我们方程里的量。朝外为正的热流密度是 $q_r = -k \frac{\partial T}{\partial r}$ 。朝内传入的热量等于朝外热流的负值，所以

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h (T(R_0, t) - T_{\text{air}}(t)). \quad (52)$$

左边是导热给出的通量，右边是对流给出的通量，两者必须相等，否则表面处的能量就不守恒。

同样的推理用在水分上。水分由空气带走，传质系数记作 h_m ，单位是米每秒，符号 m/s，题目给出 $h_m = 8 \times 10^{-7}$ 。

朝外为正的水分通量是 $-D \frac{\partial C}{\partial r}$ ，于是

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R_0} = h_m (C(R_0, t) - C_{\text{air}}(t)). \quad (53)$$

为什么要这样做：为什么左右两边的单位能对上

以温度为例。左边是导热系数乘以温度梯度，单位是 $\text{W}/(\text{m} \cdot \text{K}) \times \text{K}/\text{m} = \text{W}/\text{m}^2$ 。右边是换热系数乘以温差，单位是 $\text{W}/(\text{m}^2 \cdot \text{K}) \times \text{K} = \text{W}/\text{m}^2$ 。两边都是热流密度，单位一致。这是一次必要的量纲检查，如果对不上就说明公式写错了。

3.3 初始条件

题目给出烘干开始时药材温度 28 摄氏度，干基含水率 2.55 千克每千克，且药材内部各处都相同。于是

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}, \quad 0 \leq r \leq R_0. \quad (54)$$

这里的 $0 \leq r \leq R_0$ 表示整个径向范围都要满足，不是只在表面或者只在圆心。

3.4 烘房环境数据的处理

边界条件 (52) 和 (53) 里出现了两个量，烘房温度 $T_{\text{air}}(t)$ 和烘房水分浓度 $C_{\text{air}}(t)$ 。题目没有给出公式，而是给了附件一，里面是一张表：

时间 / s	温度 / °C	水分浓度 / (kg/kg)
0	28.000	0.01963
60	28.528	0.02002
120	29.192	0.02053
⋮	⋮	⋮
14400	50.165	0.04986

一共 241 行，每 60 秒一行，时间从 0 到 14400 秒。

问题来了：方程里的 $T_{\text{air}}(t)$ 需要任意时刻的值，而表格只给了 60 秒一个点。中间那些时刻怎么办？

有两种做法。

第一种做法是直接插值。两个相邻数据点之间连一条直线，任意时刻的值取直线上的对应点。这种做法最简单，但把测量噪声也带进来了。

第二种做法是先识别数据的规律，用一条光滑曲线去代表它，再用这条曲线作为边界条件。这种做法叫数据拟合。

从表里能看出明显的规律：温度从一个较低的值开始，快速上升，然后逐渐变慢，最后稳定在一个固定值附近。这种形状叫一阶惯性上升。它的数学形式是

$$T_{\text{air}}(t) = T_{\text{set}} - (T_{\text{set}} - 28) e^{-t/\tau_T}. \quad (55)$$

这个式子有三个参数，逐个解释。

T_{set} 是最终稳定的温度。当 t 很大时， e^{-t/τ_T} 趋向于零，式子只剩下 T_{set} ，所以它代表终值。

28 是初始温度，直接取表格第一行的实测值，不参与拟合。

τ_T 叫时间常数，它控制上升的快慢，单位是秒。当 $t = \tau_T$ 时，指数项等于 $e^{-1} = 0.368$ ，也就是说温度已经走完了全程的 63.2%。

含湿量用同样的形式，

$$C_{\text{air}}(t) = C_{\text{set}} - (C_{\text{set}} - 0.01963) e^{-t/\tau_C}. \quad (56)$$

3.5 最小二乘拟合是怎么做的

现在要确定 T_{set} 和 τ_T 这两个数，使得曲线尽量贴近表格里的 241 个实测点。

贴近的程度用残差平方和衡量。对第 i 个数据点，记实测温度是 y_i ，时间是 t_i ，模型给出的值是 $T_{\text{air}}(t_i)$ ，则残差是

$$r_i = y_i - T_{\text{air}}(t_i). \quad (57)$$

残差平方和是

$$S(T_{\text{set}}, \tau_T) = \sum_{i=1}^{241} r_i^2 = \sum_{i=1}^{241} [y_i - (T_{\text{set}} - (T_{\text{set}} - 28) e^{-t_i/\tau_T})]^2. \quad (58)$$

我们要找让 S 最小的那一组参数。这就是最小二乘法的含义：让所有点的偏差平方加起来最小。

为什么要这样做：为什么用平方而不是绝对值

用平方有两个好处。一是平方函数处处可导，可以用求导的办法找最小值；绝对值在零点不可导，处理起来麻烦。二是平方会让大偏差受到更重的惩罚，拟合结果不会为了照顾某个点而牺牲整体。

为什么要这样做：为什么把 S 对参数求导等于零就能找到最小值

在最小值点，函数沿任何方向的切线都是水平的，也就是偏导数为零。所以令

$$\frac{\partial S}{\partial T_{\text{set}}} = 0, \quad \frac{\partial S}{\partial \tau_T} = 0, \quad (59)$$

解这个方程组就得到最优参数。因为式 (58) 里 τ_T 出现在指数上，这两个方程无法用代数方法解出来，只能用迭代的办法逐步逼近。这种算法叫 Levenberg-Marquardt 算法，是数值优化的常用方法。实际计算时我们调用现成的程序，不需要自己写。

3.6 拟合结果

对附件一的数据做上述拟合，得到

参数	拟合值	标准误	均方根误差	决定系数
T_{set}	50.2121 °C	0.0303	0.336 °C	0.9957
τ_T	1877.49 s	12.74		
C_{set}	0.050908 kg/kg	9.3×10^{-5}	8.1×10^{-4}	0.9901
τ_C	2776.34 s	31.71		

表里几个量要解释一下。

标准误衡量这个参数估得准不准。 T_{set} 的标准误是 0.0303 摄氏度，意思是这个值大概有正负 0.03 度的不确定度。

均方根误差是残差平方和除以点数再开方，

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n r_i^2}, \quad (60)$$

它表示平均每个点偏离曲线多少。温度的平均偏差是 0.336 摄氏度，这个量级和测量仪表的噪声相当，说明拟合是合理的。

决定系数 R^2 衡量曲线解释了数据的多大比例，定义是

$$R^2 = 1 - \frac{\sum_{i=1}^n r_i^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (61)$$

其中 $\bar{y}$ 是实测值的平均。 R^2 越接近 1 越好。温度是 0.9957，含湿量是 0.9901，都在 0.99 以上，说明拟合质量可以接受。

为什么要这样做：计算时为什么把参数按有效位数取整

上表里的拟合值是 $T_{\text{set}} = 50.2121$ 、 $\tau_T = 1877.49$ 、 $C_{\text{set}} = 0.050908$ 、 $\tau_C = 2776.34$ 。实际计算用的是一组按有效位数报告的数：

$$T_{\text{set}} = 50.212, \quad \tau_T = 1877.5, \quad C_{\text{set}} = 0.050908, \quad \tau_C = 2776.3. \quad (62)$$

这四个数一起让表面温度变化约 6×10^{-5} 摄氏度。其中设定值 T_{set} 的取整贡献 4.14×10^{-5} 摄氏度，时间常数 τ_T 的取整贡献 1.91×10^{-5} 摄氏度，两项相加就是 6×10^{-5} 摄氏度。而拟合本身的标准误是 0.030 摄氏度，前者比后者小三个数量级。换句话说，纠结最后一位小数是没有意义的，因为标定数据本身就没那么准。但为了结果能与他人核对，文档中明确写出使用了哪一组数值。

3.7 工艺分段

观察附件一可以发现，温度在 t 大约等于 4 小时，也就是 14400 秒时，已经升到接近终值并稳定下来。题目也说整个烘干过程分为预热平衡和恒温干燥两个阶段。

所以从 14400 秒往后，烘房环境就当作恒定的：

$$(T_{\text{air}}, C_{\text{air}}) = \begin{cases} \text{按式 (55) 和 (56) 变化,} & 0 \leq t \leq 14400 \text{ s,} \\ (50.212^\circ\text{C}, 0.050908 \text{ kg/kg}), & t > 14400 \text{ s.} \end{cases} \quad (63)$$

为什么要这样做：恒温阶段为什么含湿量也不变

烘房在持续排湿。如果不排湿，药材蒸发出来的水分会留在烘房里，含湿量会不断上升，最后趋于饱和，干燥就无法继续。实际生产中排湿速率会把含湿量稳定在一个较低的值上，所以当作常数处理是合理的。

3.8 第一部分小结

到这里，模型已经完整了。把所有式子集中写在一起：

类别	式子
传热方程	$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right)$
传质方程	$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right)$
圆心条件	$\frac{\partial T}{\partial r} = \frac{\partial C}{\partial r} = 0, \text{ 当 } r = 0$
表面条件	$-k \frac{\partial T}{\partial r} = h (T - T_{\text{air}}), \text{ 当 } r = R_0$
表面条件	$-D \frac{\partial C}{\partial r} = h_m (C - C_{\text{air}}), \text{ 当 } r = R_0$
初始条件	$T = 28, C = 2.55, \text{ 当 } t = 0$

问题二和问题三用同一组方程，只是物性换成附录三给出的关系式。问题四还要加上半径随时间变化这一条，那部分在第二部分第九章讲，本章没有展开。

这一节要记住的 第一部分讲完了从物理守恒到两个偏微分方程的全过程。核心有三点。第一，微元法加守恒律给出方程。第二，干基含水率让水分方程变得干净。第三，边界条件分对称条件和表面对流条件两类。

Part II

数值方法原理

第一部分得到了两个偏微分方程和四个条件。这一部分回答一个问题：这些方程为什么算不出来，以及我们用什么办法算出来。

4 为什么必须用数值方法

在动手之前，先说明为什么不能像解一元二次方程那样写一个公式就完事。有四个障碍，一个比一个麻烦。

4.1 障碍一：扩散系数随含水率变化

问题一的扩散系数是

$$D = 7 \times 10^{-9} e^{-0.89/C}. \quad (64)$$

这个式子里的 C 是未知的含水率。也就是说，要求解方程必须先知道 D ，而 D 又依赖于要求解的 C 。这种方程叫非线性方程，不能像线性方程那样把解写成级数叠加的形式。

先感受一下这个公式的行为。当 C 从 2.55 降到 0.15，

$$D(2.55) = 7 \times 10^{-9} e^{-0.89/2.55} = 4.94 \times 10^{-9}, \quad (65)$$

$$D(0.15) = 7 \times 10^{-9} e^{-0.89/0.15} = 7 \times 10^{-9} \times e^{-5.933} = 7 \times 10^{-9} \times 2.65 \times 10^{-3} = 1.86 \times 10^{-11}. \quad (66)$$

两者相差 266 倍。也就是说，越干的地方水分越难移动，这正好对应干燥后期速度越来越慢的现象。但这也意味着 D 在空间上是处处不同的，不能提到积分号或导数号外面。

4.2 障碍二：物性同时依赖两个未知量

在问题二到问题四中，扩散系数写成

$$D = 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}, \quad (67)$$

它同时依赖含水率 C 和温度 T 。这里要交代一个单位口径：经验式里温度那一项 $e^{-3850/T}$ 的 T 按附录规定用开尔文，而全文其他地方出现的温度都用摄氏度。所以程序里要先做一次换算 $T_K = T + 273.15$ ，再把 T_K 代入这个经验式。而温度本身也要通过传热方程求解，传热方程里的导热系数 k 又依赖 C 。两个方程互相纠缠，必须联立求解，不能分开各算各的。

4.3 障碍三：边界条件是耦合的

表面条件里出现了表面值 $T(R_0, t)$ 和 $C(R_0, t)$ ，它们本身是未知的，而且和边界的梯度联系在一起。这种边界条件不能直接代入，必须和内部的解一起确定。

4.4 障碍四：求解区域会变形

问题四里药材半径从 2.000 厘米收缩到 1.198 厘米。这意味着我们要求解的区域本身在缩小，边界在移动。这类问题叫移动边界问题，处理起来需要额外的技巧，第九章详细讲。

这一节要记住的 四个障碍导致这道题没有现成的公式解。唯一的办法是把连续的区域切成很多小格，在每个小格上用代数运算近似微分运算，把微分方程变成一大组代数方程，再交给计算机求解。这就是数值方法。

5 坐标变换：把麻烦的 $1/r$ 去掉

这一章讲本套方法最核心的一个技巧。它解决的问题是：第一章里那个 $1/r$ 在圆心附近会带来麻烦。

5.1 麻烦在哪里

回看式 (12)，算子可以写成

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) = D \frac{\partial^2 C}{\partial r^2} + \frac{D}{r} \frac{\partial C}{\partial r} + \frac{\partial D}{\partial r} \frac{\partial C}{\partial r}. \quad (68)$$

中间那一项 $\frac{D}{r} \frac{\partial C}{\partial r}$ 里有 $1/r$ 。

当 r 趋向于零时, $\frac{\partial C}{\partial r}$ 也趋向于零, 两个无穷小相除, 极限是有限的。数学上没有问题。但是对计算机来说有问题: 计算机处理的是离散的点, 在 $r = 0$ 这个点上, 它只会看到分母是零, 无法直接算。

常规的做法是对圆心附近做特殊处理。记常规 r 坐标下的节点间距为 h , 节点排在 $r = 0, h, 2h, \dots$ 上。那么最里面的那个控制体只覆盖 $r = 0$ 到 $r = h/2$, 也就是半径 $h/2$ 的小圆柱, 只有半格宽。把它单独取出来, 体积单独算, 再单独写它对应的方程。这样做当然可以, 但会让程序变得复杂, 而且要额外推导公式, 容易出错。

5.2 换元: 用 u 代替 r

我们采取另一条路。定义一个新的自变量

$$u = \left(\frac{r}{R}\right)^2 = \xi^2, \quad (69)$$

其中 $\xi = r/R$ 是归一化的半径, 取值范围从 0 到 1。新变量 u 的取值范围也是从 0 到 1。

当 $r = 0$ 时 $u = 0$, 当 $r = R$ 时 $u = 1$ 。

为什么要这样做: 为什么想到平方

因为圆柱的算子里的 r 总是以 r 乘以梯度的形式出现, 也就是 $r \frac{\partial}{\partial r}$ 。如果令 u 正比于 r^2 , 那么 du 正比于 $r dr$, 这个 r 恰好可以和算子里的 r 配对消掉。这是一个基于结构的观察, 不是碰巧试出来的。

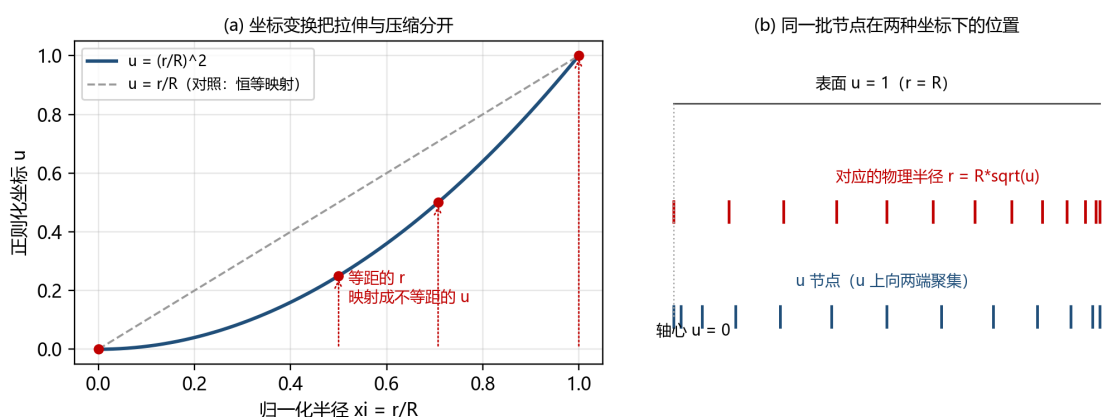


图 3: 坐标变换 u 等于半径比值的平方。(a) 等距的半径映射成不等距的 u ; (b) 同一批节点在两种坐标下的位置, 上半行是 u 坐标, 下半行是物理半径。

5.3 第一步：求出两个导数之间的换算关系

从式 (69) 出发，先反解出 r ：

$$u = \frac{r^2}{R^2} \implies r^2 = R^2 u \implies r = R\sqrt{u}. \quad (70)$$

对式 (70) 求 u 的导数。回忆 $\frac{d}{du}\sqrt{u} = \frac{1}{2\sqrt{u}}$ ，所以

$$\frac{\partial r}{\partial u} = R \cdot \frac{1}{2\sqrt{u}} = \frac{R}{2\sqrt{u}}. \quad (71)$$

链式法则告诉我们，如果 f 是 u 的函数，那么 $\frac{\partial f}{\partial u} = \frac{\partial f}{\partial r} \cdot \frac{\partial r}{\partial u}$ ，于是

$$\frac{\partial}{\partial u} = \frac{R}{2\sqrt{u}} \frac{\partial}{\partial r}. \quad (72)$$

把上式两边同乘 $\frac{2\sqrt{u}}{R}$ ，得到我们真正要用的形式

$$\boxed{\frac{\partial}{\partial r} = \frac{2\sqrt{u}}{R} \frac{\partial}{\partial u}}. \quad (73)$$

5.4 第二步：逐项改造算子

现在把式 (68) 中的每一项拆开处理。为书写方便，记算子为

$$\mathcal{L}\phi = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right), \quad (74)$$

其中 ϕ 代表 C 或 T 。

第一个环节，计算 $\frac{\partial \phi}{\partial r}$ 。直接用式 (73)：

$$\frac{\partial \phi}{\partial r} = \frac{2\sqrt{u}}{R} \frac{\partial \phi}{\partial u}. \quad (75)$$

第二个环节，计算 $D \frac{\partial \phi}{\partial r}$ 。把式 (75) 乘上 D ：

$$D \frac{\partial \phi}{\partial r} = D \cdot \frac{2\sqrt{u}}{R} \frac{\partial \phi}{\partial u}. \quad (76)$$

第三个环节，计算 $r D \frac{\partial \phi}{\partial r}$ 。把式 (70) 的 $r = R\sqrt{u}$ 和式 (76) 相乘：

$$r D \frac{\partial \phi}{\partial r} = R\sqrt{u} \cdot D \cdot \frac{2\sqrt{u}}{R} \frac{\partial \phi}{\partial u}. \quad (77)$$

注意这里出现了 $\sqrt{u} \cdot \sqrt{u} = u$ ，而且 R 和 R 相消，于是

$$r D \frac{\partial \phi}{\partial r} = 2 u D \frac{\partial \phi}{\partial u}. \quad (78)$$

这一步是整段推导的关键：原来的一堆根号全部合并成了一个干净的 u 。

第四个环节，计算 $\frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right)$ 。把式 (73) 作用到式 (78) 上：

$$\frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right) = \frac{2\sqrt{u}}{R} \frac{\partial}{\partial u} \left(2u D \frac{\partial \phi}{\partial u} \right). \quad (79)$$

第五个环节，除以 r 。用 $r = R\sqrt{u}$ ：

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right) = \frac{1}{R\sqrt{u}} \cdot \frac{2\sqrt{u}}{R} \cdot \frac{\partial}{\partial u} \left(2u D \frac{\partial \phi}{\partial u} \right). \quad (80)$$

此时 $\frac{1}{\sqrt{u}} \cdot \sqrt{u} = 1$ ，根号再次消掉，剩下

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right) = \frac{2}{R^2} \frac{\partial}{\partial u} \left(2u D \frac{\partial \phi}{\partial u} \right). \quad (81)$$

把括号外的 2 和括号内的 2 合起来，得到

$$\boxed{\frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial \phi}{\partial r} \right) = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u D \frac{\partial \phi}{\partial u} \right)}. \quad (82)$$

5.5 第三步：检查圆心处的表现

现在看式 (82) 的右边在 $u = 0$ 处的表现。把括号里的导数用乘积法则展开：

$$\frac{\partial}{\partial u} \left(u D \frac{\partial \phi}{\partial u} \right) = D \frac{\partial \phi}{\partial u} + u \frac{\partial}{\partial u} \left(D \frac{\partial \phi}{\partial u} \right). \quad (83)$$

第一项是 $D \frac{\partial \phi}{\partial u}$ 。要说明它在 $u = 0$ 处有限，不能直接断言，要按下面的链条推出来。

第一步，方程在 $u = 0$ 处是正则的。式 (82) 的右边只出现 u 的一次幂乘以导数，没有 $1/u$ ，也没有别的奇性，所以 $u = 0$ 与区间内部的点没有区别。

第二步，在这种方程下，解可以写成 u 的收敛幂级数，也就是解在 u 上是解析的。第六章讨论谱方法时会把这个幂级数写出来。

第三步，幂级数可以逐项求导，所以解对 u 的导数在 $u = 0$ 处有界，也就是存在一个有限的数 M ，使得 $\left| \frac{\partial \phi}{\partial u} \right| \leq M$ 。

第二项里有因子 u ，在 $u = 0$ 处直接等于零，不需要知道 $\frac{\partial \phi}{\partial u}$ 在这一点的大小。

把这几条合起来：右边在 $u = 0$ 处是一个有限的数，没有任何除以零的风险。麻烦被彻底消掉了。

为什么要这样做：这为什么比原来的写法好

原来在 r 坐标下，圆心处的方程需要单独推导，需要知道该处控制体的特殊体积。现在在 u 坐标下，圆心的方程就是同一个公式，直接把 $u = 0$ 代进去就行。程序的写法因此统一，也不容易出错。

5.6 第四步：把方程改写成新坐标下的样子

传质方程原来是

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (84)$$

把右边换成式 (82) 的形式，得到

$$\boxed{\frac{\partial C}{\partial t} = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u D \frac{\partial C}{\partial u} \right).} \quad (85)$$

传热方程同理，

$$\boxed{\rho c_p \frac{\partial T}{\partial t} = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u k \frac{\partial T}{\partial u} \right).} \quad (86)$$

为什么要这样做：为什么右边的未知量还是写成 C 和 T 而不是改写

C 和 T 是物理量，换坐标不改变它们的值，只是把它们看成 u 和 t 的函数而不是 r 和 t 的函数。所以式子里仍然写 C ，只是含义变成 $C(u, t)$ 。

5.7 第五步：边界条件也要改写

表面在 $r = R$ 处，对应 $u = 1$ 。把 $u = 1$ 代入式 (73)，

$$\left. \frac{\partial}{\partial r} \right|_{u=1} = \frac{2\sqrt{1}}{R} \left. \frac{\partial}{\partial u} \right|_{u=1} = \frac{2}{R} \left. \frac{\partial}{\partial u} \right|_{u=1}. \quad (87)$$

温度的表面条件原来是

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h(T(R, t) - T_{\text{air}}(t)). \quad (88)$$

把左边换成 u 的导数,

$$-k \cdot \frac{2}{R} \left. \frac{\partial T}{\partial u} \right|_{u=1} = h(T_1 - T_{\text{air}}(t)), \quad (89)$$

其中 T_1 表示 $u = 1$ 处的温度, 也就是表面温度。两边同乘 $-R/2$,

$$k \left. \frac{\partial T}{\partial u} \right|_{u=1} = -\frac{hR}{2} (T_1 - T_{\text{air}}(t)) = \frac{hR}{2} (T_{\text{air}}(t) - T_1). \quad (90)$$

水分条件同样处理,

$$D \left. \frac{\partial C}{\partial u} \right|_{u=1} = \frac{h_m R}{2} (C_{\text{air}}(t) - C_1). \quad (91)$$

写成这种形式以后, 左右两边都不含根号和分数, 程序里直接照抄这两个式子就行。

5.8 第六步：圆心处还需要加条件吗

不需要。理由要用下面这条链条说, 不能只看通量。

先说一个不能用的理由。当 $u = 0$ 时, 式 (85) 右边的通量 $uD \frac{\partial C}{\partial u}$ 前面有因子 u , 等于零, 所以从圆心流出的总通量是零。但这只是因为那里的面积是零, 面积为零推不出一阶导数 $\frac{\partial C}{\partial r}$ 为零。这两件事必须分开。

正确的理由链条有第四步。

第一步, 方程在 $u = 0$ 处正则。式 (82) 的右边没有 $1/u$ 之类的奇性, $u = 0$ 与区间内部的点没有区别。

第二步, 在这种方程下, 解可以写成 u 的收敛幂级数, 也就是解在 u 上解析。

第三步, 幂级数逐项可导, 所以解对 u 的导数在 $u = 0$ 处有界, 记这个界为 M , 即 $\left| \frac{\partial C}{\partial u} \right| \leq M$ 。

第四步, 用链式法则回到 r 。由式 (73),

$$\frac{\partial C}{\partial r} = \frac{2\sqrt{u}}{R} \frac{\partial C}{\partial u}. \quad (92)$$

当 u 趋于零时, 右边的因子 $\frac{2\sqrt{u}}{R}$ 趋于零, 而 $\frac{\partial C}{\partial u}$ 有界, 所以整个右边趋于零。

于是 $r = 0$ 处的一阶导数确实等于零, 这正是对称条件。它由方程本身和解的正则性保证, 不需要另外写进程序。

这一节要记住的 坐标变换做了三件事。第一，把 $1/r$ 消掉了，圆心不再是特殊点。第二，两个方程形式完全一样，只是系数不同，程序可以共用一套代码。第三，表面条件化成整洁的代数式。

6 谱方法：用很少的点算得很准

有了新坐标，接下来要把区间 $0 \leq u \leq 1$ 切成有限个点，在点上求近似解。这一章讲怎么切、怎么把导数变成代数运算。

6.1 基本想法：用多项式代替未知函数

假设我们不知道 $C(u, t)$ 的具体形式，但知道它在若干个点上的值。能不能构造一个多项式，让它在这几个点上的值正好等于已知值，然后用这个多项式代表未知函数？

可以。这个多项式叫插值多项式，这几个点叫插值节点。

一旦有了多项式，求导数就变成对多项式求导，结果仍然是多项式，可以精确算出它在各点的值。这样微分运算就变成了有限的代数运算。

6.2 节点怎么选：一个反面教材

最自然的想法是把点均匀分布。比如在 0 到 1 之间取 11 个等距点。这种做法在点数少的时候还行，点数一多就出问题。

问题出在函数形状复杂的地方。均匀分布的点无法同时照顾区间两端和中间，当多项式次数升高时，两端会出现剧烈的上下摆动。这个现象叫龙格现象，是插值理论里的经典结论。

为什么要这样做：为什么会摆动

直觉的解释是：区间两端的点比较少的信息去约束多项式，而高次多项式在端点的变化率很大，稍微改变一点系数就会让端点附近剧烈起伏。均匀分布没有在两端加密，所以控制不住。

6.3 切比雪夫节点

解决办法是在区间两端加密。具体做法是取

$$x_j = \cos \frac{j\pi}{N}, \quad j = 0, 1, 2, \dots, N. \quad (93)$$

这些点分布在 -1 到 1 之间。当 $j = 0$ 时 $x_0 = \cos 0 = 1$, 当 $j = N$ 时 $x_N = \cos \pi = -1$ 。

这些点有一个特点：靠近两端时， $\cos$ 的变化变慢，所以点会自然地在两端聚集。可以验证，最靠近端点的两个点之间的距离大约是 $\frac{\pi^2}{2N^2}$ ，而中间的间距大约是 $\frac{\pi}{N}$ 。把这两个间距相除，

$$\frac{\pi/N}{\pi^2/(2N^2)} = \frac{2N}{\pi} \approx 0.64N, \quad (94)$$

所以两端的点比中间密约 $\frac{2N}{\pi}$ 倍，也就是约 $0.64N$ 倍。

再把它映射到区间 0 到 1 ：

$$u_j = \frac{1 - x_j}{2}. \quad (95)$$

当 $x = 1$ 时 $u = 0$ ，当 $x = -1$ 时 $u = 1$ 。这样 $u_0 = 0$ ， $u_N = 1$ ，正好对应圆心和表面。

为什么要这样做：映射公式里的 1 和 2 是怎么来的

把区间 -1 到 1 线性映射到 0 到 1 ，需要先加 1 把区间移到 0 到 2 ，再除以 2 缩放到 0 到 1 。也就是 $u = \frac{x+1}{2}$ 。而我们的 $x_j = \cos$ 是从 1 递减到 -1 的，所以写成 $u = \frac{1-x}{2}$ 正好让 u 从 0 递增到 1 。

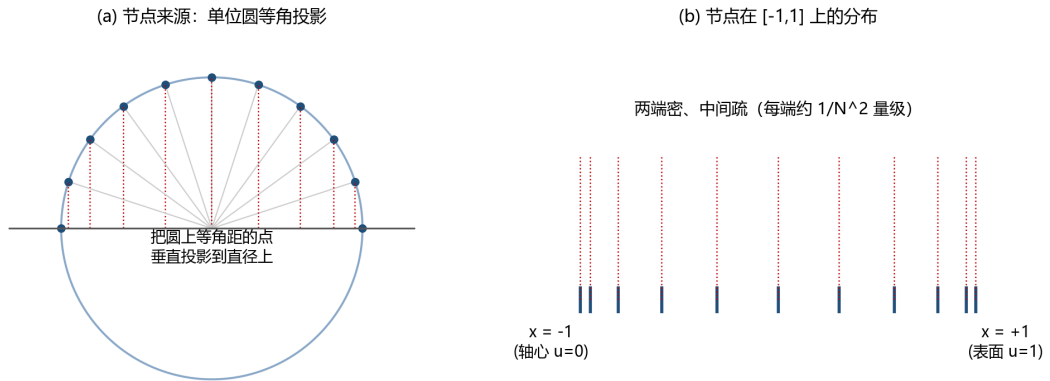


图 4：切比雪夫节点的来源。(a) 在单位圆上按等角距取点，再垂直投影到直径上；(b) 投影结果是两端密、中间疏的分布，两端间距约为 N 平方分之一量级。

6.4 插值多项式的唯一性

有一个重要的事实：给定 $N + 1$ 个互不相同的节点和对应的函数值，次数不超过 N 的插值多项式是唯一的。

理由是这样。设两个不同的多项式都满足条件，它们的差也是一个次数不超过 N 的多项式，但在这个 $N + 1$ 个节点上都等于零。一个次数不超过 N 的非零多项式最多只有 N 个根，现在有了 $N + 1$ 个根，只能是恒等于零的多项式。所以两个多项式其实是同一个。

这个事实保证了插值结果是确定的，不受计算方法影响。

6.5 拉格朗日插值公式

插值多项式可以写成

$$p(u) = \sum_{j=0}^N C_j \ell_j(u), \quad (96)$$

其中 C_j 是函数在第 j 个节点上的值， $\ell_j(u)$ 叫基函数，定义是

$$\ell_j(u) = \prod_{\substack{k=0 \\ k \neq j}}^N \frac{u - u_k}{u_j - u_k}. \quad (97)$$

这个式子的意思需要解释一下。

分子是 N 个因子的乘积，每个因子是 u 减去一个别的节点的位置。当 u 取第 j 个节点时，分子里出现因子 $u_j - u_j = 0$ ，整个分子等于零。

分母是把分子里的 u 换成 u_j 得到的常数。

所以当 $u = u_j$ 时，分子分母相同， $\ell_j(u_j) = 1$ 。而当 $u = u_k$ ，也就是 k 不等于 j 时，分子里有一个因子 $u_k - u_k = 0$ ，所以 $\ell_j(u_k) = 0$ 。

把这两条合起来，代入式 (96)，当 $u = u_j$ 时，

$$p(u_j) = C_j \cdot 1 + \sum_{k \neq j} C_k \cdot 0 = C_j. \quad (98)$$

插值条件得到满足。

6.6 从插值多项式得到导数

对式 (96) 求导，

$$p'(u) = \sum_{j=0}^N C_j \ell'_j(u). \quad (99)$$

在节点 u_i 处取值,

$$p'(u_i) = \sum_{j=0}^N C_j \ell'_j(u_i). \quad (100)$$

把 $\ell'_j(u_i)$ 记作 D_{ij} , 就得到一个矩阵形式的关系:

$$\begin{pmatrix} p'(u_0) \\ p'(u_1) \\ \vdots \\ p'(u_N) \end{pmatrix} = \begin{pmatrix} D_{00} & D_{01} & \cdots & D_{0N} \\ D_{10} & D_{11} & \cdots & D_{1N} \\ \vdots & \vdots & \ddots & \vdots \\ D_{N0} & D_{N1} & \cdots & D_{NN} \end{pmatrix} \begin{pmatrix} C_0 \\ C_1 \\ \vdots \\ C_N \end{pmatrix}. \quad (101)$$

这个矩阵叫微分矩阵。它的作用是: 输入一组函数值, 输出一组导数值。求导这个微分运算, 被彻底变成了矩阵乘法。

6.7 微分矩阵的元素公式

对切比雪夫节点, 微分矩阵的元素有封闭形式的公式。记

$$c_0 = 2, \quad c_N = 2, \quad c_j = 1 \quad (1 \leq j \leq N-1). \quad (102)$$

当 i 不等于 j 时,

$$D_{ij} = \frac{c_i}{c_j} \cdot \frac{(-1)^{i+j}}{x_i - x_j}. \quad (103)$$

当 i 等于 j 时, 不能直接用这个公式, 因为分母为零。对角线元素用另一条性质确定: 常数函数的导数处处为零。把常数函数 $f \equiv 1$ 代入式 (101), 左边全是零, 右边是对每一行求和, 于是

$$\sum_{j=0}^N D_{ij} = 0 \quad \text{对每个 } i, \quad (104)$$

所以

$$D_{ii} = - \sum_{\substack{j=0 \\ j \neq i}}^N D_{ij}. \quad (105)$$

为什么要这样做：为什么要用这个办法求对角元

因为直接计算对角元需要求极限，数值上不稳定。用行和为零这条性质，既精确又简单。这条性质来自一个简单事实：常数的导数为零，而插值多项式对常数函数是精确的。

6.8 记号的换算

式 (103) 里的 x 是切比雪夫变量。我们的自变量是 u ，两者关系是 $u = \frac{1-x}{2}$ ，反过来 $x = 1 - 2u$ 。

对 u 求导和对 x 求导的关系由链式法则给出：

$$\frac{\partial}{\partial u} = \frac{\partial x}{\partial u} \cdot \frac{\partial}{\partial x} = \frac{\partial(1-2u)}{\partial u} \cdot \frac{\partial}{\partial x} = -2 \frac{\partial}{\partial x}. \quad (106)$$

所以

$$D_u = -2D_x. \quad (107)$$

程序里先算 D_x ，再乘 -2 得到 D_u ，最后对 u 求导就用 D_u 。

6.9 一个三节点的手算例子

上面的公式看着抽象。这一段把节点数取成 $N = 2$ ，也就是只用三个点，把整个过程从零算一遍。每一个数字都会写出来，你可以拿计算器逐个复核。

第一步，算节点位置。

按式 (93)，让 j 依次取 0、1、2：

$$x_0 = \cos \frac{0 \cdot \pi}{2} = \cos 0 = 1, \quad (108)$$

$$x_1 = \cos \frac{1 \cdot \pi}{2} = \cos \frac{\pi}{2} = 0, \quad (109)$$

$$x_2 = \cos \frac{2 \cdot \pi}{2} = \cos \pi = -1. \quad (110)$$

再按式 (95) 换算到 u 坐标：

$$u_0 = \frac{1-1}{2} = 0, \quad u_1 = \frac{1-0}{2} = 0.5, \quad u_2 = \frac{1-(-1)}{2} = 1. \quad (111)$$

三个节点分别落在圆心、径向中点、表面。

第二步，算系数 c 。

按定义，首尾两个取 2，中间取 1，所以

$$c_0 = 2, \quad c_1 = 1, \quad c_2 = 2. \quad (112)$$

第三步，算节点之间的差值 $x_i - x_j$ 。

把所有九个组合列成表。先写对角线为零，稍后修正。

i 与 j	$j = 0$	$j = 1$	$j = 2$
$i = 0$	0	$1 - 0 = 1$	$1 - (-1) = 2$
$i = 1$	$0 - 1 = -1$	0	$0 - (-1) = 1$
$i = 2$	$-1 - 1 = -2$	$-1 - 0 = -1$	0

第四步，算非对角线上的元素。

按式 (103)，公式是

$$D_{ij} = \frac{c_i}{c_j} \cdot \frac{(-1)^{i+j}}{x_i - x_j}. \quad (113)$$

逐个算六个非对角线元素。

$i = 0, j = 1$:

$$D_{01} = \frac{c_0}{c_1} \cdot \frac{(-1)^1}{x_0 - x_1} = \frac{2}{1} \cdot \frac{-1}{1} = -2. \quad (114)$$

$i = 0, j = 2$:

$$D_{02} = \frac{2}{2} \cdot \frac{(-1)^2}{2} = 1 \times \frac{1}{2} = 0.5. \quad (115)$$

$i = 1, j = 0$:

$$D_{10} = \frac{1}{2} \cdot \frac{(-1)^1}{-1} = 0.5 \times \frac{-1}{-1} = 0.5. \quad (116)$$

$i = 1, j = 2$:

$$D_{12} = \frac{1}{2} \cdot \frac{(-1)^3}{1} = 0.5 \times (-1) = -0.5. \quad (117)$$

$i = 2, j = 0$:

$$D_{20} = \frac{2}{2} \cdot \frac{(-1)^2}{-2} = 1 \times \frac{1}{-2} = -0.5. \quad (118)$$

$i = 2, j = 1$:

$$D_{21} = \frac{2}{1} \cdot \frac{(-1)^3}{-1} = 2 \times \frac{-1}{-1} = 2. \quad (119)$$

为什么要这样做：程序里为什么不写这个公式

程序里写的是先把 $(-1)^j$ 乘进系数 c ，于是 c_i/c_j 这一个因子就同时包含了 $\frac{c_i}{c_j}$ 和 $(-1)^{i+j}$ ，公式变成一次除法。这样写更短，结果完全一样。如果用上面这个公式逐个手算，得到的是同一组数字。

第五步，用行和为零求对角线元素。

对角线上不能直接用上面的公式，因为分母为零。改用另一条性质：常数函数的导数处处为零。把常数函数 $f \equiv 1$ 代入，左边每行都是零，右边是矩阵每一行的和，于是每一行的和必须等于零：

$$D_{ii} = - \sum_{j \neq i} D_{ij}. \quad (120)$$

第一行：

$$D_{00} = -(D_{01} + D_{02}) = -(-2 + 0.5) = -(-1.5) = 1.5. \quad (121)$$

第二行：

$$D_{11} = -(D_{10} + D_{12}) = -(0.5 - 0.5) = -0 = 0. \quad (122)$$

第三行：

$$D_{22} = -(D_{20} + D_{21}) = -(-0.5 + 2) = -(1.5) = -1.5. \quad (123)$$

把全部元素拼起来，得到

$$D_x = \begin{pmatrix} 1.5 & -2 & 0.5 \\ 0.5 & 0 & -0.5 \\ -0.5 & 2 & -1.5 \end{pmatrix}. \quad (124)$$

第六步，验证这个矩阵对不对。

用两个已知导数的函数来检验。

第一个用常数函数 $f(x) = 1$ ，它的导数是零。把 f 在三个节点上的值写成列向量 $(1, 1, 1)$ ，用矩阵去乘：

$$\begin{pmatrix} 1.5 & -2 & 0.5 \\ 0.5 & 0 & -0.5 \\ -0.5 & 2 & -1.5 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.5 - 2 + 0.5 \\ 0.5 + 0 - 0.5 \\ -0.5 + 2 - 1.5 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}. \quad (125)$$

三个分量都是零，正确。

第二个用一次函数 $f(x) = x$ ，它的导数是常数 1。 f 在三个节点上的值是 $(1, 0, -1)$ ，

$$\begin{pmatrix} 1.5 & -2 & 0.5 \\ 0.5 & 0 & -0.5 \\ -0.5 & 2 & -1.5 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 1.5 + 0 - 0.5 \\ 0.5 + 0 + 0.5 \\ -0.5 + 0 + 1.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}. \quad (126)$$

三个分量都是 1，也正确。

第七步，换成对 u 求导的矩阵。

按式 (107)，把上面每个元素乘 -2 ：

$$D_u = -2D_x = \begin{pmatrix} -3 & 4 & -1 \\ -1 & 0 & 1 \\ 1 & -4 & 3 \end{pmatrix}. \quad (127)$$

再用一次函数检验。在 u 坐标下取 $f(u) = u$ ，它在三个节点上的值是 $(0, 0.5, 1)$ ，而 $\frac{du}{du} = 1$ ，

$$\begin{pmatrix} -3 & 4 & -1 \\ -1 & 0 & 1 \\ 1 & -4 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 0.5 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 + 2 - 1 \\ 0 + 0 + 1 \\ 0 - 2 + 3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}. \quad (128)$$

三个分量都是 1，说明程序里那行 $D_u = -2D_x$ 得到的结果是对的。

这一节要记住的 三个节点的手算结果给出两条检验标准。第一，微分矩阵每一行的和必须等于零，因为常数的导数为零。第二，把节点坐标本身当作函数代进去，结果必须是 1，因为一次函数的导数是常数。这两条只要有一条不满足，矩阵就一定写错了。

6.10 为什么谱方法精度高

常规的差分方法，比如用相邻两点做差商近似导数，误差是步长的平方量级。步长减半，误差降到四分之一。要想精度提高一千倍，得把点数增加三十多倍。

谱方法不一样。如果函数足够光滑，插值多项式的误差随节点数增加而指数下降。也就是说，点数稍微增加一点，误差就急剧变小。

原因可以这样理解。差分方法只用了相邻几个点的信息，而谱方法的插值多项式用了所有点的信息，等价于用整个区间上的信息去估计每一点的导数，所以效率高得多。

为什么要这样做：本题能用谱方法吗

可以，而且效果很好。原因在第五章已经埋下伏笔。在 u 坐标下，解是关于 u 的解析函数，具体说，第一类贝塞尔函数 J_0 的展开式

$$J_0\left(\frac{\beta r}{R}\right) = \sum_{k \geq 0} \frac{(-1)^k}{k! k!} \left(\frac{\beta^2}{4}\right)^k u^k \quad (129)$$

只含 u 的整数次幂，是一个处处收敛的幂级数。这类函数正是多项式插值最擅长逼近的对象。

实际计算中，节点参数 N 由 4 加到 6，与精确解的偏差就从 7.59×10^{-6} 掉到 2.88×10^{-10} ，只多用了两个节点。误差不是慢慢下降，而是跳台阶，这才是谱收敛的准确表现。

再往上加节点，偏差就停在 10^{-10} 附近不再继续下降，节点参数从 8 加到 64 都是这个水平。这个平台不是空间离散的极限，而是时间积分的相对容差造成的地板：节点数不变、只把时间相对容差收紧，偏差就跟着变小。作收敛性检验时时间相对容差取 10^{-12} 。

这一节要记住的 谱方法的三步是：选切比雪夫节点，构造微分矩阵，用矩阵乘法代替求导。本题里节点参数 N 从 4 加到 6，偏差就从 7.59×10^{-6} 掉到 2.88×10^{-10} ，说明空间精度随节点数急剧提高。偏差停在 10^{-10} 是时间积分容差造成的地板，不是空间离散的极限。

7 时间推进：从当前时刻推到下一时刻

到上一章为止，我们把空间方向的导数处理完了。剩下时间方向的导数 $\frac{\partial C}{\partial t}$ 还没处理。这一章讲怎么处理。

7.1 半离散化

先只对空间做离散。在 $N + 1$ 个节点上，方程

$$\frac{\partial C}{\partial t} = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u D \frac{\partial C}{\partial u} \right) \quad (130)$$

变成 $N + 1$ 个常微分方程。比如第 j 个节点上的方程是

$$\frac{dC_j}{dt} = \frac{4}{R^2} [\text{第 } j \text{ 个节点上的空间算子离散值}]. \quad (131)$$

这个过程叫半离散化，意思是空间已经离散，时间还是连续的。

为什么要这样做：为什么先做空间再做时间

因为空间离散化以后，问题的规模就定下来了，只剩下一组关于时间的常微分方程。这种方程有很多成熟的求解程序可以直接用，不需要自己从零开始写。这也是一种分工策略：难的部分自己做，成熟的部分用现成的。

7.2 最简单的方法：显式欧拉法

把时间也切成小段，步长记作 Δt 。最简单的方法是用前向差商近似时间导数：

$$\frac{dC_j}{dt} \approx \frac{C_j^{n+1} - C_j^n}{\Delta t}, \quad (132)$$

其中上标 n 表示第 n 个时刻， $n + 1$ 表示下一个时刻。

把式 (132) 代入式 (131)，得到

$$C_j^{n+1} = C_j^n + \Delta t \cdot \frac{4}{R^2} [\text{空间算子}(C^n)]. \quad (133)$$

右边全部是已知的旧值，直接算出来就是新值，不需要解方程。这个方法叫显式欧拉法。

7.3 显式欧拉法的问题：稳定性

显式欧拉法有一个致命缺点。它可以写成

$$C^{n+1} = (I + \Delta t L) C^n, \quad (134)$$

其中 L 是空间离散矩阵, I 是单位矩阵。要让解不发散, 要求 Δt 足够小, 具体来说要求 Δt 小于某个与 Δu^2 同量级的阈值。

而谱方法的节点在两端非常密集。 u 坐标下最小的节点间距大约是 $\frac{\pi^2}{4N^2}$, 换成 x 坐标就是 $\frac{\pi^2}{2N^2}$ 。节点参数取 64 时, 实测的最小间距是 6.023×10^{-4} , 公式给出 6.024×10^{-4} , 两者可以对上。

显式方法要求 Δt 小于某个与最小节点间距的平方成正比的阈值。这句话的意思是: 节点越密, 允许的步长越小, 而且是按间距的平方缩小。节点参数取 64 时, 实测的最小间距是 6.023×10^{-4} , 它已经是网格中最小的一个, 所以它决定步长上界。

现在算量级。先看物理过程本身的时间尺度。时间方向的量纲因子是 $\frac{R^2}{4D}$, 取 $C = 2.55$ 时的扩散系数, 它约等于 2.03×10^4 秒。也就是说, 单靠扩散把水分走完两厘米, 需要两万秒的量级。

再看显式方法允许的步长。把同一套空间离散接到显式欧拉上实测: 步长取 2×10^{-4} 秒时稳定, 取 4×10^{-4} 秒时发散, 所以稳定上界约为 4×10^{-4} 秒。

两个数一比就看出问题了: 允许的步长比物理过程的时间尺度小了约 5×10^7 倍。也就是说每一秒要算约 2500 步, 而整个过程要算 2×10^5 秒, 总步数在 5×10^8 量级。每步还要解一个含 65 个未知量的方程组, 这个计算量是无法接受的。

这里只给出趋势与实测上界, 不去套一个解析的步长公式。原因是本题的右端项同时含温度与含水率, 扩散系数本身随含水率变化, 最大特征值没有简单的解析表达式, 老老实实测一次更可靠。

这种要求时间步长远小于物理过程时间尺度的现象, 叫做刚性。本题就是典型的刚性问题。

为什么要这样做: 刚性的物理来源

表面处的干燥非常快, 因为一开始表面含水率是 2.55, 而空气中的含湿量只有 0.0196, 落差很大, 表面迅速失水。在很短的时间内, 表面附近形成一层很薄的水分变化区。要准确刻画这一层, 时间步长必须很小。但整个干燥过程长达五十多个小时。快慢两个尺度相差极大, 这就是刚性的根源。

7.4 隐式方法

换一个思路。用后向差商近似时间导数:

$$\frac{dC_j}{dt} \approx \frac{C_j^{m+1} - C_j^m}{\Delta t}. \quad (135)$$

式子的形式看起来和式 (132) 一样, 区别在于右端空间算子取的是哪个时刻的值。显

式方法取旧时刻 n ，隐式方法取新时刻 $n + 1$ ：

$$\frac{C_j^{n+1} - C_j^n}{\Delta t} = \frac{4}{R^2} [\text{空间算子}(C^{n+1})]. \quad (136)$$

现在右边含有未知的新值，两边都有未知量，需要解一个方程组才能得到新值。这就是隐式的含义。

为什么要这样做：隐式方法为什么能允许大得多的步长

可以这样理解。显式方法相当于说我用现在的趋势直接外推，如果趋势变化很快，外推就会跑偏。隐式方法相当于说我先猜一个新值，然后检查它是否满足方程，不满足就调整。这种自我修正的机制让方法对快速变化有更强的抵抗力。数学上可以证明，后向欧拉法是无条件稳定的，不管步长多大解都不会发散。

代价是每一步都要解方程组。但本题的方程组规模很小，用现成的线性代数程序在毫秒级就能解完，完全可以接受。

7.5 更高阶的方法：BDF

后向欧拉法只有一阶精度，也就是说误差与步长成正比。步长减半，误差减半。要提高精度，可以用更高阶的公式。

BDF 是向后差分公式的缩写。它的想法是利用前面若干个时刻的解，构造一个更高精度的差商近似。

一阶 BDF 就是后向欧拉法，用到 1 个历史时刻。二阶 BDF 用到 2 个历史时刻，公式是

$$\frac{3C^{n+1} - 4C^n + C^{n-1}}{2\Delta t} = \frac{4}{R^2} [\text{空间算子}(C^{n+1})]. \quad (137)$$

分子上的系数 3、-4、1 来自对二次插值多项式求导，不是随便凑出来的。三阶、四阶、五阶的形式类似，分别用到 3、4、5 个历史时刻。

阶数越高，精度越高，但稳定性区域越小，对步长的限制越多。实践中的做法是在程序里同时实现多个阶数，根据当前的误差估计自动选择用几阶。

7.6 自适应步长

步长固定不变有一个弊端。干燥刚开始时变化剧烈，需要小步长；过了两个小时，温度和水分都变化很慢，大步长就够用了。如果全程都用小步长，会浪费大量时间。

自适应步长的做法是：先按当前步长算一步，同时估算这一步的误差。如果误差超过容忍值，就把步长减小重算；如果误差远小于容忍值，就把步长增大。

误差估计的常用办法是比较两个不同阶数方法的结果，两者的差可以作为误差的近似。这个技巧叫嵌入式方法。

程序里的两个控制参数是

$$\text{rtol (相对容差)}, \quad \text{atol (绝对容差)}. \quad (138)$$

判据是每一步的误差要满足

$$\text{误差} \leq \text{atol} + \text{rtol} \times |\text{解}|. \quad (139)$$

本题取的数值是 $\text{rtol} = 10^{-11}$ ，温度分量的 atol 取 10^{-11} ，水分分量的 atol 取 10^{-13} 。

为什么要这样做：为什么水分和温度的容差取得不一样

因为两者的量级不一样。温度是几十摄氏度，水分浓度是零点几到两点几。如果都用同一个绝对容差，水分那边可能过于宽松，温度那边可能过于严格。按各自量级分别设置更合理。

7.7 一个必须注意的细节：系数用什么时刻的值

方程里的 D 、 k 、 ρc_p 都依赖 C 和 T ，而 C 和 T 本身是未知的。这就带来一个选择：

可以在每个时间步开始时，用旧的 C 和 T 算出系数，然后在整个时间步内把系数当作常数使用。这种做法叫系数冻结。它的优点是每一步只需要解一次线性方程组。

也可以在每一步内部迭代若干次，每次都最新的 C 和 T 重新计算系数。这种做法更准，但每一步要解若干次方程组。

本题的主求解器采用的是一种折中：右端项每次被调用时，都用当时传入的 C 和 T 重新计算系数。由于自适应积分器在每一步内部会多次调用右端项，实际上相当于做了隐式的迭代，精度更高。代价是每一步的计算量稍大，但程序更简洁，不容易出错。

这一节要记住的 时间推进的核心结论是：本题是刚性问题，必须用隐式方法；用自适应变阶的 BDF 方法可以在保证精度的同时节省计算量。

8 烘干时间怎么确定

前几章解决的是怎么算出任意时刻的温度和水分场。问题三和问题四要回答的是另一个问题：什么时候算烘干完成。这一章讲这个判断怎么做。

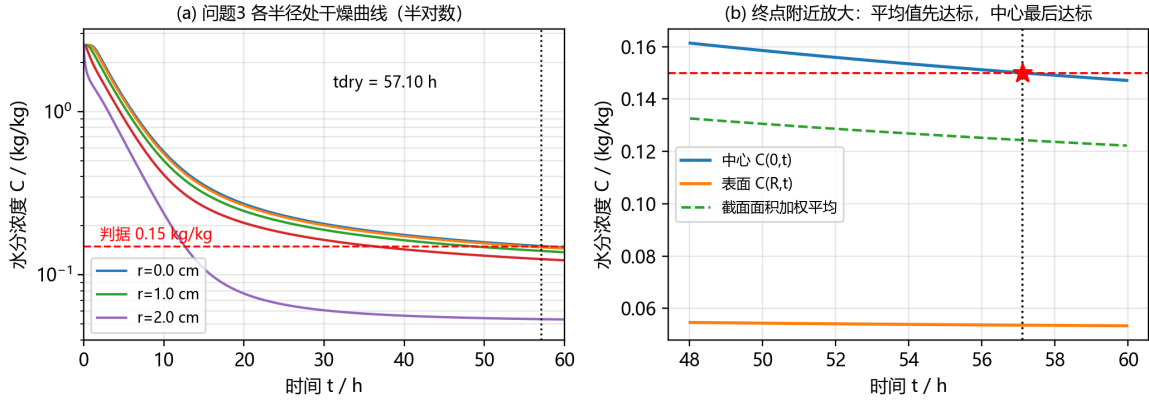


图 5: 问题 3 的干燥曲线。(a) 各半径处水分浓度随时间下降，横轴到 60 小时；(b) 终点附近放大，截面平均值先达标，中心最后达标，所以判据必须取全场最大值而不是平均值。

8.1 判据的数学表述

题目说药材各处的水分浓度应低于 0.15 千克每千克。这句话要翻译成数学语言。

关键字是各处。也就是整个径向范围内每一个位置都要满足，不能只看平均值，也不能只看表面。

写成式子就是

$$\max_{0 \leq r \leq R_0} C(r, t) < 0.15. \quad (140)$$

烘干时间是第一次满足这个条件的那一时刻，写成

$$t_{\text{dry}} = \min \left\{ t > 0 : \max_{0 \leq r \leq R_0} C(r, t) < 0.15 \right\}. \quad (141)$$

这个式子的读法是：在所有使最大值小于 0.15 的时刻里，取最小的那一个。

为什么要这样做：为什么必须逐点判断而不能用平均值

因为干燥是从表面往内部推进的。在干燥的中后期，表面已经很干，而中心还比较湿。如果看平均值，平均值可能早就低于 0.15 了，但中心还没达标，药材实际上还是湿的。

本题的实际数据是：截面面积加权平均含水率在 35.9 小时就降到了 0.15，而中心要到 57.1 小时才达标，晚了将近 21 小时。用平均值判断会把时间低估约 37%。

这里有一个必须交代的细节，否则上面这个数会被算错。截面平均的定义是

$$\bar{C}(t) = \frac{2}{R^2} \int_0^R C(r, t) r \, dr, \quad (142)$$

其中权重 r 来自环面积与半径成正比这一事实。计算这个积分时，如果直接用谱方法节点上的值做加权平均，结果会被拉低约 0.2，达标时刻提前十个多小时。原因是切比雪夫节点在表面附近极密，而表面正是含水率最低的地方，按节点直接平均等于给表层过高的权重。

正确的做法是先把解插值到均匀的半径网格，再按面积加权积分。用交付的结果文件 result3.xlsx 复核时也要注意：该文件只给出 21 个半径列，用它做梯形积分得到 35.2 小时，把半径点数加到 401 个得到 35.8 小时，再加到 2001 个仍是 35.8 小时。两者相差 0.6 小时，差别来自积分点数，不是物理差异。本文取收敛值 35.9 小时。

8.2 哪一点最后干

从物理上可以预判：中心应该是最后干的地方。理由是水分要从内部扩散到表面才能被带走。中心到表面的距离最长，扩散路程最长，而且它周围都是含水率更高的地方，浓度梯度最小，驱动力最弱。

但是本文没有把这个预判直接写进程序，而是在每一个时刻对全部节点取最大值，看最大值出现在哪里。这样做的原因是：如果预判错了，程序不会跟着错。实际运行的每一时刻结果都显示最大值出现在中心，预判得到了验证。

为什么要这样做：为什么要在全部分节点上取最大而不是只在输出位置上取最大

因为输出的位置是每隔 0.1 厘米一个点，共 21 个点。而计算的节点有 $N+1$ 个，在两端更密集。如果只在 21 个输出点上取最大，有可能真正的最大值出现在两个输出点之间而没有被发现。虽然本题中最大值总在中心，但把范围取全是一个更安全的做法。

8.3 怎么求这个时刻

式 (141) 定义的是一个事件的时刻。程序里的处理办法是构造一个函数

$$e(t) = \max_{0 \leq j \leq N} C_j(t) - 0.15. \quad (143)$$

当 $e(t) > 0$ 时说明还没干，当 $e(t) < 0$ 时说明已经干了。烘干时间就是 $e(t)$ 第一次从正变负的时刻，也就是 $e(t) = 0$ 的第一个根。

求根用二分法或者布伦特方法。这两种方法都是标准的数值方法，程序里由积分器自动调用。

两个方法的基本原理很简单。二分法是：如果知道 e 在 t_1 处为正、在 t_2 处为负，那么根一定在两者之间。取中点 t_3 ，看 $e(t_3)$ 的符号，就能把根所在的范围缩小一半。反复这样做，区间会越来越小，最后得到一个足够精确的近似值。

为什么要这样做：为什么不用在网格序列上做外推

有一种常见做法是：用几组不同的网格算若干次，得到若干个烘干时间，然后按某种规律外推到一个无限细的网格。这种做法需要假设收敛的阶数，如果假设错了结果就会偏。

本文的做法更直接：直接用自适应积分器给出的高精度解来求根，不需要任何关于收敛阶的假设。后来在第四部分会看到，这两种做法的结果确实相差约 15 秒，差异来自外推所用的序列本身没有收敛。

这一节要记住的 烘干时间的判定要注意三点。第一，判据是逐点的最大值，不是平均值。第二，程序在所有节点上取最大，不预设极值位置。第三，用求根而不是外推来确定时刻。

8.4 第二部分小结

第二部分解决了从方程到程序的全部原理问题：

问题	办法	在本文中的名称
圆心处 $1/r$ 的麻烦	换元 $u = (r/R)^2$	坐标正则化
空间导数怎么算	切比雪夫节点加微分矩阵	谱配置法
时间方向怎么推进	隐式变阶 BDF 加自适应步长	自适应时间积分
导数怎么取近似	先由谱微分矩阵算出节点通量，再对通量整体求导	通量形式配置
烘干时刻怎么确定	构造函数求零点	终端事件求交

表中的通量形式配置，做法是先用谱微分矩阵算出每个节点上的通量，再对整个通量向量求一次导，它不是有限体积那种相邻控制体之间的通量差格式。本文的对照格式才用有限体积。

9 问题四的收缩怎么处理

问题一到问题三都假设药材半径不变。问题四不一样，题目给了附件二，里面记录了半径随时间的变化，从最初的 2.000 厘米一直降到 1.198 厘米，缩了四成。

半径变化意味着求解区域的边界在移动。这一章专门讲怎么处理这种情况。

9.1 第一种想法为什么是错的

最自然的想法是：半径坐标下的方程不是已经写好了吗，直接让 R 随时间变化不就行了。

也就是说，直接使用式 (42)

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) \quad (144)$$

再把表面条件里的 R 换成 $R(t)$ 。

这个做法是错的。原因要讲清楚，因为直觉上很难发现。

式 (42) 里的时间导数是在固定空间位置求的。意思是：站在空间里某个固定的点，看那里的含水率怎么变。

但药材在收缩，干物质在移动。水分是跟着干物质一起走的。站在固定的空间点上，含水率的变化有两个来源：一个是水分的扩散，另一个是干物质带着水分从旁边走过。

后一个来源在原方程里没有被描述，因为原方程推导时假设了干物质不动。

9.2 用一个极端例子把这个错误暴露出来

为了看清楚，考虑一个极端情况：假设扩散系数等于零。

此时水分完全不能扩散，全部被困在干物质里面。那么每个材料点的含水率永远不会变，一直等于它自己的初始值。

再假设初始含水率不均匀，取中心为 2.55、表面为 1.5。题目给的初值本来是处处相同的 2.55，这里换成不均匀的分布，是为了让两种写法的差别显出来。

两个条件合起来得到什么结论？每个材料点的含水率都保持自己的初始值，但是材料点在向内移动。本来在半径 1.5 厘米处的干物质，随着收缩慢慢移到半径 1.0 厘米处，它带着自己的含水率一起走。

所以在固定的空间点 $r = 1.0$ 厘米处，一开始那里是内部的水，含水率高；后来移过来的是表层的水，含水率低。含水率随时间变化，也就是

$$\left. \frac{\partial C}{\partial t} \right|_r \neq 0. \quad (145)$$

现在看错误的方程给出什么。把 $D = 0$ 代入式 (42)，右边变成零，所以方程给出 $\left. \frac{\partial C}{\partial t} \right|_r = 0$ 。

两个结果矛盾：物理推理给出变化，错误的方程给出不变。矛盾的原因是，式 (42) 的推导假设了干物质不动，它的时间导数是在固定空间位置求的。收缩时干物质在移动，带着水分一起走，所以固定空间点上的含水率会因为材料流过来而改变。错误方程漏掉的正是这一部分变化。

为什么要这样做：这个错误为什么不容易发现

因为当初始含水率处处相同时，两种写法给出的结果都是常数，看起来都合理。只有在含水率不均匀时错误才会暴露。而实际干燥过程中含水率恰恰是不均匀的，所以错误的模型会给出错误的干燥时间。

本文没有给出偏差倍数的数值结论，误差的量级取决于收缩速率：收缩越快、半径变化越大，漏掉的对流项越重要。本文只用两条证据说明这一点。一条是后面给出的解析验证，也就是把物质坐标下的解代回固定半径坐标的方程，验证两种写法正好相差对流项。另一条就是本节这个扩散系数为零的检验，它说明固定半径坐标下的方程在含水率不均匀时给出的时间导数是错的。

9.3 正确的做法：跟着干物质走

既然干物质在动，那就把坐标系挂在干物质上。

引入物质坐标

$$\xi = \frac{r}{R(t)}. \quad (146)$$

这个坐标的含义是相对位置。当 $r = 0$ 时 $\xi = 0$ ，当 $r = R(t)$ 时 $\xi = 1$ 。不管药材怎么缩，圆心永远是 $\xi = 0$ ，表面永远是 $\xi = 1$ 。

关键性质是：对同一个材料点， ξ 永远不变。

验证一下。设某个材料点在 $t = 0$ 时位于半径 r_0 处，那么它的标签是 $\xi_0 = r_0/R_0$ 。由于收缩是各向同性的，也就是所有位置按同样的比例缩小，这个点在时刻 t 的位置是

$$r(t) = r_0 \cdot \frac{R(t)}{R_0}. \quad (147)$$

代入 ξ 的定义，

$$\xi = \frac{r(t)}{R(t)} = \frac{r_0 R(t)/R_0}{R(t)} = \frac{r_0}{R_0} = \xi_0. \quad (148)$$

分子分母里的 $R(t)$ 约掉了，结果与时间无关。所以 ξ 确实是一个不随时间变化的标签。

为什么要这样做：各向同性这个假设为什么必须写明

如果药材在长度方向不缩、只在半径方向缩，那么材料点的运动就不是简单的按比例缩小， $r(t)$ 和 $R(t)$ 的比值就不再是常数，上面的推导就不成立，方程会多出额外的项。题目只给了半径方向的收缩数据，按各向同性处理是同口径的做法，论文里明确写成了假设。

9.4 干物质守恒给出的一个重要关系

现在对一个材料壳层做干物质衡算。

取标签在 ξ 到 $\xi + d\xi$ 之间的那一层材料。它在当前构形下的体积（单位长度）是多少？

径向位置是 $r = \xi R$ ，厚度是 $dr = R d\xi$ 。体积（单位长度）等于周长乘以厚度，

$$dV = 2\pi r dr = 2\pi \xi R \cdot R d\xi = 2\pi R^2 \xi d\xi. \quad (149)$$

这个壳层里的干物质质量是

$$dm_d = \rho_d dV = \rho_d 2\pi R^2 \xi d\xi. \quad (150)$$

干物质不会迁移，也不会凭空产生或消失，所以这个质量不随时间变化：

$$\frac{\partial}{\partial t} (\rho_d 2\pi R^2 \xi d\xi) = 0. \quad (151)$$

其中 ξ 和 $d\xi$ 都是常数， 2π 也是常数，约掉以后得到

$$\frac{\partial}{\partial t} (\rho_d R^2) = 0 \implies \rho_d(\xi, t) R(t)^2 = \rho_d(\xi, 0) R_0^2. \quad (152)$$

这个关系的含义是：干物质被压缩了。半径缩小时体积变小，同样的干物质挤在更小的空间里，所以干物质密度变大。

再进一步。初始含水率处处相同，所以初始时刻的干物质密度 $\rho_d(\xi, 0)$ 与 ξ 无关，是一个常数。代入式 (152) 得到

$$\rho_d(\xi, t) = \rho_{d0} \frac{R_0^2}{R(t)^2}. \quad (153)$$

右边只依赖时间，不依赖 ξ 。这个结论非常重要：物质坐标下的干物质密度处处相同。

为什么要这样做：这一点为什么关键

因为它意味着在后面的推导中， ρ_d 可以当作常数从导数符号里提出来约掉，方程的形式会变得非常简单。如果 ρ_d 随位置变化，方程里会多出一阶导数项，处理起来麻烦得多。

9.5 水分守恒的完整推导

现在对材料区域 $[0, \xi]$ ，也就是从圆心到这个标签之间的全部材料，做水分衡算。

这个区域里的水分总量是多少？含水率 C 的定义是每千克干物质含多少水，所以水分总量等于

$$W = \int_0^\xi C \, dm_d = \int_0^\xi C \rho_d 2\pi R^2 \xi' \, d\xi'. \quad (154)$$

水分总量的变化率是

$$\frac{dW}{dt} = \int_0^\xi \frac{\partial C}{\partial t} \rho_d 2\pi R^2 \xi' \, d\xi'. \quad (155)$$

注意这里可以把偏导数拿进积分号里面，因为积分限 ξ 是固定的材料标签，不随时间变化。这正是用物质坐标的好处。

接下来算水分从哪里进来。

水分只能通过边界 ξ 处的那个材料面进出，因为 $\xi = 0$ 那端是圆心，没有面积。

在物质坐标下，水分扩散通量怎么表示？从半径坐标的菲克定律 $j_r = -D \frac{\partial C}{\partial r}$ 出发。用链式法则换算到 ξ ，

$$\frac{\partial C}{\partial r} = \frac{\partial C}{\partial \xi} \cdot \frac{\partial \xi}{\partial r} = \frac{\partial C}{\partial \xi} \cdot \frac{1}{R}, \quad (156)$$

最后一个等号用了 $\xi = r/R$ ，所以 $\frac{\partial \xi}{\partial r} = \frac{1}{R}$ 。

于是

$$j_r = -D \frac{\partial C}{\partial r} = -\frac{D}{R} \frac{\partial C}{\partial \xi}. \quad (157)$$

注意 $j_r = -D \frac{\partial C}{\partial r}$ 的单位是米每秒。原因是干基含水率 C 是一个比值，单位是千克水每千克干物质，它对 r 求导以后的单位是每米，再乘上单位是平方米每秒的 D ，得到的就是米每秒。这个量乘上干物质密度 ρ_d 以后，单位才是千克每平方米每秒。

在物质坐标下做衡算时，用带上干物质密度的通量更方便，也就是

$$j_\xi = -\frac{\rho_d D}{R} \frac{\partial C}{\partial \xi}, \quad (158)$$

它等于 j_r 乘上 ρ_d ，单位是千克每平方米每秒，含义是单位时间穿过单位面积的水的质量。

这里要说明一个记号上的差别。第一部分的式 (36) 里，通量写成 $j_r = -\rho_d D \frac{\partial C}{\partial r}$ ，把 ρ_d 含在里面，单位是千克每平方米每秒。本节为了做坐标换算方便，先把不带 ρ_d 的那个通量记成 j_r ，它的单位是米每秒，两者相差一个 ρ_d 因子。

材料面 ξ 处的面积（单位长度）是多少？径向位置是 $r = \xi R$ ，周长是 $2\pi r = 2\pi \xi R$ 。所以单位时间内流出这个区域的水分是

$$\text{流出} = j_\xi \cdot 2\pi \xi R = -\frac{\rho_d D}{R} \frac{\partial C}{\partial \xi} \cdot 2\pi \xi R = -2\pi \xi \rho_d D \frac{\partial C}{\partial \xi}. \quad (159)$$

流入等于流出的负值，

$$\text{流入} = 2\pi \xi \rho_d D \frac{\partial C}{\partial \xi}. \quad (160)$$

把式 (155) 和上式相等，

$$\int_0^\xi \frac{\partial C}{\partial t} \rho_d 2\pi R^2 \xi' d\xi' = 2\pi \xi \rho_d D \frac{\partial C}{\partial \xi}. \quad (161)$$

这一步就是完整的水分守恒式。

9.6 从积分式得到微分方程

式 (161) 还是一个积分式，要化成微分方程。

做法是两边对 ξ 求导。左边用微积分基本定理：积分上限是 ξ ，对上限求导就是被积函数在上限处的值，所以

$$\frac{\partial}{\partial \xi} \left[\int_0^\xi \frac{\partial C}{\partial t} \rho_d 2\pi R^2 \xi' d\xi' \right] = \frac{\partial C}{\partial t} \rho_d 2\pi R^2 \xi. \quad (162)$$

右边求导得到

$$\frac{\partial}{\partial \xi} \left[2\pi \xi \rho_d D \frac{\partial C}{\partial \xi} \right] = 2\pi \frac{\partial}{\partial \xi} \left[\xi \rho_d D \frac{\partial C}{\partial \xi} \right]. \quad (163)$$

两边相等，

$$\frac{\partial C}{\partial t} \rho_d 2\pi R^2 \xi = 2\pi \frac{\partial}{\partial \xi} \left[\xi \rho_d D \frac{\partial C}{\partial \xi} \right]. \quad (164)$$

约掉两边的 2π ，再把 ρ_d 提出来。这里用到式 (153) 的结论，也就是 ρ_d 与 ξ 无关，所以可以从导数符号里提出来，

$$\frac{\partial C}{\partial t} \rho_d R^2 \xi = \rho_d \frac{\partial}{\partial \xi} \left[\xi D \frac{\partial C}{\partial \xi} \right]. \quad (165)$$

两边约掉 ρ_d ，再除以 $R^2 \xi$ ，

$$\boxed{\left. \frac{\partial C}{\partial t} \right|_{\xi} = \frac{1}{\xi R(t)^2} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial C}{\partial \xi} \right)}. \quad (166)$$

这就是物质坐标下的水分方程。

9.7 换成 u 坐标

式 (166) 里还有 ξ ，用起来和第一部分不一样。把它换成 $u = \xi^2$ ，就能和前面统一。

换算关系和第五章完全一样。由 $\xi = \sqrt{u}$ 得

$$\frac{\partial}{\partial \xi} = 2\sqrt{u} \frac{\partial}{\partial u}. \quad (167)$$

逐项改造式 (166) 的右边。

先算括号里面，

$$\xi D \frac{\partial C}{\partial \xi} = \sqrt{u} \cdot D \cdot 2\sqrt{u} \frac{\partial C}{\partial u} = 2uD \frac{\partial C}{\partial u}. \quad (168)$$

再求一次导，

$$\frac{\partial}{\partial \xi} \left(2uD \frac{\partial C}{\partial u} \right) = 2\sqrt{u} \frac{\partial}{\partial u} \left(2uD \frac{\partial C}{\partial u} \right). \quad (169)$$

最后除以 ξR^2 ，

$$\frac{1}{\sqrt{u}R^2} \cdot 2\sqrt{u} \frac{\partial}{\partial u} \left(2uD \frac{\partial C}{\partial u} \right) = \frac{2}{R^2} \frac{\partial}{\partial u} \left(2uD \frac{\partial C}{\partial u} \right) = \frac{4}{R^2} \frac{\partial}{\partial u} \left(uD \frac{\partial C}{\partial u} \right). \quad (170)$$

所以

$$\boxed{\frac{\partial C}{\partial t} = \frac{4}{R(t)^2} \frac{\partial}{\partial u} \left(uD \frac{\partial C}{\partial u} \right)}. \quad (171)$$

和式 (85) 比较一下，两者的形式完全一样，差别只有一个： R 变成了 $R(t)$ ，随时间变化。

温度方程同理，

$$\rho c_p \frac{\partial T}{\partial t} = \frac{4}{R(t)^2} \frac{\partial}{\partial u} \left(uk \frac{\partial T}{\partial u} \right). \quad (172)$$

9.8 一个重要的结论：对流项为零

对比一下两种写法。

在固定半径坐标下，正确的方程是要补一项的，

$$\left. \frac{\partial C}{\partial t} \right|_r = \frac{1}{r} \frac{\partial}{\partial r} \left(rD \frac{\partial C}{\partial r} \right) - \frac{\dot{R}}{R} r \frac{\partial C}{\partial r}. \quad (173)$$

最后那一项叫对流项，描述干物质带着水分移动的效应。其中 $\dot{R}$ 表示 dR/dt 。收缩时半径不断减小，所以 $\dot{R}$ 是负的；收缩速率要取它的负值，也就是 $-\dot{R}$ ，这样才是正数。

在物质坐标下，这一项消失了，如式 (171) 所示。原因是控制体跟着材料一起走，材料本身不迁移，所以不需要额外描述它的运动。

为什么要这样做：哪一种写法更好

物质坐标的写法更好，理由有两条。第一，方程更短，少一个需要离散的项，程序里不容易出错。第二，也是更重要的，求解区域是固定的 $0 \leq u \leq 1$ ，不需要处理移动的网格。如果用式 (173)，每走一步网格都要重新分布，还要处理网格移动带来的额外误差。

为什么要这样做：怎么验证两种写法是等价的

可以做一个代换检验。设式 (171) 成立，把 $\xi = r/R(t)$ 代进去，利用链式法则算出 $\left. \frac{\partial C}{\partial t} \right|_r$ ，应该正好得到式 (173)。

具体计算如下。设 $C(r, t) = \tilde{C}(\xi, t)$ ，其中 $\xi = r/R(t)$ 。固定 r 对 t 求导，

$$\left. \frac{\partial C}{\partial t} \right|_r = \left. \frac{\partial \tilde{C}}{\partial t} \right|_\xi + \frac{\partial \tilde{C}}{\partial \xi} \left. \frac{\partial \xi}{\partial t} \right|_r. \quad (174)$$

其中

$$\left. \frac{\partial \xi}{\partial t} \right|_r = \frac{\partial}{\partial t} \left(\frac{r}{R(t)} \right) = -\frac{r\dot{R}}{R^2} = -\frac{\xi\dot{R}}{R}. \quad (175)$$

把式 (166) 的右边代入第一项，再把上面的结果代入第二项，

$$\left. \frac{\partial C}{\partial t} \right|_r = \frac{1}{\xi R^2} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial \tilde{C}}{\partial \xi} \right) - \frac{\xi \dot{R}}{R} \frac{\partial \tilde{C}}{\partial \xi}. \quad (176)$$

现在把 ξ 的导数换回 r 的导数。由 $\xi = r/R$ 得 $\frac{\partial}{\partial \xi} = R \frac{\partial}{\partial r}$ 。

第一项变成

$$\frac{1}{(r/R)R^2} \cdot R \frac{\partial}{\partial r} \left(\frac{r}{R} D \cdot R \frac{\partial C}{\partial r} \right) = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (177)$$

第二项变成

$$-\frac{(r/R)\dot{R}}{R} \cdot R \frac{\partial C}{\partial r} = -\frac{\dot{R}}{R} r \frac{\partial C}{\partial r}. \quad (178)$$

两项合起来正好是式 (173)。两种写法确实等价。

9.9 输出表格时的坐标换算

求解器工作在 u 坐标上，而题目要求的表格列名是当前物理距离 r 。这两者不是一回事，必须换算。

换算关系由 $r = \xi R(t)$ 和 $u = \xi^2$ 联立得到

$$u = \left(\frac{r}{R(t)} \right)^2. \quad (179)$$

注意分母是当前的半径 $R(t)$ ，不是初始半径 R_0 。

还有一个情况要处理：如果某一系列的距离大于当前半径，也就是 $r > R(t)$ ，那么这个位置已经不存在了。例如 $r = 1.5$ 厘米这一列，当 $t > 3.65$ 小时之后半径已经小于 1.5 厘米，药材外面没有东西，表格里应该留空。

为什么要这样做：如果换算错了会有什么后果

后果很严重。举一个实际例子。

在 $t = 6$ 小时，半径是 $R = 1.374$ 厘米。如果要查 $r = 1.0$ 厘米处的含水率。

正确的算法是

$$u = \left(\frac{1.0}{1.374} \right)^2 = 0.5297. \quad (180)$$

如果误用初始半径，算成

$$u = \left(\frac{1.0}{2.0} \right)^2 = 0.25, \quad (181)$$

那就完全错了。 $u = 0.25$ 对应的是 $t = 0$ 时刻位于半径 1.0 厘米处的那个材料点，而现在它已经收缩到 $1.0 \times \frac{1.374}{2.0} = 0.687$ 厘米处了。

两种算法给出的数值分别是 1.0215 和 1.3782，相对误差百分之三十五。

这一节要记住的 问题四的关键有三点。第一，收缩时不能用固定半径坐标下的原方程，会漏掉对流项。第二，要用物质坐标 $\xi = r/R(t)$ ，方程里对流项自动为零。第三，输出表格时要用当前半径换算坐标， r 超过当前半径的位置要留空。

Part III

代码逐行解释

前两部分讲清了原理。这一部分把原理和代码对应起来，逐段说明每一行在做什么，以及为什么必须这么写。

阅读这一部分之前，建议先打开交付包里的 `code/` 目录，对照着看。本部分涉及的代码行数都按最终交付版本标注。

10 代码总览

10.1 文件分工

整套代码一共 13 个 Python 文件，分成三类。求解器负责算出结果；对照格式负责用完全不同的方法再算一遍；驱动脚本负责调用它们、输出表格和图。

文件	类型	职责
<code>common.py</code>	基础	全部参数、四套物性公式、附件读取、环境标定、Bessel 特征根
<code>console_utf8.py</code>	基础	把控制台输出切成 UTF-8，否则中文进度是乱码
<code>spectral.py</code>	求解器	主求解器： $u = \xi^2$ 谱配置加自适应隐式 BDF，四个问题共用
<code>analytic_p1.py</code>	求解器	问题一温度场的闭式解，用来给主求解器当标尺
<code>fv_theta.py</code>	求解器	对照格式一：节点中心有限体积加定步长 θ 法
<code>fv_uniform.py</code>	求解器	对照格式二：守恒型有限体积加后向欧拉加 Picard 迭代
<code>run_all.py</code>	驱动	一键复跑全部结果与验证，写出复跑报告
<code>make_results.py</code>	驱动	生成四个结果文件 <code>result1</code> 到 <code>result4.xlsx</code>
<code>sensitivity.py</code>	驱动	12 组参数扰动，外加环境口径的分叉对比
<code>verify_data.py</code>	驱动	四类扫描，写 <code>verification.json</code> ，并做一致性校验
<code>fig_core.py</code>	驱动	统一绘图样式与出图入口，缓存四个问题的解
<code>make_figures.py</code>	驱动	生成论文的 25 张插图
<code>make_controlled_comparison.py</code>	驱动	补算受控算例，画物性与收缩的效应分解图

先解释这张表里最容易让人困惑的一点：为什么画图要两个文件。`fig_core.py` 负责三件事：设定中文字体与字号、把出图分辨率钉死在一个入口、缓存四个问题的解；`make_figures.py` 才是真正一张一张画图的地方。把样式和解缓存抽出来，是为了让每一张图都走同一个入口，不会出现某些图忘了设分辨率、某些图字体不对的情况。

为什么用三套求解器。`spectral.py` 是主力，另外两套用完全不同的方法算同一个问题。它们的价值不在于更快或更准，而在于当三套方法给出一致结果时，说明它们解的是同一个方程，而不是三份代码犯了同一个错误。

`fv_theta.py` 用节点中心有限体积配定步长 θ 法：取 $\theta = \frac{1}{2}$ 是 Crank–Nicolson，时间二阶；前 4 步改用 $\theta = 1$ 即后向欧拉，这叫 Rannacher 启动，用来压掉初值不光滑引

起的振荡；物性在整个时间步内冻结在步首，不再更新。`fv_uniform.py` 用守恒型有限体积配等步长后向欧拉，时间一阶；非线性物性用 Picard 迭代处理，每个时间步里把物性按新解重算并重新组装，一共迭代 3 次。于是三套方法在坐标、空间离散、时间格式、非线性处理四个维度上全都不同，这正是交叉验证需要的。

为什么要这样做：为什么要写三个求解器

因为要判断结果对不对。一个程序跑出来的数字，如果自己写错了公式，它也不会报错，只会给出一个看起来正常的错数。唯一的办法是用另一种完全不同的方法算同一个问题，如果两者结果一致，那么同时写错的可能性就很小。这叫做交叉验证。

10.2 为什么把参数单独放一个文件

如果每个脚本里都写一遍参数数值，改一个数就要改很多地方，很容易漏改，导致不同脚本用的参数不一样，结果对不上。把所有参数集中在 `common.py` 里，其他脚本都从这里取，就避免了这个问题。

具体到本题，`common.py` 里集中了这些东西：药材初始半径与长度、对流换热与传质系数、初值温度与含水率、烘房环境的一阶惯性拟合结果、四套物性经验式、附件 1 与附件 2 的读取函数、Bessel 特征根的求解函数。

其中附件路径的查找尤其能说明「集中」的好处。代码不写死路径，而是在四个候选目录里按文件名搜索：

```
_HERE = os.path.dirname(os.path.abspath(__file__))
_CANDIDATES = [
    os.path.join(_HERE, "..", "题目"),
    os.path.join(_HERE, ".."),
    os.path.join(_HERE, "..", ".."),
    _HERE,
]
```

上面这六行是 `common.py` 的第 38 行到第 44 行。`__file__` 是 Python 的一个内置变量，表示当前这个文件自己的路径。`os.path.abspath` 把它变成绝对路径，`os.path.dirname` 取出所在目录。`os.path.join(_HERE, "..", "题目")` 表示从当前目录往上退一级，再进入题目目录... 在路径里的含义就是上一级。四个候选依次是「包根的题目目录」「包根」「上一级」「本目录」，覆盖了压缩包原样解压与只取 `code/` 单独运行两种用法。

这样做之后，整个交付包可以随便移动位置、随便改名，只要内部的相对结构不变就能直接跑。写死绝对路径的代码，换一台电脑就废了。

10.3 数据是怎样从附件走到论文表格的

把整条链条走一遍，后面每一段代码在看的时候就都有位置了。

1. `common._load_env()` 读附件 1 的 241 行实测数据，用最小二乘拟合成一阶惯性式，得到烘房温度与含湿量随时间的两条曲线。
2. `common.props(prob, C, T)` 按 `prob` 的值从四套物性经验式里选一套，返回密度、比热、导热系数、扩散系数四个数组。
3. `spectral.HerbSolver` 把温度场与水分场拼成一个长度为 $2(N + 1)$ 的向量交给积分器，解出每个节点上两个场随时间的演化。
4. `interp_u()` 把节点上的解插值到任意目标坐标；`area_weights()` 给出截面平均的积分权重。
5. 三条出口：`make_results.py` 写成 `xlsx`；`fig_core.build_cache()` 写成解缓存供出图；`run_all.py` 写成文本报告供人工核对。

这一节要记住的 代码组织的原则是：参数集中存放，求解器与驱动分离，关键结果用多个独立方法验证。三条出口共用同一份解，所以论文里的数、结果文件里的数、图上的数只有一个来源。

11 主求解器 `spectral.py` 逐段解释

11.1 文件头部与导入

```
# -*- coding: utf-8 -*-
#
# =====
# spectral.py —— 主求解器：u=(r/R)^2 坐标下的 Chebyshev 谱配置 + 自适应隐式
# 时间积分
#
# =====
# 做什么：在"径向坐标平方" u=(r/R)^2 上同时求解水分浓度 C 与温度 T 两个耦合场，
# 四问共用。
... (docstring 的其余各行在这里省略，内容见下文)
```

```
import numpy as np
from scipy.integrate import solve_ivp
import common as cm
```

开头三段。第 1 行是编码声明，告诉 Python 这个文件用 UTF-8，这样文件里的中文注释才不会乱码。第 2 行到第 21 行是模块的 docstring，也就是一段说明文字，外层用三个连续的双引号括起来。它在程序运行时只是一段字符串，不参与任何计算，但可以被文档工具自动收集。最后三行是 import：numpy 提供数组，solve_ivp 是 SciPy 的自适应变阶积分器，common 是我们自己写的参数文件，起个别名 cm，后面写 cm.R0 就能取到半径。

清单里的省略号代表 docstring 里被省掉的十几行，它们按「做什么、输入、输出、关键变量」四段写，这里逐条说清。

- 坐标：取 $u = (r/R)^2$ 为自变量，使圆柱的拉普拉斯算子在 $u = 0$ 处完全正则，不含 $1/u$ 的奇异项，轴心与表面都不需要任何特殊的控制体处理。这是本方法与常规 r 坐标离散最核心的差别。
- 空间：切比雪夫-高斯-洛巴托谱配置，谱精度、全局耦合；解在 u 上解析，所以谱收敛成立。
- 时间：自适应变阶隐式 BDF（即 scipy 的 solve_ivp），物性在每个右端项求值处重新计算，误差由 rtol 与 atol 直接控制；另外提供 Radau 与 LSODA 供交叉验证。
- 边界：以通量形式配置。内部节点的通量由谱微分给出，表面节点的通量直接取 Robin 条件的物理值，因此边界条件以物理通量的身份精确进入离散方程。

为什么要这样做：docstring 里为什么要把参数和单位写全

因为这份 docstring 按「做什么、输入、输出、关键变量」四段写，里面每一条都带单位，例如 rtol 是相对容差、atol 是绝对容差，以及 C 的单位 kg/kg 与 T 的单位摄氏度纲不同这件事。这些约定如果不写下来，过几天再回来看代码就得重新推一遍。特别是 atol 要按状态分组传数组这件事，不写清楚就会有人把两个场用同一个容差，白白浪费计算量。

11.2 函数 cheb_matrices：构造微分矩阵

文件第 27 行到第 42 行，函数 cheb_matrices 的全部内容：

```
def cheb_matrices(N):
    """构造 CGL 节点  $x_j = \cos(j\pi/N)$ （自 1 降到 -1）上的 Chebyshev 微分矩阵。
```

```

N 为多项式阶数（节点共 N+1 个）；返回 (D, D@D, x)：D 为一阶微分矩阵，
二阶直接取 D@D——在 CGL 节点上用一阶矩阵平方仍给出精确的谱微分矩阵，
不必另推二阶系数（二阶矩阵随结果一并给出，当前只用一阶）。
"""
x = np.cos(np.pi * np.arange(N + 1) / N)
c = np.hstack([2.0, np.ones(N - 1), 2.0]) * ((-1.0) ** np.arange(N + 1)
)
X = np.tile(x, (N + 1, 1)).T
dX = X - X.T
D = np.outer(c, 1.0 / c) / (dX + np.eye(N + 1))
# 上面的公式在 i=j 处是 0/0，对角元改用"本行其余元素之和取负"：
# 这样 D 作用在常向量上严格得到 0（常数的导数为零），不吃浮点残差。
D -= np.diag(D.sum(axis=1))
return D, D @ D, x

```

这个函数实现第二部分第六章里的式 (93)、式 (103) 和式 (105)。逐行说明。

第 34 行

```
x = np.cos(np.pi * np.arange(N + 1) / N)
```

`np.arange(N+1)` 生成数组 $[0, 1, 2, \dots, N]$ 。乘以 π 再除以 N ，得到 $[0, \pi/N, 2\pi/N, \dots, \pi]$ 。最后取余弦得到 $x_j = \cos(j\pi/N)$ ，这正是式 (93) 的定义。注意这个数组是从 1 递减到 -1 的，因为 $\cos 0 = 1$ ， $\cos \pi = -1$ 。

第 35 行

```
c = np.hstack([2.0, np.ones(N - 1), 2.0]) * ((-1.0) ** np.arange(N + 1))
```

`np.ones(N-1)` 生成 $N-1$ 个 1。`np.hstack` 把三部分横向拼起来，得到 $[2, 1, 1, \dots, 1, 2]$ ，长度是 $N+1$ ，对应微分矩阵公式里 $c_0 = c_N = 2$ 、中间都是 1 的规定。后面乘上 $(-1)^j$ 得到带符号的系数。这样做是为了把公式分子上的 $(-1)^{i+j}$ 收进系数里： c_i 含 $(-1)^i$ ， c_j 含 $(-1)^j$ ，又因为 $1/(-1)^j$ 等于 $(-1)^j$ ，所以下一步算出的商 c_i/c_j 正好给出 $(-1)^{i+j}$ ，符号一次到位。

第 36 行到第 38 行

```

X = np.tile(x, (N + 1, 1)).T
dX = X - X.T
D = np.outer(c, 1.0 / c) / (dX + np.eye(N + 1))

```

`np.tile` 把向量 x 复制成 $N+1$ 行，得到矩阵的每一行都等于 x ，再取转置，于是 X 的第 i 行全是 x_i ，写成元素形式是 $X_{ij} = x_i$ 。`X.T` 是转置，它的第 i 行全是 x_j ，两者

相减得到

$$dX_{ij} = x_i - x_j. \quad (182)$$

这正是式 (103) 分母里需要的 $x_i - x_j$ ，符号完全一致。

`np.outer(c, 1.0/c)` 生成矩阵，第 i 行第 j 列等于 c_i/c_j ，是式 (103) 的第一个因子。把它按元素除以 dX 加单位矩阵，就得到 D ：当 i 不等于 j 时

$$D_{ij} = \frac{c_i/c_j}{x_i - x_j}, \quad (183)$$

与式 (103) 完全一致。

为什么要这样做：为什么要加 `np.eye(N+1)`

原公式在 $i = j$ 处是 $0/0$ ，直接算会得到 `nan`。给分母加单位阵，对角位置的分母被强行变成 1，算出来的对角元素是个无关紧要的值，反正下一行会覆盖它，所以除法不会出错。用矩阵运算还有一个好处：一个转置相减就得到全部两两差值，不需要写双重循环，这是 `numpy` 的广播机制，也是科学计算代码的典型写法。

第 41 行

```
D -= np.diag(D.sum(axis=1))
```

`D.sum(axis=1)` 把每一行的元素求和，得到一个长度 $N + 1$ 的向量。此时对角线上的值是 1，所以第 i 行的和等于 1 加上该行其余元素之和。`np.diag` 把这个和放到一个对角矩阵的对角线上，`D -=` 表示从 D 中减去这个对角矩阵。相减以后，对角线上的那个 1 被换成该行其余元素之和的相反数，因为

$$1 - \left(1 + \sum_{j \neq i} D_{ij}\right) = - \sum_{j \neq i} D_{ij}, \quad (184)$$

而不在对角线上的元素没有变动。于是第 i 行的和变成

$$\sum_{j \neq i} D_{ij} + \left(- \sum_{j \neq i} D_{ij}\right) = 0, \quad (185)$$

这正是式 (105) 要求的行和为零。

对角元素不用公式算，而是取本行其余元素之和的相反数，数学上这是标准做法，但代码这么写还有一个额外好处：它保证 D 作用在常向量上严格得到 0，也就是常数的导数为零在浮点意义下也成立。如果不这么做，一个常数场会被慢慢「积分」出虚假的梯度。

为什么要这样做：为什么不直接算出对角线再填进去

减对角矩阵这个写法同时完成了修正和清零两件事，一行代码解决，而且不需要把对角线元素单独挑出来处理。

第 42 行返回三个值：一阶微分矩阵 D 、它的平方 D^2 （也就是 D 与自身相乘）、以及节点数组 x 。注意第二个返回值在整个项目里从未被用到——这是因为下面要讲的通量形式只需要两次一阶微分，永远不需要显式构造二阶矩阵。直接写 D^2 的元素量级达到 $O(N^4)$ ，舍入误差比两次一阶应用更大，能不用就不用。

11.3 类 HerbSolver 的初始化

文件第 55 行到第 65 行：

```
def __init__(self, N, prob, shrink=False):
    self.N = N
    self.Np = N + 1
    self.prob = prob
    self.shrink = shrink
    # CGL 节点  $x_j = \cos(j\pi/N)$  自 1 降到 -1, 与  $u$  的换元关系是  $x = 1 - 2u$ ,
    # 故  $u$  自  $u_0 = 0$  (轴心) 升到  $u_N = 1$  (表面), 且  $d/du = -2 d/dx$ 。
    Dx, D2x, x = cheb_matrices(N)
    self.u = (1.0 - x) / 2.0
    self.Du = -2.0 * Dx
    self.Np = N + 1
```

这个类把四个问题需要的东西全部装在一起。 N 是谱节点数， N_p 是节点总数 $N + 1$ (N 表示阶数，节点比它多一个，所以专门起个名字避免每次写 $N + 1$ 写错)， $prob$ 决定用哪一套物性， $shrink$ 决定半径是否随时间变化，只有问题四才设为真。

后三行是坐标映射。`cheb_matrices` 给出的 x 从 1 递减到 -1，而我们要的是 u 从 0 递增到 1，所以取

$$u = \frac{1 - x}{2}, \quad (186)$$

这就是式 (95) 的映射，结果满足 $u_0 = 0$ 、 $u_N = 1$ ，对应圆心和表面。两边求导得 $\frac{du}{dx} = -\frac{1}{2}$ ，于是 $\frac{d}{du} = -2\frac{d}{dx}$ ，这就是第 64 行 `self.Du = -2.0 * Dx` 的来历，也就是式 (107)。

为什么要这样做：为什么用一个类而不是一堆函数

因为四个问题共用同一套离散，差别只在物性和几何。把这些差异放进构造函数的两个参数里，后面的求解、插值、求平均全部复用同一份代码。如果用函数，就得把 N 、 $prob$ 、 $shrink$ 一路当参数传下去，每个函数签名都会变长。

为什么要这样做：为什么把微分矩阵存起来而不是每次重算

因为它在整个求解过程中不变，算一次就够了。存下来可以节省时间。这叫空间换时间。

11.4 几何与环境：三个小方法

文件第 70 行到第 80 行：

```
def R_of(self, t):
    """当前半径 R(t) [m]: shrink=True 时取附件 2 实测半径插值，否则恒为 cm
    .R0。"""
    return float(cm.R_of(t)) if self.shrink else cm.R0

def T_air(self, t):
    """烘房空气温度 T_air(t) [degC]: t <= 14400 s 用一阶惯性式，其后恒为
    Tset。"""
    return float(cm.T_air(t))

def C_air(self, t):
    """烘房空气含湿量 C_air(t) [kg/kg]: 分段口径与 T_air 相同（同一 T_END
    ）。"""
    return float(cm.C_air(t))
```

三个方法都把外部的函数包一层，返回 Python 的浮点数而不是数组。**shrink** 为真时半径取附件 2 的插值，否则恒为初始半径。

为什么要这样做：为什么要 float() 一下

积分器在每一步都会调用右端项，如果里面返回的是零维数组，后面和标量混算时形状容易出问题。显式转成浮点数可以避免这类隐蔽的错误。

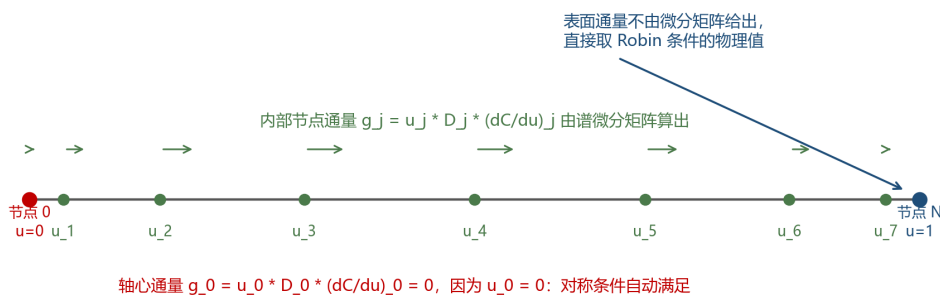


图 6: 通量形式的谱配置。内部节点的通量由谱微分矩阵算出，表面节点的通量直接取自 Robin 边界条件的物理值，轴心节点的通量因为 u 等于零而自动为零。

11.5 右端项函数：核心中的核心

文件第 84 行到第 111 行，函数 `rhs` 的全部内容：

```
def rhs(self, t, y):
    """右端项  $dy/dt$ ，供 solve_ivp 调用； $t$  [s]， $y$  为状态向量（含义见类说明）。

    返回与  $y$  同形状的导数： $dC/dt$  [kg/(kg s)] 与  $dT/dt$  [degC/s]。
    """
    N, Np = self.N, self.Np
    C = y[:Np]
    T = y[Np:]
    R = self.R_of(t)
    rho, cp, k, D = cm.props(self.prob, C, T)

    # 水分：通量形式  $g = u * D * dC/du$ ，方程即  $dC/dt = (4/R^2) dg/du$ 
    gC = self.u * D * (self.Du @ C)
    # 表面节点不取谱微分算出的通量，而由 Robin 条件  $-D dC/dr = hm(C_s - C_{air})$ 
    # 除以  $dr/du = R/(2 \sqrt{u})$  反解： $g_N = (R/2) * hm * (C_{air} - C_N)$ ，
    # 于是边界条件以物理通量的形式精确进入，不引入谱微分的边界误差。
    gC[N] = 0.5 * cm.hm * R * (self.C_air(t) - C[N]) # Robin 物理通量

    dC = (4.0 / R ** 2) * (self.Du @ gC)

    # 温度：同上，只是扩散系数换成  $k$ ，且被体积热容  $\rho * cp$  除
    gT = self.u * k * (self.Du @ T)
```

```

gT[N] = 0.5 * cm.h * R * (self.T_air(t) - T[N])
dT = (4.0 / (R ** 2 * rho * cp)) * (self.Du @ gT)

out = np.empty_like(y)
out[:Np] = dC
out[Np:] = dT
return out

```

rhs 是 right hand side 的缩写，意思是方程的右端，它做的事情是：给定当前时刻 t 和当前状态 y ，算出状态的变化率 $\frac{dy}{dt}$ 。积分器在每一步会反复调用这个函数。这个函数把两个场的方程都写完了，也就是整个求解最核心的部分。

状态向量的长相。第 90 行和第 91 行把 y 切开：

```

C = y[:Np]
T = y[Np:]

```

前 $N + 1$ 个分量是水分浓度 C ，后 $N + 1$ 个是温度 T ，两个场共用同一组节点。用切片而不是复制，是为了省内存；这里不会修改 C 和 T ，只读它们。

为什么要这样做：为什么把两个未知量放进同一个数组

因为积分器只能处理一个未知向量。把含水量和温度首尾相接拼成一个长向量，就能用同一个积分器同时推进两个场，而且能自动处理两者之间的耦合。

物性每步重算。第 92 行和第 93 行取当前半径、算四个物性：

```

R = self.R_of(t)
rho, cp, k, D = cm.props(self.prob, C, T)

```

第一行取当前半径。问题一到问题三里它恒等于 R_0 ，问题四里它按附件二的数据插值得到。第二行调用 `common.py` 里的 `props` 函数，根据当前的含水率和温度算出四个物性，一次返回四个值，这叫元组解包。

这一行看起来很普通，却是本题非线性处理的关键：物性依赖当前的 C 和 T ，而 C 、 T 又是未知量，所以每调用一次右端项就用当前状态重新算一遍物性，而不是冻结在某一步的值上。

为什么要这样做：为什么不写牛顿迭代

因为时间方向用的是隐式 BDF，`solve_ivp` 内部每一步都要解一个隐式方程，那个过程本身就包含迭代。我们只要把右端项写对，非线性它自己会处理。对照一下 `fv_uniform.py`：那里用后向欧拉，就必须自己写 Picard 迭代。同样是隐式，不同积分器的「非线性归谁管」是不同的。

为什么要这样做：物性为什么在每个右端项求值处重新计算

因为物性依赖未知量 C 和 T 。如果只在每一步开始时算一次，整个步内都当常数用，就相当于把方程线性化了，精度会下降。每次求值都重算，虽然麻烦一点，但积分器在每一步内部本来就会多次调用右端项，这样相当于做了隐式迭代，精度更高。

水分场的通量形式。第 96 行到第 101 行：

```
gC = self.u * D * (self.Du @ C)
gC[N] = 0.5 * cm.hm * R * (self.C_air(t) - C[N])          # Robin 物理通量
dC = (4.0 / R ** 2) * (self.Du @ gC)
```

先解释通量这个概念。第二部分第五章的式 (85) 里，空间部分写成

$$\frac{4}{R^2} \frac{\partial}{\partial u} \left(u D \frac{\partial C}{\partial u} \right). \quad (187)$$

如果记

$$g(u) = u D \frac{\partial C}{\partial u}, \quad (188)$$

那么整个空间项就是 $\frac{4}{R^2} \frac{\partial g}{\partial u}$ 。这个 g 叫通量，它表示单位时间通过单位截面的物质质量。

把这个式子离散化，要做两件事：第一，在每个节点上算出通量 g 的值；第二，对通量再求一次导数。

第 96 行 `gC = self.u * D * (self.Du @ C)` 完成第一件事。这一行算的是通量，不是散度。`self.Du @ C` 表示矩阵乘向量，符号 `@` 是 Python 的矩阵乘法运算符，这一步算出所有节点上 $\frac{\partial C}{\partial u}$ 的近似值，这正是式 (101) 的作用。`self.u * D * (...)` 是两个逐元素乘法：`self.u` 是节点坐标数组，`D` 是各节点上的扩散系数数组，逐元素相乘就是让第 j 个节点上的 u_j 、 D_j 和导数值相乘，得到该节点的通量 g_j ，这正是式 (188)。

第 100 行单独修改最后一个节点，也就是表面的通量。

为什么要这样做：为什么表面的通量不能直接用上面的公式算

因为上面的公式算的是插值多项式的导数。在表面处，物理条件式 (91) 告诉我们通量必须等于

$$g_N = D \left. \frac{\partial C}{\partial u} \right|_{u=1} = \frac{h_m R}{2} (C_{\text{air}} - C_1). \quad (189)$$

推导过程是：在 $u = 1$ 处 $\frac{du}{dr} = \frac{2}{R}$ ，所以 $\frac{dC}{du} = \frac{R}{2} \frac{dC}{dr}$ ；Robin 条件说 $-D \frac{dC}{dr} = h_m (C_s - C_{\text{air}})$ 。两式合起来

$$g_N = uD \frac{\partial C}{\partial u} \Big|_{u=1} = 1 \cdot D \cdot \frac{R}{2} \cdot \left(-\frac{h_m}{D} (C_s - C_{\text{air}}) \right) = \frac{R}{2} h_m (C_{\text{air}} - C_s). \quad (190)$$

注意 D 被精确约掉了。这是一个很干净的化简：物理通量里含 D （由 $dC/dr = -h_m(C_s - C_{\text{air}})/D$ 引入），而通量的定义里也含一个 D ，两者相消，最终表达式 $\frac{1}{2}h_m R(C_{\text{air}} - C_s)$ 里根本不出现 D 。这就避免了「在表面取一个代表性的 D 」这种近似。代码里写的是 `0.5 * cm.hm * R * (C_air(t) - C[N])`，和上式完全一致。这样写的好处是边界条件以物理量的身份精确进入离散方程，而不是通过外推近似得到。

第 101 行 `dC = (4.0 / R ** 2) * (self.Du @ gC)` 完成第二件事。再对通量做一次谱微分，就得到散度，乘上前面的系数 $\frac{4}{R^2}$ 即得 dC/dt 。这里发生了两件事：谱微分矩阵作用在通量上，用的是同一个 `self.Du`；而表面那个被替换过的值也参与这次微分。这个替换为什么仍然是谱精度：解收敛时，插值多项式的通量本来就等于物理通量，两者之差指数衰减，而微分矩阵的放大是多项式量级，指数赢过多项式。`R ** 2` 是 Python 的幂运算，表示 R^2 。

为什么要这样做：圆心处要不要也单独处理

不需要。因为代码里 `self.u` 的第一个元素是 0，所以第 96 行算出来的 g_0 自动等于零。而 $g_0 = 0$ 正好就是对称条件的含义，正如第二部分第五章最后一节所说。

温度场完全同构。第 104 行到第 106 行：

```
gT = self.u * k * (self.Du @ T)
gT[N] = 0.5 * cm.h * R * (self.T_air(t) - T[N])
dT = (4.0 / (R ** 2 * rho * cp)) * (self.Du @ gT)
```

三行与水分那三行逐项对应，对应式 (86)，只换了三处：扩散系数 D 换成导热系数 k ，传质系数 h_m 换成换热系数 h ，环境值 C_{air} 换成 T_{air} 。另外多除以一个体积热容 ρc_p ，因为能量方程的左边是 $\rho c_p \frac{\partial T}{\partial t}$ 而不是 $\frac{\partial T}{\partial t}$ ，移项以后变成 $\frac{\partial T}{\partial t}$ 等于右端除以 ρc_p 。表面条件用的是式 (90)，代码里写成 `0.5 * cm.h * R * (T_air(t) - T[N])`，和式 (90) 完全一致。

这种「同一个算子套两组系数」的写法，使得轴心与表面都不需要任何特判：在 $u = 0$ 处 $u_0 = 0$ ，于是 g_0 自动为零，对称性条件被坐标变换自动满足了，一行代码都不用写。对比一下第五部分的有限体积格式，那里必须为轴心与表面各写一段分支。

打包返回。第 108 行到第 111 行：

```
out = np.empty_like(y)
out[:Np] = dC
out[Np:] = dT
return out
```

`np.empty_like(y)` 创建一个和 y 同样形状的空数组。然后把含水率的导数和温度的导数分别填进前后两半，返回。

为什么要这样做：为什么要先创建空数组再填

直接拼接两个数组也可以，但那样会多分配一次内存。对于要在积分过程中被调用成千上万次的函数，减少一次内存分配能明显提高速度。

这一节要记住的 右端项一共六行有效代码，水分三行、温度三行，结构完全一样。核心思想只有一句：把方程写成通量散度形式，然后在表面那个节点上把通量换成物理条件。

11.6 初值与求解

文件第 115 行到第 117 行：

```
def y0(self):
    """初始状态 [C0 ... C0, T0 ... T0]: C0 = 2.55 [kg/kg]、T0 = 28.0 [
degC]。"""
    return np.concatenate([np.full(self.Np, cm.C0), np.full(self.Np, cm
.T0)])
```

初值就是式 (54)： $N + 1$ 个 2.55 后面接 $N + 1$ 个 28。`np.full(n, 值)` 生成 n 个相同的数，`np.concatenate` 把两段首尾拼接，和 `rhs` 里的拆分方式对应。

文件第 122 行到第 131 行：

```
def solve(self, t_end, t_eval=None, rtol=1e-10, atol=1e-13, method="BDF
",
        events=None, max_step=np.inf, first_step=None, dense=False):
    """积分到 t_end [s]，返回 scipy 的 OdeResult (sol.t [s]、sol.y = 状态
矩阵)。


t_eval 指定输出时刻 [s]；rtol/atol 控制时间积分误差；events 用于定位烘
干
时刻（判据 C_max 下穿 0.15 kg/kg）；method 换积分器做上述交叉验证。
"""
    return solve_ivp(self.rhs, (0.0, t_end), self.y0(), method=method,
                    rtol=rtol, atol=atol, t_eval=t_eval, events=events
,
                    max_step=max_step, first_step=first_step,
dense_output=dense)


```

这个方法只做参数透传，把右端项、初值、时间区间和容差交给 `solve_ivp`。第二部分讲过的半离散方程组 (131) 就是它要积分的东西。各个参数的含义是

- 第一个参数 `self.rhs` 是右端函数，也就是上一小节那个。
- 第二个参数 `(0.0, t_end)` 是时间区间。
- 第三个参数 `self.y0()` 是初值。
- `method` 指定积分方法，默认用 BDF。也可以改成 Radau 或 LSODA 做交叉验证，换积分器重算同一个问题是为了考察时间离散误差，不是为了换更快的算法。
- `rtol` 和 `atol` 是相对和绝对容差，用来控制精度。
- `t_eval` 指定希望输出结果的时刻，不指定的话积分器自己决定在哪些时刻输出。
- `events` 用来指定事件函数，烘干时间的判定就靠它。

为什么要这样做：为什么要写一层包装而不是直接调用

因为可以把常用的默认值固定下来，调用时只写关心的参数。而且以后要换求解器，只要改这一个函数，其他代码不用动。

为什么要这样做：`atol` 为什么要传一个数组

调用处普遍这么写：

```
at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
```

因为 C 的量级是 1、 T 的量级是 300，相差两个数量级。绝对容差的意义是「低于此值的变化不予理会」。若统一取 10^{-13} ，对温度来说相当于 3×10^{-16} 的相对精度，低于双精度机器精度，积分器会被迫把步长压到极小却换不来任何精度；反过来若统一取 10^{-11} ，对水分又太松。分组给容差，是让每个状态量在自己的尺度上被同等对待。

11.7 采样之一：物理半径节点与重心插值

求解器在 u 坐标上工作，但我们最终要的是物理半径上的剖面，所以需要一套从 u 回到物理量的工具。

文件第 135 行到第 137 行：

```
def r_nodes(self, t):
    """当前时刻的物理半径节点  $r_j = R(t)*\sqrt{u_j}$  [m]，用于把谱解画到  $r$ 
    坐标上。"""
```

```
return np.sqrt(self.u) * self.R_of(t)
```

物理半径 $r_j = R(t)\sqrt{u_j}$ 。一行。

文件第 139 行到第 145 行：

```
def _bary_weights(self):
    """CGL 节点的重心插值权重 w_j（两端点折半），供 _bary 与面积权重共用。"""
    N = self.N
    w = (-1.0) ** np.arange(N + 1)
    w[0] *= 0.5
    w[N] *= 0.5
    return w
```

这个函数计算重心插值的权重。式子是

$$w_j = (-1)^j \times \begin{cases} \frac{1}{2}, & j = 0 \text{ 或 } j = N, \\ 1, & \text{其他.} \end{cases} \quad (191)$$

代码先算出 $(-1)^j$ ，再把首尾两个元素各自乘 $\frac{1}{2}$ 。首尾两个系数为什么要取 $\frac{1}{2}$ ，理由是：切比雪夫-高斯-洛巴托节点包含两个端点，而端点在插值公式里只贡献半个权，所以首尾两个系数各乘 $\frac{1}{2}$ ，其余节点不带这个因子，系数就是正负一交替。

为什么要这样做：为什么需要重心插值

因为我们要知道的不是节点上的值，而是某些指定位置上的值。比如表格要求给出到中心距离 0.5、1.0 厘米处的含水率，这些位置不一定正好落在节点上。重心插值能在任意位置给出高精度的插值结果。

为什么要这样做：为什么不用普通的拉格朗日插值公式

因为式 (97) 那个分子分母连乘的公式在节点数多的时候会发生严重的舍入误差，甚至上溢或下溢。重心形式的数学结果和它完全相同，但数值上稳定得多，计算量也小得多。这是一个已经被广泛采用的改进形式。

文件第 169 行到第 176 行：

```
def interp_u(self, y, u_t):
    """把状态向量按 u 变量重心插值到 u_t（谱精度），返回 (C, T) 两个数组。

    y 为某一时刻的状态向量；u_t 为目标 u 坐标，与物理半径的换算见 r_nodes。"""
```

```

C = y[:self.Np]
T = y[self.Np:]
return (self._bary(C, u_t), self._bary(T, u_t))

```

把状态向量切开，两个场分别插值到目标坐标 u_t 。

主要的插值函数 `_bary` 在第 178 行到第 201 行，里面有两处防御性写法，值得单独讲。

```

def _bary(self, f, u_t):
    """单场重心插值：f 为 CGL 节点上的场值，u_t 为目标 u 坐标，返回同形状数组。"""
    u_t = np.atleast_1d(np.asarray(u_t, dtype=float))
    w = self._bary_weights()
    u = self.u
    out = np.empty(u_t.shape, dtype=float)
    for k, ut in enumerate(u_t):
        diff = ut - u
        # 命中判据用相对容差：绝对容差在 u_t 极接近某节点却不严格相等时会漏判，
        # 该分支的 1/diff 会把插值结果污染成无意义的巨值。
        hit = np.where(np.abs(diff) <= 1e-14 * max(1.0, abs(ut)))[0]
        if hit.size:
            out[k] = f[hit[0]]
            continue
        num = np.sum(w * f / diff)
        den = np.sum(w / diff)
        if den == 0.0:
            # 数学上 den = 1/prod(u_t-u_j) 恒不为零，但 N 较大且 u_t 落在
            # [0,1] 之外时，交替符号的项会在浮点下抵消到恰好 0，此时 num/
            den
            # 变成 inf。兜底取最近节点的值，保证输出有限。
            out[k] = f[int(np.argmin(np.abs(diff)))]
        else:
            out[k] = num / den
    return out

```

第一行把输入统一转成数组。`np.atleast_1d` 保证即使传进来一个数，也会被包成长度为一的数组，这样后面的处理逻辑统一；`np.asarray(..., dtype=float)` 保证元素是浮点数。

然后是循环，对每个目标位置 `ut` 逐个处理。`diff = ut - u` 算出目标位置与全部节点的距离。

第一处防御性写法是第 188 行与第 189 行：目标点恰好落在某个节点上时直接取该节点的值。判断用的是相对容差

$$|u_* - u_j| \leq 10^{-14} \max(1, |u_*|), \quad (192)$$

而不是一个固定的绝对容差。原因很实际：若用绝对容差，当目标点极接近某节点却不严格相等时会漏判，而下一行的 $\frac{1}{u_* - u_j}$ 会把插值结果污染成没有意义的巨值。

否则执行后面的重心公式计算

$$C(u_*) = \frac{\sum_j \frac{w_j}{u_* - u_j} C_j}{\sum_j \frac{w_j}{u_* - u_j}}. \quad (193)$$

代码里的 `np.sum(w * f / diff)` 对应分子，`np.sum(w / diff)` 对应分母，权重 w_j 就是式 (191) 算出来的。

第二处防御性写法是第 194 行到第 200 行。数学上分母等于 $1/\prod_j(u_* - u_j)$ ，恒不为零；但当目标点落在 $[0, 1]$ 之外、节点数又较多时，交替符号的项会在浮点下抵消到恰好为零，此时相除会得到无穷大。兜底取最近节点的值 `f[int(np.argmin(np.abs(diff)))]`，保证输出是有限数。这与第四部分讲的「域外的点不能外推」是同一件事的两道防线：调用方本来就不该把域外的点送进来，万一送进来了，这里也不会返回一个巨大的假数。

为什么要这样做：为什么不用显式的拉格朗日基函数求和

重心形式在数学上与拉格朗日插值多项式完全等价，但每求一个点只需要 $O(N)$ 次运算，而且数值上稳定得多。显式基函数在节点数稍大时系数会变得巨大，精度损失明显。

11.8 采样之二：面积平均权重

文件第 147 行到第 167 行：

```
def area_weights(self):
    """截面面积平均  $\bar{C} = \int_0^1 C(u, t) du$  在 CGL 节点上的权重（缓存）。

    论文 S11.6 给出的截面平均即此定义 ( $u=(r/R)^2$  下  $du$  已含  $2r dr$  的测度)。
```


逐行看代码。第 158 行用 `getattr` 检查缓存：算过一次就存进 `self._area_w`，下次直接取，不再重复算。第 159 行取求积点数 $n_q = 2N + 2$ ；第 160 行 `leggauss` 给出标准区间 $[-1, 1]$ 上的高斯-勒让德节点与权重；第 161 行和第 162 行把它们映射到 $[0, 1]$ ： $u_q = \frac{1}{2}(x_g + 1)$ ，权重 $w_q = \frac{1}{2}w_g$ 。这样取点的精度是无条件够的： $2N + 2$ 点高斯求积能精确积分 $4N + 3$ 次多项式，而被积的基函数只有 N 次，余量很大。

第 163 行取上一小节算出的重心权重；第 164 行算出求积节点与全部 CGL 节点的两两距离，`uq[:, None] - self.u[None, :]` 用广播把长度 n_q 的向量与长度 $N + 1$ 的向量变成形状 $(n_q, N + 1)$ 的矩阵。第 165 行用式 (193) 的重心公式求出每个求积节点上的拉格朗日基函数值，得到一个 $(n_q, N + 1)$ 的矩阵；第 166 行用求积权重对这组基函数加权求和，得到长度 $N + 1$ 的权重向量，正是式 (195) 里的那些积分值。第 167 行返回它。此后任意时刻的截面平均只要写 `area_weights() @ C_nodes` 就行。

这一节要记住的 从 u 节点回到物理量，一共三件工具：`r_nodes` 换算物理半径，`interp_u` 做重心插值，`area_weights` 做面积平均。三件工具都不依赖任何采样点数的选取。

12 参数文件 `common.py` 逐段解释

12.1 文件定位：让程序可以搬动

这一节的内容在总览里已经出现过，这里补充一个细节，也就是文件第 47 行到第 58 行的查找函数：

```
def _find_attach(name):
    """在候选目录下查找 附件/<name>，返回绝对路径；全部落空则报错并列出搜索过的目录。

    宁可直接抛 FileNotFoundError 也不要返回 None：附件缺失时后续拟合会静默读到错误数据，报错比出错数好查。
    """
    for base in _CANDIDATES:
        p = os.path.abspath(os.path.join(base, "附件", name))
        if os.path.isfile(p):
            return p
    raise FileNotFoundError(
        "找不到 附件/%s；已搜索：%s" % (name, [os.path.abspath(b) for b in _CANDIDATES]))
```

搜索失败时直接把搜索过的目录列出来，而不是返回一个空值。返回空值的后果是后面的拟合会静默读到错误数据，报错总比出错数好查。

12.2 烘房环境的标定

附件 1 给了 241 组实测数据，温度与含湿量都是先上升后稳定的形态。代码把它拟合成一阶惯性式。文件第 92 行到第 103 行：

```
def _fT(t, Tset, tau):
    """温度惯性式（供 curve_fit 调用）：t [s]，返回烘房温度 [degC]。"""
    return Tset - (Tset - T_air0) * np.exp(-t / tau)

def _fC(t, Cset, tau):
    """含湿量惯性式（供 curve_fit 调用）：t [s]，返回空气含湿量 [kg/kg]。"""
    return Cset - (Cset - C_air0) * np.exp(-t / tau)

(Tset_fit, tauT_fit), _ = curve_fit(_fT, _t_env, _Ta_env, p0=[50.2,
    1800.0])
(Cset_fit, tauC_fit), _ = curve_fit(_fC, _t_env, _Ca_env, p0=[0.0509,
    2800.0])
```

`curve_fit` 是 `scipy` 提供的最小二乘拟合函数，实现的就是第一部分第三章讲的 Levenberg-Marquardt 算法。它返回两样东西：第一样是拟合出来的参数，第二样是参数的协方差矩阵。左边写成 `(Tset_fit, tauT_fit), _ =`，就是把参数接出来，把协方差用下划线忽略掉。参数的含义是：第一个是模型函数，第二个是自变量的数据，第三个是因变量的数据，第四个是参数的初始猜测值。

注意初值被固定成实测第一点 `T_air0`（第 84 行），不作为自由参数。所以每个式子只有两个待定参数：稳态值与时间常数。

为什么要这样做：为什么用乘法形式而不是加法形式

一阶惯性式写成 $T_{\text{set}} - (T_{\text{set}} - T_{\text{air0}})e^{-t/\tau}$ ，在 $t = 0$ 处自动等于 T_{air0} ，也就是实测的初值。如果写成「拟合曲线加一个偏移量」的形式，扰动设定值时会把初值一起改掉，那就不是「只改设定值」了。

拟合结果按有效位数取用。文件第 108 行到第 116 行：

```
Tset = 50.212
tauT = 1877.5
```

```

Cset = 0.050908
tauC = 2776.3
VARIANTS = {
    "reported_precision": dict(Tset=Tset, tauT=tauT, Cset=Cset, tauC=tauC),
    "full_precision_fit": dict(Tset=Tset_fit, tauT=tauT_fit,
                               Cset=Cset_fit, tauC=tauC_fit),
}

```

前四行是报告值，也就是论文里实际用的、按有效位数截断后的数。**VARIANTS** 里同时保留了完全精度的拟合值，这样如果以后要做「参数精度的影响」分析，不用重新拟合；主程序读的是前四个常数，拟合出来的原始值留在带 **_fit** 的名字里备用。

为什么要这样做：为什么要给初始猜测值

因为式 (58) 无法直接求解，只能用迭代法，而迭代法必须从一个起点开始。从零点附近开始可能会收敛到错误的解或者根本不收敛。给一个接近真值的起点，收敛又快又稳。起点 50.2 是看着附件数据的终值给的，1800 是看着曲线上升的快慢估的。这两个数只要落在正确的量级，迭代就会收敛到同一个最小值，所以初值本身不需要很准。

环境随时间变化的最终形式分两段。文件第 128 行到第 142 行：

```

def T_air(t):
    """烘房温度 [degC]: t<=14400 s 取一阶惯性式，其后取稳态设定值。

    入参 t 可为标量或数组；返回同形状数组（后续用 numpy 广播，标量入参也返回 0 维数组）。
    """
    t = np.asarray(t, dtype=float)
    ramp = Tset - (Tset - T_air0) * np.exp(-t / tauT)
    return np.where(t <= T_END, ramp, Tset)

def C_air(t):
    """烘房空气含湿量 [kg/kg]: 分段口径与 T_air 完全相同（同一时刻同一个 T_END）。"""
    t = np.asarray(t, dtype=float)
    ramp = Cset - (Cset - C_air0) * np.exp(-t / tauC)
    return np.where(t <= T_END, ramp, Cset)

```

T_END 在第 86 行取附件 1 的采样终点 14400 秒（也就是 4 小时，见文件第 25 行的说明）。在这之前按惯性式上升，之后直接取设定值。**np.where** 是向量化的条件选择，

对数组里每个元素分别判断，不用写循环。温度与含湿量用同一个分段时刻，这样两条边界条件的口径完全一致。

为什么要这样做：这个分段为什么允许不连续

因为它是「实测已达稳态、故按稳态取值」这一建模选择的结果。文件第 123 行到第 127 行的注释把这件事写清楚了：一阶惯性式在 $t = 14400$ 秒处与设定值还差 T_{air} 约 1.037×10^{-2} 摄氏度、 C_{air} 约 1.748×10^{-4} kg/kg。把这段话写在代码里，是为了让人一眼看到这个跳变有多大、以及它为什么可以接受。

12.3 两种 RMSE 口径

`common.py` 里专门用一段注释记下了一件容易混淆的事，也就是文件第 118 行到第 120 行：

```
# 拟合残差有两种报告口径，看数时别混：论文表 1 与 run_all.py 报的是 RMSE 用
# ÷n 的 0.336 degC / 8.1e-4 kg/kg（把标定值当作对整段序列的整体描述）；
# curve_fit 的 pcov 走的是 ÷(n-2) 的自由度口径，只在需要参数标准误时使用。
```

RMSE 是均方根误差，定义是把每个时刻的残差平方求平均再开方。两种口径的差别只在「除以几」：

$$\text{RMSE}_n = \sqrt{\frac{1}{n} \sum_{i=1}^n r_i^2}, \quad \text{RMSE}_{n-p} = \sqrt{\frac{1}{n-p} \sum_{i=1}^n r_i^2}, \quad (196)$$

其中 r_i 是第 i 个时刻实测值与拟合值之差， $n = 241$ 是数据点数， $p = 2$ 是拟合参数的个数，所以后一种要除以 $n - 2 = 239$ 。

两者回答的是两个不同的问题。除以 n 的那种，把标定值当作对整段序列的整体描述，问的是「这条曲线整体拟合得好不好」，论文表 1 与 `run_all.py` 报的就是这个口径，温度 0.336 摄氏度、含湿量 8.1×10^{-4} kg/kg。除以 $n - p$ 的那种，是统计学里对总体方差的无偏估计口径，问的是「这两个参数本身有多不确定」，只在需要参数标准误时才用，`curve_fit` 内部算协方差矩阵走的就是它。

为什么要这样做：为什么非要把两种口径写在代码里

因为两种口径算出来的数不一样。如果不写清楚，同一份数据在不同脚本里报出两个不同的 RMSE，看的人会以为哪里算错了。而且这两个数的差不是舍入误差，是定义不同，除以 239 比除以 241 大约大百分之零点四，在温度上就是 0.336 与 0.337 的区别。把口径写下来，以后两个脚本报的数对不上时，一眼就知道该去查哪一条。

12.4 物性公式与浓度下限

文件第 148 行与第 151 行到第 184 行：

```
C_FLOOR = 1.0e-3

def props(prob, C, T):
    """返回 (rho, cp, k, D)，四个量与入参 C、T 同形状。

    prob 决定用哪套物性（对应题目三个附录）：
        1 -> 附录 2：常数物性（问题 1 用，温度场与水分场解耦）
        2、3 -> 附录 3：含 C、T 的强非线性关系（问题 2、3 用）
        4 -> 附录 4：另一套系数（问题 4 用，配合实测收缩）

    量纲：C 为干基含水率 [kg/kg]；T 为温度 [degC]，仅 Arrhenius 项内部换算成 K
    。
    返回：rho [kg/m3]、cp [J/(kg K)]、k [W/(m K)]、D [m2/s]。
    """
    C = np.asarray(C, dtype=float)
    T = np.asarray(T, dtype=float)
    Cf = np.maximum(C, C_FLOOR)
    TK = T + 273.15
    if prob == 1:
        one = np.ones_like(C)
        rho = 820.0 * one
        cp = 2600.0 * one
        k = 0.36 * one
        D = 7.0e-9 * np.exp(-0.89 / Cf)
    elif prob in (2, 3):
        rho = 650.0 + 128.0 * C
        cp = 1450.0 + 2736.0 * C / (C + 1.0)
        k = 0.21 + 0.38 * C / (C + 1.0)
        D = 2.4e-3 * np.exp(-0.45 / Cf) * np.exp(-3850.0 / TK)
    elif prob == 4:
        rho = 760.0 + 90.0 * C
        cp = 1850.0 + 2150.0 * C / (C + 1.0)
        k = 0.12 + 0.20 * C / (C + 1.0)
        D = 4.2e-4 * np.exp(-0.30 / Cf) * np.exp(-3850.0 / TK)
    else:
```

```
    raise ValueError("prob must be 1,2,3,4")
    return rho, cp, k, D
```

这个函数按 **prob** 的值从四套经验式里选一套。**prob=1** 是问题一的常数物性, **prob=2** 与 **prob=3** 共用附录 3 的关系, **prob=4** 用附录 4 的关系。最后那个 **else** 分支不是多余的: 如果传进来一个 5, 与其算出一个莫名其妙的数, 不如立刻报错。

两处必须解释的细节。

第一, $C_f = \text{np.maximum}(C, C_FLOOR)$ 。扩散系数里含因子 $e^{-a/C}$, 当 C 趋于零时这个因子会发散。把这个因子对 C 求导, 得到 $\frac{a}{C^2} \exp(-a/C)$, 也就是导数里含因子 $1/C^2$, 当 C 趋向于零时发散, 扩散系数对含水率的变化率就变得任意大, 自适应积分器为了跟上这种陡峭的变化会把步长压缩到极小, 甚至直接失败。物理上表面含水率确实不会降到零 (烘干判据是 0.15 kg/kg), 但数值迭代过程中可能出现极小的值, 所以把求值用的 C 截断到 10^{-3} kg/kg, 避免出现无穷大或者非数。这个下限比烘干判据 0.15 小两个数量级, 所以在真正关心的含水率范围内它根本不起作用, 不影响任何结果。

第二, $TK = T + 273.15$ 。Arrhenius 因子 $e^{-3850/TK}$ 里的温度必须用绝对温度。这一点如果弄错, 把摄氏度 50 直接代进去会得到 e^{-77} , 这是一个几乎等于零的数, 水分扩散系数会变成零, 干燥就无法进行。代码把换算明确写在一行上, 就是为了让这个约定显式可见。量纲检查也能发现这个错误: 3850 的量纲是温度, 只有和同样量纲的开尔文相除才是无量纲的。

12.5 附件 2 与半径插值

文件第 188 行到第 202 行:

```
def _load_radius():
    """读附件 2, 返回 (时间 [s], 半径 [m]); 附件以 cm 记录, 此处统一乘 1e-2 换成 m。"""
    ws = openpyxl.load_workbook(_find_attach("附件2.xlsx"), data_only=True)
    .active
    rows = list(ws.iter_rows(min_row=2, values_only=True))
    t = np.array([float(r[0]) for r in rows])
    R = np.array([float(r[1]) for r in rows]) * 1e-2    # cm -> m
    return t, R

_t_R, _R_hist = _load_radius()
```

```
def R_of(t):
    """问题4: 半径分段线性插值 (t 超出范围则取端点值)。"""
    return np.interp(t, _t_R, _R_hist)
```

附件 2 的半径列以厘米为单位, 读进来时统一乘 10^{-2} 换成米, 这样后面所有几何量都在同一个单位制里。`data_only=True` 表示只读单元格的值、不读公式, `min_row=2` 表示跳过表头行。

`R_of` 用分段线性插值。附件 2 每 1800 秒给一个点, 而求解器需要任意时刻的半径, 插值是必须的。超出附件范围时 `np.interp` 默认取端点值, 本题不会出现这种情况 (积分区间比附件短)。

12.6 Bessel 特征根

文件第 206 行到第 228 行:

```
def bessel_roots(Bi, n):
    """求  $\beta J_1(\beta) = Bi * J_0(\beta)$  的前 n 个正根 (升序)。

    Bi =  $hR_0/k$  为传热 Biot 数; 递推关系  $J_0' = -J_1$  保证该式正是 Robin 边界条件
     $k \cdot dT/dr = -h \cdot T$  分离变量后的特征方程。求根用"先用  $J_1$  的零点把实轴分段,
    再在每段内用 brentq 收缩"的办法——不依赖初值猜测, 且不会漏根或串根。
    """
    from scipy.special import jnp_zeros
    z = jnp_zeros(1, n + 2)  # J1 的正零点
    roots = []
    f = lambda b: b * j1(b) - Bi * j0(b)
    # 第一个根在 (0, z0)
    lo = 1e-12
    hi = z[0] - 1e-12
    if f(lo) * f(hi) < 0:
        roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
    for i in range(len(z) - 1):
        lo, hi = z[i] + 1e-12, z[i + 1] - 1e-12
        if f(lo) * f(hi) < 0:
            roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
        if len(roots) >= n:
            break
    return np.array(roots[:n])
```

这个函数求特征方程

$$\beta J_1(\beta) = \text{Bi } J_0(\beta) \quad (197)$$

的前 n 个正根。这个方程没有解析解，只能用数值求根，程序调用 `scipy` 的 `brentq`，用的是布伦特法。

为什么要这样做：为什么不能用一般的求根方法

因为贝塞尔函数的零点分布需要专门的策略：如果随便给一个区间让求根器去找，很可能漏根或者串根。布伦特法要求先给出一个两端函数值异号的区间，这样的区间叫括号。代码的做法是用 J_1 的零点把实轴分段，因为在相邻两个 J_1 零点之间，函数 f 恰好只有一个变号点，这样可以保证不漏根、不串根，而且不依赖初值猜测。逐行看：第 214 行 `jnp_zeros(1, n+2)` 取 J_1 的前 $n+2$ 个正零点，多取两个是为了保证区间够用；第 216 行定义被求根的函数 $f(\beta) = \beta J_1(\beta) - \text{Bi } J_0(\beta)$ 。第 218 行到第 221 行是特殊处理的第一段：第一个根落在 0 到 J_1 的第一个零点 z_0 之间，不在两个零点之间，所以要单独给一个区间 $[10^{-12}, z_0 - 10^{-12}]$ ，两端各让开一点点是为了避开端点处的零值。第 222 行到第 227 行在每两个相邻零点之间检查两端是否异号，异号才调用 `brentq` 求根；第 226 行到第 227 行在凑够 n 个根以后提前跳出。第 228 行只返回前 n 个。

`xtol` 与 `rtol` 设到浮点极限附近 (10^{-15} 与 8.9×10^{-16})，是因为特征根要用来做解析解，多一位精度都是好的。

这一节要记住的 `common.py` 是整份代码的单一数据源：参数、物性、环境、附件读取、特征根全部在这里，其他文件只调用不重复定义。

13 闭式解文件 `analytic_p1.py` 逐段解释

13.1 它解决什么问题

这个文件给出问题一温度场的解析解。它的意义不是「多算一个结果」，而是给数值方法当标尺。

问题一的物性都是常数，温度方程里不含含水率，所以温度场与水分场完全解耦，可以单独求解。又因为烘房温度恰好是指数型变化，温度场可以不离散地求出闭式表达式。有了它，后面所有关于谱方法精度的讨论才有依据：如果问题一也只有数值解，那整篇的验证就只剩「两种数值方法互相对得上」，而两只手互相挠背是证明不了任何事的。

为什么要这样做：有了数值解为什么还要写公式解

因为公式解是精确的，可以作为数值解的检验标准。如果数值解和公式解吻合，说明数值程序写对了。如果没有公式解，就只能相信程序自己的输出，出了问题也发现不了。这在第四部分会详细讲。

13.2 准静态分解的思路

闭式解的推导在第六部分完整展开，这里只说代码里的对应关系。方程是

$$\frac{\partial w}{\partial t} = \alpha \nabla^2 w - \frac{A}{\tau_T} e^{-t/\tau_T}, \quad (198)$$

其中 $w = T - T_{\text{air}}$ 是药材温度与烘房温度之差。

这个方程右边有两项。第一项是扩散，第二项是外界温度上升带来的驱动。两项对应的解可以叠加，所以尝试写成

$$w(r, t) = W(r) e^{-t/\tau_T} + \hat{w}(r, t). \quad (199)$$

第一部分是跟着外界温度走的准静态部分，第二部分是从初值出发的瞬态衰减部分。

代入原方程整理，可以证明如果令

$$\nabla^2 W + \kappa^2 W = \kappa^2 A, \quad \kappa^2 = \frac{1}{\alpha \tau_T}, \quad (200)$$

那么 $\hat{w}$ 满足的是一个齐次方程，也就是没有外界驱动的方程。齐次方程可以用分离变量法求解，得到贝塞尔函数的级数。

最终结果就是三部分相加：烘房温度本身、跟着外界温度走的准静态部分、从初值出发的瞬态级数部分。代码里的做法是先把各级常数算好，再在函数里把这三部分加起来。

文件第 32 行到第 51 行是模块级的常数与系数：

```
RHO, CP, K = 820.0, 2600.0, 0.36      # [kg/m3] [J/(kg K)] [W/(m K)]
R = cm.R0                             # 药材半径 R0 [m]
ALPHA = K / (RHO * CP)                 # 热扩散率 alpha = k/(rho*cp) [m2/s]
BI = cm.h * R / K                      # 传热 Biot 数 h*R0/k (无量纲)
A = cm.Tset - cm.T0                    # 惯性式幅值 A = Tset - T0 [degC]
# 准静态项的波数 kappa [1/m]: 把 w = W(r)exp(-t/tauT) 代回方程后，指数因子上
# 剩下的常数项给出 kappa^2 = 1/(alpha*tauT)，量纲上正是 1/热扩散长度。
KAPPA = np.sqrt(1.0 / (ALPHA * cm.tauT))

# ----- 特征函数展开的系数（取 NROOT 项） -----
```

```

# 特征值 beta_n 由 Robin 条件分离变量得到, 即 beta*J1(beta) = Bi*J0(beta) 的正
    根。
NROOT = 200
BET = cm.bessel_roots(BI, NROOT)
GAM = ALPHA * BET ** 2 / R ** 2          # 模态衰减率 gamma_n = alpha*beta_n^2/
    R0^2 [1/s]
# _NN 是归一化内积 N_n = 2*J1/(beta*(J0^2+J1^2)) (J0、J1 在 beta_n 处取值),
# BN 则是瞬态级数系数 b_n: 把初值 w_hat(r,0) = -W(r) 按 J0(beta_n r/R0) 展开得
    到,
# 其分母 beta_n^2/R0^2 - kappa^2 是本征波数与准静态波数的平方差: kappa 恰等于某
    个
# beta_n/R0 时该项发散, 本题二者相差较远, 各 b_n 都是有限值。
_NN = 2 * j1(BET) / (BET * (j0(BET) ** 2 + j1(BET) ** 2))
BN = A * KAPPA ** 2 * _NN / (BET ** 2 / R ** 2 - KAPPA ** 2)

```

逐个说明。

第 32 行取出常数物性 ρ 、 c_p 、 k , 第 33 行取出半径。第 34 行算热扩散率 $\alpha = \frac{k}{\rho c_p}$ 。
 第 35 行算传热 Biot 数 $Bi = \frac{hR_0}{k}$, 它是无量纲的, 衡量表面换热与内部导热谁更强。第
 36 行算惯性式的幅值 $A = T_{\text{set}} - T_0$ 。第 39 行算准静态项的波数 $\kappa = \sqrt{1/(\alpha\tau_T)}$, 它的物
 理意义是「热扩散长度的倒数」, 量纲是 1/ 米。

第 43 行取截断项数 200; 第 44 行求特征值 β_n , 也就是方程 $\beta J_1(\beta) = Bi J_0(\beta)$ 的根,
 求根用的函数就是上一节讲的 `cm.bessel_roots`。第 45 行算衰减率

$$\gamma_n = \frac{\alpha \beta_n^2}{R^2}. \quad (201)$$

第 50 行算归一化系数

$$N_n = \frac{2J_1(\beta_n)}{\beta_n [J_0^2(\beta_n) + J_1^2(\beta_n)]}, \quad (202)$$

`j0` 和 `j1` 是 `scipy` 提供的第一类贝塞尔函数。第 51 行算展开系数

$$b_n = \frac{A \kappa^2 N_n}{\beta_n^2 / R^2 - \kappa^2}. \quad (203)$$

这四行在模块导入时就算好了, 一共取 200 阶。这些式子的完整推导在第六部分, 那里
 从齐次化、准静态分解、特征值问题一直写到第 n 个展开系数 b_n 的来历, 一步都不跳。

13.3 代码实现: T_exact

文件第 54 行到第 73 行:

```

def T_exact(r, t, Tset=None, tauT=None, nterm=NROOT):
    """闭式解 T(r,t) [degC], r [m]、t [s]; 返回与 r 同形状的数组。

    Tset、tauT 不给就取 common.py 的标定值，传别的值可做环境参数敏感性分析；
    nterm 为瞬态级数的截断项数，默认 NROOT = 200 (n >= 2 的模态衰减很快)。
    级数形式: T = T_air(t) + (A + B*J0(kappa*r))*exp(-t/tauT)
                + sum_n b_n*J0(beta_n*r/R0)*exp(-gamma_n*t)
    """
    Ts = cm.Tset if Tset is None else Tset
    tau = cm.tauT if tauT is None else tauT
    a = Ts - cm.T0
    kap = np.sqrt(1.0 / (ALPHA * tau))
    B = cm.h * a / (K * kap * j1(kap * R) - cm.h * j0(kap * R))
    bn = a * kap ** 2 * _NN / (BET ** 2 / R ** 2 - kap ** 2)
    r = np.atleast_1d(np.asarray(r, dtype=float))
    val = np.full(r.shape, Ts - (Ts - cm.T0) * np.exp(-t / tau))
    val += (a + B * j0(kap * r)) * np.exp(-t / tau)
    for n in range(nterm):
        val += bn[n] * j0(BET[n] * r / R) * np.exp(-GAM[n] * t)
    return val

```

逐行看。

第 62 行和第 63 行允许调用者临时换用别的设定值与时间常数，不给就用参数文件里的标准值，这为后续的参数敏感性分析留了接口。

第 64 行 $a = T_{\text{set}} - T_0$ ，是惯性式的幅值，也就是温度从初值升到设定值要涨多少。

第 65 行算出准静态项的波数 $\kappa = \sqrt{1/(\alpha\tau_T)}$ 。

第 66 行算准静态项的系数

$$B = \frac{ha}{k\kappa J_1(\kappa R) - hJ_0(\kappa R)}. \quad (204)$$

它就是准静态解 $W(r) = A + BJ_0(\kappa r)$ 里的系数 B ，分母 $\kappa J_1(\kappa R) - hJ_0(\kappa R)$ 来自把 W 代入 Robin 边界条件，推导在第六部分第三章。

这里要特别说明一点：最终版本里系数 B 是在函数内部计算的，不再是一个叫 `_B` 的模块级常量。原因就是上面第 62 行和第 63 行允许换标定值：如果 B 在导入时算好一次，那么换一组 τ_T 调用时，它还会沿用旧的值，敏感性分析的结果就是错的。所以函数把 a 、 κ 、 B 、 b_n 全部按这一次传进来的 T_{set} 与 τ_T 重算一遍。旧的模块级 `_B` 已经删掉了；留在模块级的只有那些不随标定值变化的量，也就是 `BET`、`GAM`、`_NN` 与 `BN` 里的 β_n 、

γ_n 、 N_n 。

第 67 行算瞬态级数的系数

$$b_n = \frac{a\kappa^2 N_n}{\beta_n^2/R^2 - \kappa^2}, \quad (205)$$

`_NN` 是归一化内积，分母 $\beta_n^2/R^2 - \kappa^2$ 是本征波数与准静态波数的平方差。

第 68 行把输入统一转成数组，保证标量也能处理。接下来三行就是级数本身，写成一个初值再加上两段修正，形式上更接近「从环境值出发，再补上两个偏离量」：

- 第 69 行的 `val` 是烘房环境温度本身 $T_{\text{set}} - (T_{\text{set}} - T_0)e^{-t/\tau}$ 。
- 第 70 行加上准静态部分 $(a + BJ_0(\kappa r))e^{-t/\tau}$ ，也就是温度场相对于环境的稳定偏离。
- 第 71 行和第 72 行用循环把各阶瞬态模态 $b_n J_0(\beta_n r/R)e^{-\gamma_n t}$ 累加上去，每一阶都带一个随时间指数衰减的因子。

为什么要这样做：循环只需要很少几次

因为衰减因子 $\gamma_n t$ 随 n 增长很快。在 $t = 1800$ 秒时，前四项的 $\gamma_n t$ 分别是 1.529、13.191、39.483、80.742。第二项就已经被压到 $e^{-13.2} \approx 2 \times 10^{-6}$ 以下，第三项更是小于 10^{-17} 。所以实际上取两项就能达到机器精度，默认的 200 阶是留了很大余量的。

13.4 自检函数：让解析解自己证明自己

文件第 76 行到第 98 行，函数 `_selftest` 的全部内容：

```
def _selftest():
    """返回自检报告文本：特征值方程残差、t=0 处的全场极差与 nterm 敏感性表。

    t=0 的极差就是瞬态级数在初值处的截断残差（理论上为 0），nterm 加密后该值
    与 T(0,1800)、T(R0,1800) 是否变化，说明截断项数取得够不够。供 run_all.py 打
    印。
    """
    out = []
    out.append("alpha = %.6e m^2/s ; Bi = %.6f ; kappa = %.6f 1/m ; kappa*R
    = %.6f"
               % (ALPHA, BI, KAPPA, KAPPA * R))
    out.append("beta_1..4 = %s" % np.array2string(BET[:4], precision=10))
    out.append("特征方程残差 max|beta*j1-Bi*j0| = %.3e"
               % np.max(np.abs(BET * j1(BET) - BI * j0(BET))))
```

```

out.append("b_1..4   = %s" % np.array2string(BN[:4], precision=6))
out.append("gamma_n*t(1800) n=1..4 = %s" % np.array2string(GAM[:4] *
1800, precision=3))
rchk = np.linspace(0, R, 41)
out.append("t=0 全场极差（截断残差，应趋于 0）= %.3e" % np.ptp(T_exact(
rchk, 0.0)))
out.append(" 注：t>0 时 n>=2 项已被 exp(-gamma_n t) 压到 e^-13 以下，")
out.append("      故 t=1800 s 的结果对 nterm 不敏感（见下）。")
for nt in [1, 2, 3, 4, 8]:
    out.append("nterm=%d  T(0,1800)=%.10f  T(R,1800)=%.10f"
               % (nt, T_exact(0.0, 1800.0, nterm=nt)[0],
                  T_exact(R, 1800.0, nterm=nt)[0]))
return "\n".join(out)

```

它检查三件事。

第一，特征方程残差。第 86 行和第 87 行把求出来的根代回方程，看左边减右边有多接近零。实际运行结果是 1.8×10^{-12} ，属于机器精度量级，说明特征根求对了。

第二， $t = 0$ 时的全场极差。第 90 行和第 91 行在 0 到 R_0 上取 41 个点，算 $T_{\text{exact}}(r, 0)$ 的最大值减最小值。理论上 $t = 0$ 处级数应该精确还原初值，极差为零。实际算出来是 3.5×10^{-8} ，这个小量就是级数截断到 200 阶留下的残差。它同时告诉我们：级数本身的收敛速度足够快，取 200 阶已经让截断误差降到可忽略。

第三，截断项数的敏感性。第 94 行到第 97 行把 `nterm` 分别取 1、2、3、4、8，看 $T(0, 1800)$ 与 $T(R_0, 1800)$ 变不变。实际结果是 `nterm` 从 2 开始数值就完全不变了。原因是第二阶之后的衰减因子 $\gamma_n t$ 在 $t = 1800$ 秒时已经达到 13.2、39.5、80.7，对应的指数 $e^{-13.2} \approx 1.8 \times 10^{-6}$ 以下。

为什么要这样做：一个解析解为什么要带自检

因为解析解是基准。如果基准本身算错了，用它去校核数值方法就会得出「数值方法错了」的错误结论。`_selftest` 让这段解析解能够自己证明自己：特征根残差说明求根对， $t = 0$ 极差说明级数对，截断敏感性说明项数够。三件事都过关，它才有资格当标尺。这个函数不做自动断言、只打印出来供人判读，是因为它要报的是一组量级，不是一个非黑即白的判断。

这一节要记住的 闭式解文件的结构是：模块级算好不随标定值变化的特征根与归一化内积，函数内部按这一次传入的标定值重算 a 、 κ 、 B 、 b_n ，最后把环境温度、准静态项、瞬态级数三部分相加；`_selftest` 负责让这个基准自己证明自己。

14 驱动脚本逐段解释

14.1 run_all.py：一键复跑

这个文件把四个问题的求解、三族离散的互证、网格与时间收敛性、烘干时刻与关键答案串成一趟跑完，写成一份文本报告。

文件第 59 行到第 62 行：

```
def w(s=""):
    """写一行报告：同时进内存缓冲与屏幕（flush=True 便于重定向时实时看到进度）。
    """
    OUT.append(str(s))
    print(str(s), flush=True)
```

报告不是边算边写文件的，而是先攒在内存列表 OUT 里，最后一次性落盘，也就是第 386 行：

```
open("复跑报告.txt", "w", encoding="utf-8").write("\n".join(OUT))
```

这样做的原因是：如果中途报错，磁盘上不会留下半份看起来像完整报告的文件。encoding="utf-8" 必须写，否则在中文系统上会用 GBK 写盘。

报告分 12 节，从环境标定一直排到关键答案汇总，每一节对应论文里的某一张表或某一组验证数据。读报告时可以拿论文逐条对位。十二节的内容依次是：第 0 节烘房环境标定、第 1 节闭式解自检、第 2 节问题一谱解的空间收敛性、第 3 节与第 4 节问题一的表 1 与表 2、第 5 节水分场的网格收敛性、第 6 节第三族离散的互证、第 7 节问题二的表 3 与表 4、第 8 节两种离散族在一点的互证、第 9 节问题三的烘干时间、第 10 节问题四含收缩的烘干时间、第 11 节对照格式的时间步敏感性、第 12 节关键答案汇总。

文件开头还有一道防护，也就是第 40 行到第 43 行：

```
if __name__ != "__main__":
    raise RuntimeError(
        "run_all 的复跑逻辑在模块顶层，import 会立刻重跑全流程并覆盖 复跑报告.txt。"
        "请用 `python run_all.py` 运行。")
```

因为这个脚本的逻辑写在模块顶层而不是 main() 函数里，别人误写一句 import run_all 就会触发十几分钟的完整计算，还会覆盖掉已有的报告。拦一道比事后解释便宜。

第 386 行写的是相对路径，所以报告落在运行命令所在的目录里，在 code/ 目录下运行就落在 code/ 目录里。

14.2 sensitivity.py: 参数扰动与环境口径

这个脚本回答两个问题：哪个参数最要紧，以及「怎么把实测序列变成边界条件」这件事影响多大。

文件第 47 行到第 79 行，它把物性公式重写了一遍：

```
def props_pert(prob, C, T, pert):
    """按扰动因子重算物性，返回 (rho, cp, k, D)，与 common.props 的结构一致。
    ...
    """
    C = np.asarray(C, dtype=float)
    T = np.asarray(T, dtype=float)
    Cf = np.maximum(C, C_FLOOR)
    TK = T + 273.15
    E = 3850.0 * pert.get("E", 1.0)
    fD = pert.get("D0", 1.0)
    fa = pert.get("a", 1.0)
    if prob == 1:
        rho = np.full_like(C, 820.0)
        cp = np.full_like(C, 2600.0)
        k = np.full_like(C, 0.36)
        D = 7e-9 * fD * np.exp(-0.89 * fa / Cf)
    elif prob in (2, 3):
        rho = 650.0 + 128.0 * C
        cp = 1450.0 + 2736.0 * C / (C + 1.0)
        k = 0.21 + 0.38 * C / (C + 1.0)
        D = 2.4e-3 * fD * np.exp(-0.45 * fa / Cf) * np.exp(-E / TK)
    else:
        rho = 760.0 + 90.0 * C
        cp = 1850.0 + 2150.0 * C / (C + 1.0)
        k = 0.12 + 0.20 * C / (C + 1.0)
        D = 4.2e-4 * fD * np.exp(-0.30 * fa / Cf) * np.exp(-E / TK)
    return rho, cp, k, D
```

每个因子乘上一个扰动倍数。`pert` 是一个字典，缺省值是 1.0 也就是不扰动；取值写成 `pert.get(键, 缺省值)` 的形式，例如取 E 的倍数就写 `pert.get("E", 1.0)`，意思是「没有这个键时就取 1」。清单里第一行的省略号代表 docstring 的其余各行，它们写明了每个键的含义与量纲。

扰动的施加方式是临时替换模块属性，算完立刻还原。文件第 98 行到第 122 行与第

161 行到第 165 行：

```
import spectral
pert = pert or {}
base_props = cm.props
base_T, base_C = cm.T_air, cm.C_air
base_h, base_hm = cm.h, cm.hm
base_Tset = cm.Tset
...
cm.props = _props
cm.h = base_h * pert.get("h", 1.0)
cm.hm = base_hm * pert.get("hm", 1.0)
if "Tset" in pert:
    cm.Tset = base_Tset + pert["Tset"]
if ambient == "raw":
    ...
finally:
    cm.props = base_props
    cm.T_air, cm.C_air = base_T, base_C
    cm.h, cm.hm = base_h, base_hm
    cm.Tset = base_Tset
```

try/finally 保证不管中间出不出错都会还原。这样各组扰动之间互不影响，也不会污染后面其他脚本的运行环境。

为什么要这样做：为什么要用这种临时替换的做法

因为主求解器在内部调用的是 **common.props**。如果在主求解器里加一个扰动参数，就会污染主求解器的接口，让代码变复杂。临时替换只需要在驱动脚本里写几行，主求解器一行都不用改。这种做法叫猴子补丁，在科学计算脚本里很常见。

为什么要这样做：设定值扰动为什么用加减而物性扰动用乘除

因为两者的量纲不同。 E 、 a 、 D_0 是系数，按百分比扰动比较自然，所以用乘法；而烘房设定值 T_{set} 是一个具体的温度，正负 0.2 开尔文这样的小平移更符合「标定误差」的物理含义，所以用加法。另外，一阶惯性式的初值锚定在实测的 $T_{\text{air}}(0) = 28$ 度上，平移设定值不会改变初值，被扰动的是设定值与整条升温轨迹。这一点写在文件第 113 行到第 115 行的注释里。

环境口径的分叉也在同一个脚本里。文件第 116 行到第 122 行：

```
if ambient == "raw":
```

```

        cm.T_air = lambda t: np.where(np.asarray(t, dtype=float) <= cm.
T_END,
                                np.interp(np.asarray(t, dtype=float),
                                cm._t_env, cm._Ta_env), cm.
Tset)
        cm.C_air = lambda t: np.where(np.asarray(t, dtype=float) <= cm.
T_END,
                                np.interp(np.asarray(t, dtype=float),
                                cm._t_env, cm._Ca_env), cm.
Cset)

```

ambient="fit" 用拟合曲线，**ambient="raw"** 把温度与含湿量同时换回附件 1 实测序列的线性插值，4 小时之后仍取各自设定值。这一对比回答的是「环境建模方式的选择对结果有多大影响」，不需要另写一个脚本。

一共 12 组参数扰动。文件第 184 行到第 197 行：

```

cases = [
    ("E   +2%   (3850 -> 3927 K)",      {"E": 1.02}),
    ("E   -2%   (3850 -> 3773 K)",      {"E": 0.98}),
    ("a   +2%   (0.45 -> 0.459)",      {"a": 1.02}),
    ("a   -2%   (0.45 -> 0.441)",      {"a": 0.98}),
    ("D0  +2%",                          {"D0": 1.02}),
    ("D0  -2%",                          {"D0": 0.98}),
    ("h   +2%   (25 -> 25.5 W/m2K)",    {"h": 1.02}),
    ("h   -2%",                          {"h": 0.98}),
    ("hm  +2%   (8e-7 -> 8.16e-7)",    {"hm": 1.02}),
    ("hm  -2%",                          {"hm": 0.98}),
    ("Tset +0.2 K (50.212 -> 50.412)", {"Tset": 0.2}),
    ("Tset -0.2 K",                      {"Tset": -0.2}),
]

```

六个参数各正负两档：物性侧的活化温度 E 、活化浓度指数 a 、扩散系数前因子 D_0 取 $\pm 2\%$ ，传递侧的对流换热系数 h 、对流传质系数 h_m 取 $\pm 2\%$ ，环境侧的烘房设定温度 T_{set} 取 ± 0.2 开尔文。每个扰动都要从头重解一次问题三，合计 12 次，再加上环境口径的两组，结果写进中间数据/sensitivity.txt。在 0.4 秒就能算完一次烘干时长的规模下，这种扫描是划算的。

基准值随网格而变，这一点脚本自己声明了。文件第 172 行到第 174 行：

```

log("基准网格 N=%d, 基准口径（问题 3 物性 + 一阶惯性拟合环境）" % BASE_N)
log("注意：基准值随网格而变（N=48 得 205572.3 s, N=64 得 205574.5 s, ）")

```

`log(" 连续极限 205575.5 s)。本表所有相对变化均以本基准为分母。")`

因为表里所有的相对变化都是以本网格的基准 t_{dry} 为分母算出来的，换一个网格再跑，分母就变了，百分比也会跟着变。把这句话打在最前面，是为了防止有人拿不同网格的两张表直接对比。

14.3 `verify_data.py`: 四类扫描与一致性校验

这个脚本负责生成收敛性与积分器稳定性两组图要用的数据，一共四类扫描。文件第 41 行到第 50 行：

```
CRIT = 0.15
# 空间扫描的网格；时间扫描的步长 (s)。两者都是论文 fig22 的自变量。
SPACE_N = [24, 32, 48, 64, 96, 128]
TIME_DT = [40.0, 20.0, 10.0, 5.0, 2.0, 1.0, 0.5, 0.25]
SPACE_RTOL = 1e-11          # 与生产设置一致
FV_N = 1600                 # 定步长 theta 法的空间网格 (论文表 11)
FV_SPACE_N = [800, 1600, 3200, 6400] # fig22(a) 的经典 FV 空间对照
FV_SPACE_DT = 1.0           # 该对照固定用的时间步 (时间误差按 c*dt
                             扣除)
INTEG_CFG = [("BDF", 1e-9), ("BDF", 1e-10), ("BDF", 1e-12),
              ("Radau", 1e-11), ("LSODA", 1e-10)]
```

四类分别是：谱解网格扫描 (`SPACE_N`)、定步长格式的时间步扫描 (`TIME_DT`，空间网格固定 1600)、经典有限体积的空间扫描 (`FV_SPACE_N`，步长固定 1 秒)、以及积分器与容差的组合 (`INTEG_CFG`，5 组)。结果写进中间数据/`verification.json`，带一个版本号 `DATA_VERSION` 与生成命令。

最有价值的部分是一致性校验。文件第 187 行到第 273 行，函数 `check_consistency` 拿数据文件里存的烘干时刻与刚刚算出来的解对账，一共五条：

1. 空间扫描的 t_{dry} 必须随 N 单调不减；
2. 空间扫描里 $N = 64$ 的那一项必须等于解缓存里真实算出的 t_{dry} (差超过 0.05 秒就说明数据文件过期)；
3. 时间扫描的 t_{dry} 必须随步长减小而单调增大，而且相邻增量对步长的双对数斜率必须接近 1，也就是严格一阶收敛；
4. 经典有限体积的空间对照必须单调收敛，扣掉时间误差 $c\Delta t$ 之后不能超过谱解的连续极限；

5. 积分器与容差那 6 个点的极差必须小于 0.05 秒，否则「极差只有 0.007 秒」这个结论就不成立。

任何一条不成立就抛异常并中止，报错信息里直接给出重新生成的命令。

为什么要这样做：为什么要专门写一个对账函数

因为图是「死的」，数据是「活的」。如果改了模型参数却忘了重新生成验证数据，图上画的还是旧解的收敛曲线，而正文引用的是新数，两者对不上却很难从图上发现。对账函数把这件事变成不可能：宁可不画图，也不画一张来源不明的图。`verify_data.py` 自己也有两种用法：不带参数只做校验（秒级，`run_all.py` 与 `make_figures.py` 内部就是这么调它的），带 `--recompute` 才真实复跑四类扫描并重写数据文件（约 22 分钟）。

14.4 `console_utf8.py`：一个三十行的小文件

文件第 20 行到第 33 行：

```
def fix_console():
    """把 stdout/stderr 切到 UTF-8；失败时静默跳过，不阻断主流程。

    参数与返回：均为空（就地重配两个标准流）。
    errors="replace" 是关键：即使下游终端不接受 UTF-8，也只是把个别字显示成问号，
    不会让整段复跑因为一次 print 抛 UnicodeEncodeError 而中断。
    """
    try:
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
        sys.stderr.reconfigure(encoding="utf-8", errors="replace")
    except Exception:
        # 需要 Python 3.7+ 才有的 reconfigure，且个别环境的重定向流不支持；
        # 这种情况退回平台默认编码即可，不影响任何计算结果。
        pass
```

它只做一件事：把标准输出的编码切成 UTF-8。

为什么要这样做：为什么需要它

Windows 的控制台默认用 GBK 编码，而报告里全是中文和希腊字母，直接 `print` 会抛编码异常或者输出一堆乱码。这个函数必须在任何 `print` 之前调用，所以各个脚本的第一行有效代码就是它：

```
from console_utf8 import fix_console
fix_console()
```

`errors="replace"` 是关键：即使下游终端不接受 UTF-8，也只是把个别字显示成问号，不会让整段复跑因为一次 `print` 抛异常而中断。`try/except` 那一层也有用：`reconfigure` 需要 Python 3.7 以上，而且个别环境的重定向流不支持它，这时退回平台默认编码即可，不影响任何计算结果。这就是为什么把这个函数单独放一个文件：四个会打印中文的脚本都要用它，分别是 `run_all.py`（第 34 行）、`make_results.py`（第 33 行）、`sensitivity.py`（第 26 行）与 `make_figures.py`（第 29 行），写成模块以后每个脚本只要两行。

14.5 这一部分与第五部分的分工

第三部分讲的是出结果的代码：参数文件、主求解器、闭式解基准，以及直接调用它们的驱动脚本。第五部分讲的是出验证和出表出图的代码：两套对照求解器、结果文件生成、受控对照、以及整套插图体系。

两部分的划分不是按重要性，而是按「读到哪一步」：看完第三部分，你能跑出结果；看完第五部分，你能知道结果可不可信、以及它是怎么变成论文里的表格和图的。

这一节要记住的 第三部分涉及的七个文件构成主干：`common.py` 给参数，`spectral.py` 算解，`analytic_p1.py` 给基准，`run_all.py`、`sensitivity.py`、`verify_data.py` 各自跑一类分析，`console_utf8.py` 保证中文能打出来。它们都通过 `common.py` 这一个数据源连接起来。

Part IV

如何确认结果正确

程序能跑出数字，不等于数字是对的。这一部分讲怎么判断结果可不可信，以及结果里那些数字分别说明什么。

15 为什么必须验证

15.1 程序不会告诉你它算错了

这是初学者最容易踩的坑。如果代码里公式写错了，程序不会报错，它会照常运行，然后给出一个看起来完全正常的数字。

举一个具体的例子。假设在写表面边界条件时，把传质系数的系数写成了 $\frac{1}{4}h_m R$ 而不是 $\frac{1}{2}h_m R$ 。程序照样能跑，曲线照样光滑，但表面的干燥速度会慢一半，最后的烘干时间会明显偏长。从输出上看不出任何异常。

再举一个例子。假设数组的索引搞错了，把温度的最后一个分量当成水分来用。程序也不会报错，但结果完全没有意义。

为什么要这样做：为什么这类错误难以发现

因为数值方法本身就是在做近似，输出总是带着一点误差。当误差的来源既有正常的离散误差又有真正的代码错误时，两者混在一起，从数字上分不清。

唯一的解决办法是：拿一个已知正确答案的问题去检验程序。如果程序在已知答案的问题上给出正确答案，那么在未知答案的问题上给出正确答案的可能性就大得多。

这一节要记住的 验证不是可选项，是必需项。没有验证过的程序，输出的数字不能写进论文。

15.2 验证的层次

本文用了三层验证，一层比一层强。

层次	名称	做法
一	与精确解比较	找一个有公式解的问题，看程序算得对不对
二	网格加密	把节点数加倍，看结果是否收敛到同一个值
三	多方法互证	用完全不同的方法算同一个问题，看结果是否一致

下面三章分别讲这三种。

16 第一层验证：与精确解比较

16.1 精确解从哪里来

问题一的温度场有精确解，就是第三部分第十三章讲的贝塞尔-杜阿梅尔展开式。

要特别说明的是，这个精确解不是随便找的，它是在完全相同的方程、完全相同的边界条件、完全相同的初值下推导出来的。所以它可以作为程序的标准答案。

16.2 比较的方法

具体做法是：把 $N = 64$ 的谱解和精确解在从圆心到表面的 201 个位置上逐点相减，取差值的绝对值最大的那个。

结果如表 1。

表 1：谱解与精确解的偏差（问题一温度场， $t = 1800$ 秒）

节点数 N	4	6	8	16	32	64
最大偏差 / 开尔文	7.59×10^{-6}	2.88×10^{-10}	1.09×10^{-10}	1.13×10^{-10}	1.14×10^{-10}	1.14×10^{-10}

16.3 怎么读这张表

先看最左边两格。 $N = 4$ 时的偏差是 7.59×10^{-6} 开尔文， $N = 6$ 时降到 2.88×10^{-10} 开尔文。节点数从 4 加到 6，只多了两个节点，偏差掉了四个多数量级。这是谱收敛的典型表现：精度不是慢慢爬升，而是跳台阶。

再看右边几格。从 $N = 8$ 开始，偏差稳定在 1.1×10^{-10} 附近， N 继续加到 64 也不再下降。

这个不再下降的现象需要解释清楚，否则会被误读成方法失效。它说明偏差已经不再是空间离散造成的，而是时间积分造成的。判断依据是下面这组数据：把时间相对容差收紧，偏差跟着一起变，而空间节点数不变。

时间相对容差	偏差 / 开尔文
10^{-8}	1.03×10^{-7}
10^{-10}	2.34×10^{-9}
10^{-12}	1.14×10^{-10}

容差每收紧两个数量级，偏差就下降一个多数量级，两者同步变化。所以 1.1×10^{-10}

这个平台是时间容差 10^{-12} 决定的，不是计算机能表示的极限。双精度浮点数在温度 34 附近的表示间隔约为 10^{-14} 开尔文，比这个平台还小四个数量级。

为什么要这样做：这说明什么

说明空间离散的误差已经小到看不见了。换句话说，只用 7 个点，空间精度就已经落在时间精度的水平以下。

如果在 $N = 8$ 的时候偏差还很大，或者偏差随 N 增大而缓慢下降，那就说明空间离散有问题，需要检查微分矩阵或者边界条件的写法。

16.4 这个 10 的负 10 次方意味着什么

温度的量级是几十摄氏度。偏差是 10^{-10} 开尔文，相当于把温度算准到小数点后十位。

作为对比，工程上一般认为小数点后四位就够了。所以这个精度远远超过需要。

这一节要记住的 与精确解比较的结果是：偏差约 10^{-10} ，而且这个偏差由时间容差控制，不是空间离散造成。这证明空间离散与边界条件的实现是正确的。

17 第二层验证：网格加密看收敛

17.1 收敛是什么意思

数值方法用一个有限的网格去近似连续的区域。网格越密，近似越好。当网格无限加密时，数值解应该趋近于真实的连续解。这个性质叫收敛。

如果程序写错了，加密网格往往不会让它收敛到正确答案，而是收敛到某个错误的值，或者干脆发散。

所以做法是：用一系列越来越密的网格各算一次，看结果是否稳定下来。

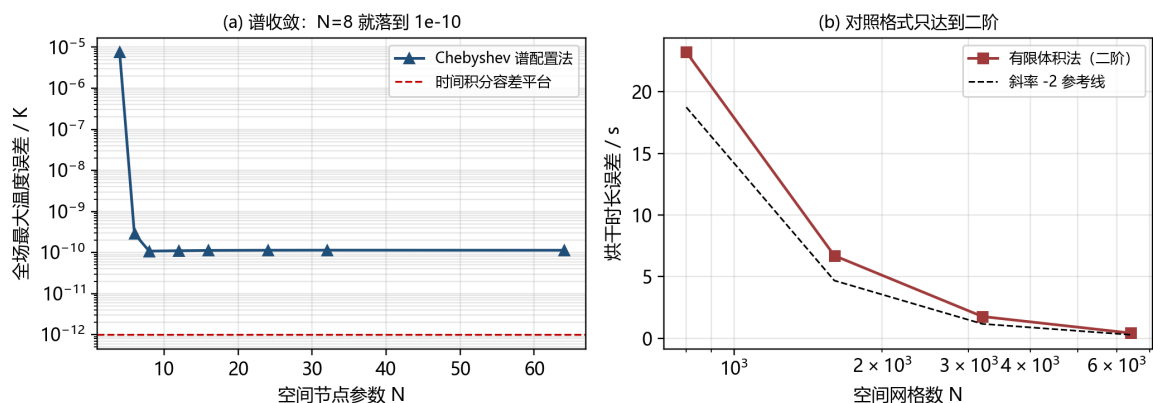


图 7：两种离散方式的空间收敛。(a) 谱配置法：节点参数 N 从 4 增到 6 时误差从 $7.6\text{e-}6$ 掉到 $2.9\text{e-}10$ ，此后落在时间积分容差平台上不再下降；(b) 有限体积对照格式只达到二阶收敛。

17.2 空间方向的加密

对问题三的烘干时间做这个实验。节点数从 24 增加到 128，每一步算出的烘干时间如表 2。

表 2：烘干时间随节点数的变化（问题三）

节点数 N	24	32	48	64	96	128
烘干时间 / 秒	205557.9	205565.2	205572.3	205574.5	205575.4	205575.5
相邻变化 / 秒	—	+7.3	+7.1	+2.2	+0.9	+0.1

17.3 怎么读这张表

看最后一行的相邻变化。从 7.3 秒降到 7.1 秒，再降到 2.2 秒、0.9 秒、最后 0.1 秒。变化量在持续减小。

当 N 从 96 增加到 128 时，结果只变了 0.1 秒。这说明再继续加密，结果也不会明显变化了。

本文最终采用的是 $N = 64$ 的生产值 205574.5 秒，因为交付的结果文件 result3.xlsx 就是按这个设置写出的。 $N = 128$ 给出 205575.5 秒，两者相差 1.0 秒，相对于五十七小时是万分之零点零五，可以忽略。

为什么要这样做：这个衰减速度说明了什么

这里的衰减是谱方法的典型特征，也就是衰减得非常快。如果换成普通的差分方法，同样的节点数范围内，变化量会缓慢地按平方反比下降，需要上千个节点才能达到这个

稳定程度。

17.4 时间方向的加密

用另一种格式，也就是第三部分提到的节点中心有限体积加定步长方法，做一次时间步长的实验。结果如表 3。

表 3: 烘干时间随时间步长的变化，对照格式，节点数 1600

时间步 Δt / 秒	10	5	2	1	0.5
烘干时间 / 秒	205553.80	205561.35	205565.88	205567.39	205568.10
相邻变化 / 秒	—	+7.55	+4.53	+1.51	+0.71

这张表的读法和上一张完全不同。

步长从 2 秒减到 1 秒，烘干时间增加 1.51 秒。再从 1 秒减到 0.5 秒，增加 0.71 秒。后一个增量大约是前一个的一半。

每把步长减半，变化量也减半。这是误差与步长成正比的特征，专业上叫做一阶收敛。对比上一张表，谱方法在空间上是跳跃式收敛，而这个格式在时间上只能按倍数慢慢逼近。

为什么要这样做：一阶收敛说明什么

说明这个格式的时间误差与步长成正比，减小得很慢。要把时间误差从秒的量级压到零点几秒，步长必须一直减下去，而且每减一半只能换来一半的改善。

这也说明，如果在干燥后期把步长放宽到 10 秒，算出来的烘干时间是 205553.8 秒，比生产值 205574.5 秒小了 20.7 秒。这个偏差只占万分之一点零，但在比较不同来源的烘干时长时足以造成困扰。

这一节要记住的 网格加密实验告诉我们两件事。第一，本文主求解器的空间收敛很快， N 到 64 以后结果就稳定了。第二，对照格式在时间方向只有一阶精度，如果步长放得太宽会带来十几秒的系统偏差。

18 第三层验证：多种方法互证

18.1 为什么要用不同方法

前两层验证都还在自己的体系内。与精确解比较用到了精确解，网格加密看的是自己的收敛行为。

第三层验证更强：用完全不同的离散思想去解同一组方程。如果两种毫不相干的方法给出同一个答案，那么两者同时出错的概率极低。

本文用了三种方法。

代号	方法	特征
甲	闭式解	用贝塞尔函数级数，没有离散网格
乙	谱配置法	切比雪夫节点，全局耦合，谱精度
丙	守恒型有限体积	均匀网格，只用到相邻点，空间二阶，时间用等步长后向欧拉，靠步长趋于零的外推消掉时间误差

三种方法的数学基础完全不同。甲是解析的，乙用的是全局多项式，丙用的是局部守恒。

18.2 比较结果

表 4: 三种方法的比较（问题一）

量	甲：闭式解	乙：谱配置法	丙：有限体积
$T(0, 1800)$ / 摄氏度	34.0424989	34.042499	—
$T(R_0, 1800)$ / 摄氏度	37.1989220	37.198922	37.1989221
$C(R_0, 1800)$ / (kg/kg)	—	1.51091810	1.51092243
$C(R_0, 1200)$ / (kg/kg)	—	1.65914239	1.65915047

18.3 怎么读这张表

先说丙列的来历，这一点必须交代清楚，否则数字无法复核。丙这一列不是直接算出来的一个数，而是这样得到的：在网格数 $N = 400$ 下取两个时间步 0.5 秒与 0.25 秒各算一次，再用后向欧拉的一阶格式做一次步长趋于零的外推，公式是外推值等于半步长的值减去两者之差。复跑报告的第六节列出了这两次计算的全部原始数据，用计算器就能复核上表里丙列的三个数。

第一行丙列留空，原因是复跑报告第六节的对照输出里没有圆心温度，这里不给出没有出处的数字。

再看数值本身。甲与乙在温度上的差别是最后一位小数，数量级是 10^{-7} 。乙与丙在表面温度上吻合到 7×10^{-8} 开尔文。

对于水分，因为没有闭式解，只能比较乙和丙。在 $C(R_0, 1800)$ 这一格，两者分别是 1.51091810 与 1.51092243，差别是 4.3×10^{-6} 。在 $C(R_0, 1200)$ 这一格，两者分别是 1.65914239 与 1.65915047，差别是 8.1×10^{-6} 。

把这三个量四舍五入到四位小数：表面温度三法都是 37.1989，表面水分在 1800 秒三法都是 1.5109，表面水分在 1200 秒乙给出 1.6591、丙给出 1.6592，差在第四位小数。交付文件统一保留四位小数，所以前两项三法一致，第三项在第四位小数上有一格之差。

为什么要这样做：有限体积的结果为什么不是恰好相等

因为它只做到空间二阶，本身带有离散误差，而且在 $N = 400$ 时空间误差还没有被外推掉。上表里丙的值已经把时间误差消掉，剩下的就是这部分空间误差，量级在 10^{-6} 到 10^{-5} 。两种方法都收敛到同一个连续解，只是在有限网格下各自带着不同的小误差。

18.4 一次特别的比较

除了上面这三种，本文还和第四种格式比过一次。在问题二的早期时刻，0.5 小时、圆心处：

方法	温度 / 摄氏度
谱配置法，节点数 $N = 64$ 至 128	32.565670279
节点中心有限体积加定步长方法	32.565670347
差值	6.8×10^{-8}

第二种方法的值是把它的一阶时间误差扣掉以后得到的。两种完全不同的离散格式在连续极限上吻合到 10^{-8} 的量级，这是一个很强的证据。

这一节要记住的 三种方法的互证结果是：在四位小数的精度上完全一致。这排除了代码存在系统性错误的可能。

19 灵敏度分析结果怎么读

19.1 做了什么事

把某个参数乘上一个系数，重新算一遍烘干时间，看结果变化多少。做了十二组。

19.2 结果

表 5: 参数扰动对烘干时间的影响（基准值 205574.519 秒）

扰动的参数	烘干时间 / 秒	相对变化	排名
活化温度 E 加百分之二	254624.0	+23.86%	1
活化温度 E 减百分之二	167355.9	-18.59%	2
活化浓度指数 a 加百分之二	215664.2	+4.91%	3
活化浓度指数 a 减百分之二	196057.7	-4.63%	4
扩散系数前因子 D_0 减百分之二	209277.8	+1.80%	5
扩散系数前因子 D_0 加百分之二	202019.9	-1.73%	6
烘房设定温度减 0.2 度	206911.0	+0.65%	7
烘房设定温度加 0.2 度	204250.0	-0.64%	8
传质系数 h_m 减百分之二	206055.5	+0.23%	9
传质系数 h_m 加百分之二	205116.3	-0.22%	10
换热系数 h 减百分之二	205580.3	+0.003%	11
换热系数 h 加百分之二	205568.9	-0.003%	12

19.3 三个要读懂的地方

第一，活化温度的影响最大，而且方向相反的两个扰动幅度不同。加百分之二带来 +23.86%，减百分之二带来 -18.59%。不对称的原因是这个参数出现在指数上，指数函数不是线性的，向上和向下的效果自然不一样。

第二，换热系数几乎没影响，不到万分之一。这个结果一开始可能让人意外，因为直觉上觉得换热快干燥就快。解释是这样的：温度在两个小时以内就已经和烘房温度基本持平了，换热系数只影响升温的快慢，不影响后期的干燥速度。而整个烘干要五十多个小时，所以前面两个小时的快慢对总时间几乎没有影响。

第三，传质系数的影响也不大，只有百分之零点二。这也不符合直觉，因为传质系数直接控制表面水分被带走的快慢。解释是：整个过程是内部扩散控制，也就是说瓶颈不在表面能不能被带走，而在内部水分能不能及时扩散到表面。表面带走得快一点慢一点，整体的速度由内部扩散决定。

为什么要这样做：这些结论有什么用

它们告诉你改进工艺应该往哪个方向使劲。比如想缩短烘干时间，调大换热系数是没用的，应该提高温度或者改善内部传质条件。

19.4 一个反直觉的结论

论文里还比较了两种处理烘房数据的方式：一种是用拟合的光滑曲线，一种是直接用表格数据做插值。

先看两种口径把空气温度本身算成多少。在 $t = 1800$ 秒，附件表格给出 41.513 摄氏度，拟合曲线给出 41.696 摄氏度，两者相差 0.183 摄氏度。

再看这个差别传递到结果上有多大。对问题一的温度场，两种口径的中心温度是 34.0425 与 33.5753 摄氏度，相差 0.4672 摄氏度；表面温度是 37.1989 与 36.7856 摄氏度，相差 0.4134 摄氏度。0.4672 摄氏度是本文数值误差 1.1×10^{-10} 开尔文的四十亿倍，所以对问题一来说，这是必须交代清楚的一个建模选择，而不是可以含糊过去的小事。

但是对烘干时长来说，两种口径给出 205574.5 秒与 205578.9 秒，只差 4.4 秒，相对偏差 0.0021%，也就是万分之零点二一。这两个数由 `env_compare.py` 一次运行同时给出，可以复现。

为什么要这样做：为什么同一个差异在两张表里的影响差别这么大

因为烘干时间主要由长达五十小时的恒温段决定。在恒温段，两种处理方式给出的烘房温度完全一样。差异只出现在最开始的两个小时，那时候药材还在升温。两个小时相对于五十小时是很小的一部分，所以对总时间影响很小。

而问题一问的正是前面三十分钟的温度，恰好落在差异最大的区间里，所以影响显著。

这一节要记住的 灵敏度分析的结论是：物性经验式里的活化温度是最重要的参数，换热系数几乎不起作用，烘房数据处理方式对短时段影响大、对总时长影响小。

20 常见疑问解答

20.1 关于建模

问：为什么可以把三维问题当成一维处理。

答：因为药材很长，长度是半径的 12.5 倍。热量和水分沿半径方向走完只需要几十分钟到几十小时，沿轴向走完需要好几天。两者相差一百多倍，所以轴向导热可以忽略。这个判断在第一部分第二章用特征时间的比值做了定量说明：径向特征时间是 $R_0^2/\alpha = 2368.9$ 秒，轴向特征时间是 L^2/α ，两者之比等于长度与半径之比的平方，即 $12.5^2 = 156$ 倍。

问：为什么水分浓度用干基而不是湿基。

答：因为干燥过程中水在减少，用湿基的话分母一直在变，写守恒方程时会多出额外的项。用干基的话分母是干物质，而干物质不增减，方程会干净很多。这一点在第一部分第二章讲过。

问：为什么不考虑蒸发吸热。

答：因为题目没有给汽化潜热和孔隙率这些数据，无法定量计算。论文里把这一条明确列为模型的局限之一，并说明了如果补上这一项，表面温度会降低，干燥会更慢。

20.2 关于数值方法

问：为什么不直接在半径坐标上算，非要做坐标变换。

答：因为半径坐标下圆心处有 $1/r$ ，离散时需要特殊处理。变换以后这个麻烦消失了，代码更短，也不容易出错。而且变换以后解在 u 上是解析函数，正好可以用谱方法，用很少的点就能算得很准。这是第二部分第五章讲的内容。

问：为什么节点要按余弦分布而不是均匀分布。

答：均匀分布在两端太稀，多项式会剧烈摆动。余弦分布在两端自动加密，能压住这种摆动。这是第二部分第六章讲的内容。

问：为什么时间方向要用隐式方法。

答：因为最开始表面失水很快，需要很小的步长才能算准；而整个过程长达五十小时。显式方法要求全程都用很小的步长，计算量大到无法接受。隐式方法可以用大得多的步长，代价是每步要解一个方程组。这是第二部分第七章讲的内容。

问：烘干时间为什么不用外推。

答：因为外推需要假设收敛的阶数。如果实际收敛阶和假设的不一样，外推结果就会偏。直接用高精度解求根不需要任何假设，更可靠。这是第二部分第八章讲的内容。

20.3 关于结果

问：四位小数是怎么保证的。

答：靠三层验证。与精确解比较的偏差是 10^{-10} ，网格加密到 $N = 64$ 以后结果不再变化，三种方法在四位小数上一致。三条证据合起来，四位小数是可靠的。

问：为什么烘干时间是 2.38 天而不是 2 天或者 3 天。

答：题目说一般持续两到三天，我们算出 2.379 天，落在区间内而且不贴边。这个吻合说明模型对整体时间尺度的刻画是可信的。如果算出来是 1 天或者 5 天，就说明哪里错了。

问：中心为什么是最后干的点。

答：因为水分要从内部扩散到表面才能被带走。中心到表面的距离最长，而它周围的含水率又比它低，浓度梯度小，驱动力弱，所以它最后干。论文里没有把这个预判写进程序，而是在每个时刻对全部节点取最大值来验证，结果确实如此。

20.4 关于写作

问：为什么要在论文里写一大堆检验。

答：因为阅卷的人无法重现你的计算，他们只能根据你提供的证据判断结果是否可信。检验章节就是提供这些证据的地方。检验写得越充分，结果越有说服力。

问：为什么要把错误也写出来。

答：因为发现错误并修正，是研究过程的一部分。把它写出来说明作者认真检查过，比假装一切顺利更可信。

21 全文回顾

这份文档一共讲了四件事。

第一部分从最基本的偏导数讲起，用守恒律推出了两个偏微分方程，写出了四个条件，并解释了烘房数据的标定方法。

第二部分讲了为什么这道题必须用数值方法，重点讲了坐标变换怎么消掉圆心处的麻烦，谱方法怎么用很少的点算得很准，时间方向为什么必须用隐式方法，以及烘干时间的判定为什么要用求根而不是外推。

第三部分把代码逐行讲了一遍，说明了每一段在做什么，为什么必须那么写。

第四部分讲了怎么验证结果，以及验证数据里的每一个数字分别说明什么。

如果只记住三件事，那就是：

第一，建模的核心是把物理守恒律翻译成微分方程，而这需要选择合适的未知量，也就是干基含水率。

第二，数值方法的核心是把连续的导数换成离散的代数运算，而这需要选择合适的坐标和合适的节点分布。

第三，验证的核心是找一个已知答案的问题去检验程序，而最有力的验证是用完全不同的方法算同一个问题。

全文完。

Part V

其余程序逐行解释

第三部分讲了 `spectral.py`、`common.py`、`analytic_p1.py` 以及三个直接调它们的驱动脚本。还有六个文件没有讲：两套对照求解器 `fv_theta.py` 与 `fv_uniform.py`，生成结果文件的 `make_results.py`，插图体系 `fig_core.py` 与 `make_figures.py`，以及补算受控算例的 `make_controlled_comparison.py`。这一部分逐个讲清楚，最后说明从运行验证到出表出图的完整顺序。

每一段代码清单都取自真实文件，清单之前写明它对应文件里的第几行到第几行。

22 对照求解器之一：节点中心有限体积 `fv_theta.py`

22.1 这个文件的两个用途

第三部分说明过为什么要有对照求解器：程序算错了不会报错，只会给出一个看起来正常的数，唯一的办法是用完全不同的方法算同一个问题。这个文件就是第二种方法，它做两件事。

第一，用节点中心有限体积法加定步长 `theta` 法求解问题一到问题四，与谱配置法对比。第二，研究一个具体问题：时间方向用固定步长、并把物性冻结在步首时，烘干时间的误差随步长怎样变化。复跑报告第 11 节给出的结论是：这个格式在时间方向只有一阶精度。

22.2 有限体积法的三个概念与网格记号

控制体是把求解区域切成的每一个小块；面通量是单位时间穿过两个小块之间那个面的物质量；守恒是指小块里的物质量要变化，只能靠物质从它的面流进或者流出。有限体积法对每个控制体写一条等式：变化率等于各面流入之和减去各面流出之和，再把时间导数换成差商、面通量用相邻节点上的值近似，就得到代数方程组。好处是守恒在离散层面也成立：两个相邻控制体共用一个面，穿过它的通量在前一个方程里算流出、在

后一个方程里算流入，数值相同，全部方程相加时内部面上两两抵消，只剩两端的边界通量。**节点中心**指未知量放在控制体中心，两个相邻节点的中点就是它们共用的面。取 $h = 1/N$ ，节点在 $\xi_i = ih$ ($i = 0, \dots, N$)，面在

$$\xi_{i+1/2} = \left(i + \frac{1}{2}\right)h, \quad i = 0, 1, \dots, N-1, \quad (206)$$

物理半径与坐标的关系是 $r = \xi R$ ；第 0 个控制体从圆心 $\xi = 0$ 到 $\xi_{1/2} = h/2$ ，第 N 个从 $\xi_{N-1/2} = 1 - h/2$ 到表面 $\xi = 1$ ，中间第 i 个占 $\xi_{i-1/2}$ 到 $\xi_{i+1/2}$ 。

22.3 体积测度的完整推导

体积测度是一个控制体在离散方程里占的权重，等于该控制体的体积（按单位长度算）除以常数 $2\pi R^2$ 。推导如下。位置 ξ 的物理半径是 $r = \xi R$ ，所以 $dr = R d\xi$ ；单位长度的柱面面积等于周长 $dA = 2\pi r = 2\pi \xi R$ ；于是单位长度、厚度 $d\xi$ 的一圈材料体积是 $dV = dA \cdot dr = 2\pi \xi R \cdot R d\xi = 2\pi R^2 \xi d\xi$ ，除以 $2\pi R^2$ 后，测度就是从 ξ 到 ξ 的简单积分： $vol_i = \int_{\text{控制体}} \xi d\xi$ 。

内部测度。取 $i = 1, \dots, N-1$ ，用原函数 $\int \xi d\xi = \xi^2/2$ 与式 (206)：

$$vol_i = \int_{\xi_{i-1/2}}^{\xi_{i+1/2}} \xi d\xi = \frac{\xi_{i+1/2}^2 - \xi_{i-1/2}^2}{2} = \frac{h^2}{2} \left[\left(i + \frac{1}{2}\right)^2 - \left(i - \frac{1}{2}\right)^2 \right]; \quad (207)$$

把两个平方展开， $\left(i + \frac{1}{2}\right)^2 = i^2 + i + \frac{1}{4}$ ， $\left(i - \frac{1}{2}\right)^2 = i^2 - i + \frac{1}{4}$ ，相减时 i^2 项与 $\frac{1}{4}$ 项消掉，得 $\left(i^2 + i + \frac{1}{4}\right) - \left(i^2 - i + \frac{1}{4}\right) = 2i$ ，所以 $vol_i = \frac{h^2}{2} \cdot 2i = ih^2 = \xi_i h$ ，这就是代码里的 `vol[1:N] = xi[1:N] * h`。**圆心处的测度**。第 0 个控制体从 $\xi = 0$ 到 $\xi = h/2$ ：

$$vol_0 = \int_0^{h/2} \xi d\xi = \left[\frac{\xi^2}{2} \right]_0^{h/2} = \frac{1}{2} \left(\frac{h}{2} \right)^2 - 0 = \frac{1}{2} \cdot \frac{h^2}{4} = \frac{h^2}{8}, \quad (208)$$

这就是 `vol[0] = h * h / 8.0` 的来历。它比内部第一个测度 $\xi_1 h = h^2$ 小八倍，因为测度含因子 ξ ，而圆心附近 ξ 接近零、权重也接近零。**表面处的测度**。第 N 个控制体从 $\xi = 1 - h/2$ 到 $\xi = 1$ ：

$$vol_N = \int_{1-h/2}^1 \xi d\xi = \frac{1^2 - \left(1 - \frac{h}{2}\right)^2}{2}. \quad (209)$$

用两数差的平方公式 $(a - b)^2 = a^2 - 2ab + b^2$ 取 $a = 1$ 、 $b = h/2$ ，得 $\left(1 - \frac{h}{2}\right)^2 = 1 - 2 \cdot 1 \cdot \frac{h}{2} + \left(\frac{h}{2}\right)^2 = 1 - h + \frac{h^2}{4}$ ，所以分子是 $1 - \left(1 - h + \frac{h^2}{4}\right) = h - \frac{h^2}{4}$ ，除以 2 得

$$vol_N = \frac{1}{2} \left(h - \frac{h^2}{4} \right) = \frac{h}{2} - \frac{h^2}{8}, \quad (210)$$

这就是 $\text{vol}[N] = h / 2.0 - h * h / 8.0$ 的来历。

为什么要这样做：最后一个控制体的测度为什么不是 h 除以 2

测度里的因子 ξ 在 $1 - h/2$ 到 1 上接近 1 而不是常数；若直接用半宽乘端点值得到 $h/2$ ，误差是 $h^2/8$ ，相对误差 $h/4$ ，当 $N = 1600$ 时约万分之一点六。这个偏差会让表面通量出现系统性误差，而表面通量直接决定烘干时间。

```
vol = np.empty(N + 1)
vol[0] = h * h / 8.0
vol[1:N] = xi[1:N] * h
vol[N] = h / 2.0 - h * h / 8.0
```

上面四行是 `fv_theta.py` 的第 49 行到第 52 行。`np.empty(N + 1)` 建一个长度 $N + 1$ 的空数组，`vol[1:N]` 是切片、包含下标 1 到 $N - 1$ 。

一个自洽检验。整个区域的测度是 $\int_0^1 \xi d\xi = [\xi^2/2]_0^1 = \frac{1}{2}$ 。用等差数列求和公式 $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ 取 $n = N - 1$ ，内部测度之和是 $h^2 \frac{(N-1)N}{2} = \frac{1}{N^2} \frac{(N-1)N}{2} = \frac{N-1}{2N}$ 。三部分相加：

$$\text{vol}_0 + \sum_{i=1}^{N-1} \text{vol}_i + \text{vol}_N = \frac{h^2}{8} + \frac{N-1}{2N} + \frac{h}{2} - \frac{h^2}{8} = \frac{N-1}{2N} + \frac{1}{2N} = \frac{N}{2N} = \frac{1}{2}, \quad (211)$$

其中首尾的 $h^2/8$ 与 $-h^2/8$ 抵消，最后一步把 $h = 1/N$ 代入 $h/2$ 。取 $N = 5$ 、 $h = 0.2$ 实算，测度数组是 $[0.005, 0.04, 0.08, 0.12, 0.16, 0.095]$ ：首项是 $h^2/8 = 0.005$ ，中间四项是 $\xi_i h$ ，末项是 $0.1 - 0.005 = 0.095$ ，六项和正好是 0.5。

这一节要记住的 体积测度是 $\int \xi d\xi$ 在一个控制体上的值：内部是 $\xi_i h$ ，圆心处是 $h^2/8$ ，表面处是 $h/2 - h^2/8$ ，三部分加起来必须等于 $1/2$ 。这三行测度代码，加上后面主轴上的两段特判，就是「不用正则化坐标」要付的代价。主求解器因为换了坐标，这三行完全不需要。

22.4 面通量与离散算子

第 i 与第 $i + 1$ 个控制体之间的面在 $\xi_{i+1/2}$ 处，穿过它的通量写成

$$g_{i+1/2} = \xi_{i+1/2} \cdot \text{coef}_{i+1/2} \cdot \frac{\phi_{i+1} - \phi_i}{h}, \quad \text{coef}_{i+1/2} = \frac{\text{coef}_i + \text{coef}_{i+1}}{2}. \quad (212)$$

第一个因子 $\xi_{i+1/2}$ 来自通量定义 $g = \xi \text{coef} \partial \phi / \partial \xi$ ；第三个因子是差商，因为两节点距离是 h ；中间的因子是面处的物性，而面上没有节点，只能用两侧节点值近似，程序取

算术平均。为什么可以，用一次展开检查。**泰勒展开**是把光滑函数在一点附近写成该点的值加上各阶导数项、第 k 阶项带因子 h^k 。记一阶导数为 c' 、二阶为 c'' ，则 $coef_{i+1} = coef_i + hc' + \frac{h^2}{2}c'' + \dots$ ，于是两侧平均是

$$\frac{coef_i + coef_{i+1}}{2} = coef_i + \frac{h}{2}c' + \frac{h^2}{4}c'' + \dots, \quad (213)$$

而面位置在 ξ_i 右侧 $h/2$ 处的真值是 $coef(\xi_i + \frac{h}{2}) = coef_i + \frac{h}{2}c' + \frac{h^2}{8}c'' + \dots$ 。两者相减， $coef_i$ 项与 $\frac{h}{2}c'$ 项消掉，剩下 $(\frac{h^2}{4} - \frac{h^2}{8})c'' + \dots = \frac{h^2}{8}c'' + \dots$ ，误差是 h^2 量级，与整个格式的空间精度同量级；若物性处处相同则 $c'' = 0$ ，算术平均精确。

为什么要这样做：什么情况下不能用算术平均

若物性在每个控制体内是常数、在面上跳跃，正确做法是取调和平均 $2coef_icoef_{i+1}/(coef_i + coef_{i+1})$ ，它保证面上通量连续。本题物性由含水率与温度决定，两者在空间上都是光滑函数，不存在跳跃。

把式 (212) 代入守恒式，得到半离散方程

$$\frac{d\phi_i}{dt} = \frac{1}{R^2\kappa_i vol_i} (g_{i+1/2} - g_{i-1/2}), \quad (214)$$

其中 κ_i 是容量：水分场取 1，温度场取 ρc_p 。

文件第 103 行到第 133 行，函数 `_operator` 的全部内容：

```
def _operator(N, h, xi, xif, vol, coef, kappa, R, beta, amb):
    """组装三对角算子 (a, up, lo) 与边界常数向量 c，满足
    dphi_i/dt = a_i*phi_i + up_i*phi_{i+1} + lo_i*phi_{i-1} + c_i。

    coef 为扩散/导热系数（面上取算术平均），kappa 为容量（水分取 1、温度取 rho*
    cp），
    beta 为表面传递系数（水分 hm、温度 h），amb 为环境值（C_air 或 T_air），单
    位同 phi。
    i=0 侧由对称性 g_{-1/2} = 0 自然封闭，故第 0 行只有一个面；i=N 侧把 Robin
    通量
    -R*beta*(phi_N - amb) 展开，未知量留在对角、环境值进常数项 c。
    """
    a = np.zeros(N + 1)      # 主对角
    up = np.zeros(N + 1)     # 上对角 (i -> i+1)
    lo = np.zeros(N + 1)     # 下对角 (i -> i-1)
    c = np.zeros(N + 1)

    face = 0.5 * (coef[:-1] + coef[1:])          # 面系数（算术平均）
```

```

gf = xif * face / h                                # 内部面系数 xi_{i+1/2}*
coef/h

inv = 1.0 / (R ** 2 * kappa * vol)                # 每行都要除的体积热容因子
# i = 0: 轴心节点, 只有外侧一个面 (g_{-1/2} = 0), 故只有一个非对角元
a[0] = -inv[0] * gf[0]
up[0] = inv[0] * gf[0]
# i = 1..N-1: 内部节点, 左右各一个面, 对角是两者的负和
i = np.arange(1, N)
a[i] = -inv[i] * (gf[i] + gf[i - 1])
up[i] = inv[i] * gf[i]
lo[i] = inv[i] * gf[i - 1]
# i = N: 表面节点, 左侧一个面再加上 Robin 通量项 R*beta, 环境值进常数项 c
a[N] = -inv[N] * (gf[N - 1] + R * beta)
lo[N] = inv[N] * gf[N - 1]
c[N] = inv[N] * R * beta * amb
return (a, up, lo), c

```

三对角矩阵是只有主对角线及其上下各一条对角线上元素非零的矩阵；本题第 i 个节点的方程只联系 ϕ_{i-1} 、 ϕ_i 、 ϕ_{i+1} ，所以矩阵是三对角的，三条对角线在代码里叫 **a**（主对角）、**up**（联系 i 与 $i+1$ ）、**lo**（联系 i 与 $i-1$ ）。

这个函数把式 (214) 组装成三对角形式。第 117 行实现式 (212) 的面系数：**coef[:-1]** 去掉最后一个元素、**coef[1:]** 去掉第一个元素，相加乘 0.5 后第 i 个元素就是 $(coef_i + coef_{i+1})/2$ ；第 118 行再乘面位置 **xif**、除以 h ，得到 $\xi_{i+1/2}coef_{i+1/2}/h$ ，数组长度 N 对应 N 个内部面。第 120 行的 **inv** 是每行都要除的体积热容因子，**face** 是面上的系数（算术平均），**gf** 是带上 ξ 因子与 $1/h$ 的面通量系数。

三段分支正是这部分开头说的代价。

第 0 个节点（轴心）只有外侧一个面，因为由 $g = \xi coef \partial\phi/\partial\xi$ 知 $\xi = 0$ 处 $g = 0$ ，圆心处的面通量恒等于零，对称条件不需要另外写，所以只有一个非对角元，对应第 122 行和第 123 行。

中间的节点左右各一个面，第 i 行右侧面的系数是 **gf[i]**、左侧是 **gf[i-1]**，把式 (212) 代入式 (214) 得

$$\frac{gf_i(\phi_{i+1} - \phi_i) - gf_{i-1}(\phi_i - \phi_{i-1})}{R^2\kappa_i vol_i} = \frac{gf_i\phi_{i+1} - (gf_i + gf_{i-1})\phi_i + gf_{i-1}\phi_{i-1}}{R^2\kappa_i vol_i}, \quad (215)$$

所以上对角是 $+inv_i gf_i$ 、主对角是 $-inv_i(gf_i + gf_{i-1})$ 、下对角是 $+inv_i gf_{i-1}$ ，与第 125 行到第 128 行一致；第 125 行用数组一次生成下标 1 到 $N-1$ ，不需要写循环。

第 N 个节点（表面）左侧一个面再加上 Robin 通量项：表面外侧的通量由边界条件给出。第二部分把表面条件化成式 (53)，即 $-coef \frac{\partial \phi}{\partial r} \Big|_{r=R} = \beta (\phi_N - \phi_{\text{amb}})$ ，水分取 $\beta = h_m$ 、 $\phi_{\text{amb}} = C_{\text{air}}$ ，温度取 $\beta = h$ 、 $\phi_{\text{amb}} = T_{\text{air}}$ ；用 $\frac{\partial}{\partial r} = \frac{1}{R} \frac{\partial}{\partial \xi}$ 并把两边乘 $-R$ ，再取表面处 $\xi = 1$ ，得到

$$g_{N+1/2} = -R\beta(\phi_N - \phi_{\text{amb}}). \quad (216)$$

代入式 (214) 后右边是 $[-R\beta(\phi_N - \phi_{\text{amb}}) - gf_{N-1}(\phi_N - \phi_{N-1})] / (R^2 \kappa_N \text{vol}_N)$ ，按未知量归并成 $-\frac{gf_{N-1}+R\beta}{R^2 \kappa_N \text{vol}_N} \phi_N + \frac{gf_{N-1}}{R^2 \kappa_N \text{vol}_N} \phi_{N-1} + \frac{R\beta}{R^2 \kappa_N \text{vol}_N} \phi_{\text{amb}}$ ，三项依次进主对角、下对角与常数向量 $\mathbf{c}$ ，对应第 130 行到第 132 行：未知量留在对角，环境值进常数项 $\mathbf{c}[\mathbf{N}]$ 。

符号检查：干燥时 $\phi_N > \phi_{\text{amb}}$ ，由式 (216) 得 $g_{N+1/2} < 0$ ，而式 (214) 里它带正号，于是 ϕ_N 减小，与水分被带走的物理事实一致。

同一个函数组装两个场。调用处只需要传两套系数，也就是文件第 86 行到第 89 行：

```
Lc, cc = _operator(N, h, xi, xif, vol, D, np.ones(N + 1), R,
                  cm.hm, Cair)
Lt, ct = _operator(N, h, xi, xif, vol, k, rho * cp, R,
                  cm.h, Tair)
```

水分场取 $\text{coef}=\mathbf{D}$ 、容量为 1、表面系数 hm 、环境值 $\mathbf{Cair}$ ；温度场取 $\text{coef}=\mathbf{k}$ 、容量 $\text{rho}*\text{cp}$ 、表面系数 $\mathbf{h}$ 、环境值 $\mathbf{Tair}$ 。这与主求解器的做法是一样的：算子写一份，两个场共用，只是系数不同。

22.5 清单：时间推进与 Rannacher 启动

文件第 63 行到第 67 行是步长查表：

```
def cur_dt(tt):
    for (t_lim, d) in dt_plan:
        if tt < t_lim:
            return d
    return dt_plan[-1][1]
```

dt_plan 是若干二元组（时刻上限，步长），函数返回第一个满足当前时刻小于上限的步长，都不满足时返回最后一个。`run_all.py` 与 `verify_data.py` 传的都是 `[(7200.0, min(1.0, d)), (1e18, d)]`，意思是前 7200 秒用不超过 1 秒的步长，之后用 d 。前两个小时是升温阶段，表面含水率从 2.55 急降、温度从 28 升到接近 50 摄氏度，而这个格式时间方向只有一阶精度、误差与步长成正比，所以变化最快的一段要用小步长。

文件第 44 行到第 46 行是网格与内部面：

```
h = 1.0 / N
```

```
xi = np.arange(N + 1) * h
xif = (np.arange(N) + 0.5) * h
```

第 44 行算 $h = 1/N$ ，第 45 行生成 $N + 1$ 个节点，第 46 行生成 N 个内部面。
文件第 69 行到第 100 行，函数 `simulate` 的推进循环：

```
while t < t_end - 1e-9:
    dt = cur_dt(t)
    if t + dt > t_end:
        dt = t_end - t
    R = float(cm.R_of(t)) if shrink else cm.R0
    Cair = float(cm.C_air(t))
    Tair = float(cm.T_air(t))
    # Rannacher 启动：CN 对初值跳变敏感、会激发非物理的高频振荡，故前 n_ram
    步
    # 改用后向 Euler (theta=1) 先把快模态阻尼掉，再切回 theta=0.5 取二阶精
    度。
    th = 1.0 if step < n_ram else theta

    # 物性冻结在步首状态上、整步不再更新：每步于是只解两个常系数三对角系统，
    # 代价是时间精度降为一阶（这也是本格式被称为"系数冻结"的原因）。
    rho, cp, k, D = cm.props(prob, C, T)

    # 同一个 _operator 组装两个场：水分取 coef=D、容量 kappa=1、表面系数
    hm、
    # 环境 C_air；温度取 coef=k、容量 kappa=rho*cp、表面系数 h、环境 T_air
    。
    Lc, cc = _operator(N, h, xi, xif, vol, D, np.ones(N + 1), R,
                      cm.hm, Cair)
    Lt, ct = _operator(N, h, xi, xif, vol, k, rho * cp, R,
                      cm.h, Tair)

    C = _theta_step(Lc, cc, C, dt, th)
    T = _theta_step(Lt, ct, T, dt, th)
    t += dt
    step += 1

    # 报告时刻可能落在步内部，故只要本步末时刻越过它就记录，快照取步末状态。
    while ri < len(report_idx) and t >= report_idx[ri] - 1e-9:
```

```

        out_t.append(t); out_C.append(C.copy()); out_T.append(T.copy())
        ri += 1
    return xi, out_t, out_C, out_T

```

第 69 行的循环条件写成 $t < t_{\text{end}} - 1e-9$ 而不是直接比较，因为浮点累加有舍入误差， $0.1 + 0.2$ 不等于 0.3 就是这个原因，留 10^{-9} 的余量可以避免多走或少走一步。第 70 行按计划取步长，第 71 行和第 72 行是收尾处理：如果这一步会跨过终了时刻，就把步长截短到恰好落在终点上。第 73 行取当前半径，问题一到问题三恒为 R_0 ，问题四由附件 2 的分段线性插值给出，这里用步首时刻的值。第 74 行和第 75 行取烘房环境值。第 78 行决定这一步的权重，也就是下面讲的 Rannacher 启动，前 n_{ram} （默认 4）步用 $\theta = 1$ 、之后才用 $\theta = 0.5$ 。第 82 行是系数冻结那一行：用旧时刻的 C 和 T 算出四个物性，整个时间步内不再改变，隐式部分与显式部分都用它。第 86 行到第 89 行构造两个离散算子，两个调用只差四处，这正是前面说的两个方程形式相同的用处。第 91 行和第 92 行各解一次三对角方程组得到新时刻的含水率与温度；含水率先更新，温度用的还是旧时刻算出的物性，同一步内不再反复迭代。第 93 行和第 94 行推进时刻与步数。第 97 行到第 99 行在走到报告时刻时把结果复制存起来， $C.\text{copy}()$ 必须写，否则存下的只是最后一次值的引用。第 100 行返回节点数组与三个输出列表。

文件第 136 行到第 154 行，函数 `_theta_step`:

```

def _theta_step(op, c, phi, dt, th):
    """推进一个时间步，解  $(I - dt*th*L)$   $\phi_{\text{new}} = \phi + dt*((1-th)*L \phi + c)$ 。

    op 为 _operator 返回的 (a, up, lo), th 为 theta (1 为后向 Euler、0.5 为 CN)。

    显式部分只做乘加，故右端不需要额外三对角求解；矩阵按 solve_banded 要求的
    (1, 1) 带宽装箱，不必构造稠密矩阵。
    """
    a, up, lo = op
    n = len(phi)
    lhs_a = 1.0 - dt * th * a
    lhs_up = -dt * th * up[:-1]
    lhs_lo = -dt * th * lo[1:]
    rhs = phi + dt * (1.0 - th) * (a * phi + up * np.r_[phi[1:], 0.0]
                                + lo * np.r_[0.0, phi[:-1]] + c) + dt *
    th * c
    ab = np.zeros((3, n))
    ab[0, 1:] = lhs_up
    ab[1, :] = lhs_a

```

```

ab[2, :-1] = lhs_lo
return solve_banded((1, 1), ab, rhs)

```

把空间离散后的方程写成 $d\phi/dt = A\phi + c$ ，其中 A 是上一节的三对角矩阵、 c 是边界条件带来的常数向量。**theta 法**是这样—个时间离散：右端项在新旧两个时刻的值按权重 θ 加权，

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = \theta (A\phi^{n+1} + c) + (1 - \theta) (A\phi^n + c). \quad (217)$$

$\theta = 0$ 时右边全是旧值，是显式欧拉法； $\theta = 1$ 时全是新值，是后向欧拉法； $\theta = \frac{1}{2}$ 时新旧各占一半，叫 Crank–Nicolson 方法。把含未知量的项移到左边、两边乘 Δt ，得到程序每一步要求解的方程组

$$(I - \Delta t \theta A) \phi^{n+1} = \phi^n + \Delta t (1 - \theta) (A\phi^n + c) + \Delta t \theta c, \quad (218)$$

其中 I 是单位矩阵。第 145 行到第 147 行构造左边的三条对角线：**lhs_a** 是主对角 $1 - \Delta t \theta a$ ，因为 a 是负数所以它大于 1；**lhs_up** 与 **lhs_lo** 是上下对角，**up[:-1]** 去掉最后一个元素，因为最后一行没有上对角，而那个数本来就是零。第 148 行和第 149 行构造右边，逐项对应式 (218)：**a * phi** 是主对角乘向量，**up * np.r_[phi[1:], 0.0]** 是上对角乘向量，其中 **np.r_** 是拼接函数，把 **phi[1:]** 与单个 **0.0** 接成长度相同的数组，末尾补的零对应最后一行为空的上对角；下对角同理，把 **0.0** 补在最前面。第 150 行到第 153 行把三条对角线装成 **solve_banded** 需要的格式。**solve_banded** 是 **scipy** 的带状矩阵求解函数，只存非零对角线，快且省内存，调用格式是 **solve_banded((1, u), ab, rhs)**，本题上下各一条；数组 **ab** 形状是 $(3, n)$ ，主对角放第 1 行，上对角放第 0 行但要向右错开一格，下对角放第 2 行但要少最后一格，所以第 151 行写 **ab[0, 1:]**。

第 78 行的 **th = 1.0 if step < n_ram else theta** 表示前 **n_ram** 步用 $\theta = 1$ 、之后才用 $\theta = 0.5$ ，这叫 Rannacher 启动。要知道它解决什么问题，先看两个格式怎么处理误差。把误差向量按空间算子的特征方向分解，**特征方向**是在空间算子作用下方向不变的向量，即 $Av = \lambda v$ 中的 v ， λ 叫**特征值**；误差在该方向的分量经过一个时间步被乘上的因子叫**放大因子**。本题 A 的特征值都是负的，记 $\mu = -\lambda > 0$ ：后向欧拉的放大因子是 $\frac{1}{1 - \Delta t \lambda} = \frac{1}{1 + \Delta t \mu}$ ，它总在 0 与 1 之间、 $\Delta t \mu$ 很大时接近零；Crank–Nicolson 的放大因子是 $\frac{1 + \Delta t \lambda / 2}{1 - \Delta t \lambda / 2} = \frac{1 - \Delta t \mu / 2}{1 + \Delta t \mu / 2}$ ， $\Delta t \mu$ 很大时它趋向 -1 ，绝对值仍不超过 1、格式稳定，但它不再压小高频误差，只让符号每步翻转，于是时间方向出现逐步交替、衰减很慢的振荡。问题出在起点：初值是处处等于常数的 C 和 T ，它不满足表面处的边界条件，因为表面一接触空气就开始失水，这个不一致让解在 $t = 0$ 附近有一个急剧变化的薄层，展开后含有与 $\Delta t^2/t$ 同量级的项，Crank–Nicolson 处理不了它。前面几步改用后向欧拉可以把这批高频分量压掉，后面的 Crank–Nicolson 才给出光滑结果。后向欧拉只有一阶精度，用它的步数越多引入的额外误差越大；用 4 步时这段时间的误差是 4 个 Δt^2 量级的量，与 Crank–Nicolson 自身的截断误差同量级，不足以改变整体性质。

为什么要这样做：为什么可以取到 10 秒这么大的步长

theta 法在 θ 不小于 $\frac{1}{2}$ 时无条件稳定，无论步长多大误差都不会被放大，显式格式没有这个性质。量级估算：温度场的热扩散系数 $\alpha = \frac{k}{\rho c_p} = \frac{0.36}{820 \times 2600} = 1.69 \times 10^{-7}$ 平方米每秒（复跑报告第 1 节记录 1.688555×10^{-7} ）；显式格式要求步长小于 $(Rh)^2/(2\alpha)$ ，取 $N = 1600$ 、 $R = 0.02$ 米，

$$\Delta t_{\max} = \frac{(Rh)^2}{2\alpha} = \frac{(0.02 \times 6.25 \times 10^{-4})^2}{2 \times 1.69 \times 10^{-7}} = \frac{1.5625 \times 10^{-10}}{3.377 \times 10^{-7}} = 4.6 \times 10^{-4} \text{ 秒}, \quad (219)$$

显式格式只能取不到 0.0005 秒，而这个文件用到 10 秒，相差约两万倍。

22.6 冻结系数为什么使时间方向只有一阶精度

局部截断误差是把精确解代进离散格式后、格式左右两边之差除以步长，它衡量格式在一步之内与微分方程差多少；若它是 Δt^{p+1} 量级，累积 $\frac{T}{\Delta t}$ 步后整体误差是 Δt^p 量级， p 就叫收敛阶。记 A^n 为用第 n 步的 ϕ^n 算出的系数矩阵、 c^n 为同一时刻的边界常数向量，程序算的是 $\frac{\phi^{n+1} - \phi^n}{\Delta t} = \theta (A^{n+1} \phi^{n+1} + c^{n+1}) + (1 - \theta) (A^n \phi^n + c^n)$ ：右边第一项代表新时刻的空间算子，按理应该用 A^{n+1} 和 c^{n+1} 。正确的 theta 法应当是

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = \theta (A^{n+1} \phi^{n+1} + c^{n+1}) + (1 - \theta) (A^n \phi^n + c^n). \quad (220)$$

把精确解代进程序的格式，局部截断误差 $\tau_n = \frac{\phi(t_{n+1}) - \phi(t_n)}{\Delta t} - \theta (A^n \phi(t_{n+1}) + c^n) - (1 - \theta) (A^n \phi(t_n) + c^n)$ ；在其中加上再减去 $\theta (A^{n+1} \phi(t_{n+1}) + c^{n+1})$ ，把式子拆成两半：

$$\tau_n = \underbrace{\left[\frac{\phi(t_{n+1}) - \phi(t_n)}{\Delta t} - \theta (A^{n+1} \phi(t_{n+1}) + c^{n+1}) - (1 - \theta) (A^n \phi(t_n) + c^n) \right]}_{\text{正确格式的截断误差}} + \theta [(A^{n+1} - A^n) \phi(t_{n+1}) + (c^{n+1} - c^n)]. \quad (221)$$

第一个方括号是式 (220) 的截断误差， $\theta = \frac{1}{2}$ 时是 Δt^2 量级。第二个方括号里 $A^{n+1} - A^n$ 是系数矩阵两个时刻之差，矩阵元素是物性的光滑函数，而物性在一步内只变化 Δt 量级，所以每个元素之差也是 Δt 量级；另一个因子 $\phi(t_{n+1})$ 是有限大小的数，整个第二项是 Δt 量级。两半合起来，

$$\tau_n = O(\Delta t^2) + O(\Delta t) = O(\Delta t). \quad (222)$$

Δt^2 那一项被 Δt 那一项盖住，局部截断误差是 Δt 量级，于是每步误差是 $\Delta t \cdot O(\Delta t) = O(\Delta t^2)$ 、累积后整体误差是 $O(\Delta t)$ ，Crank–Nicolson 本来有的二阶被冻结系数抵消掉了。复跑报告第 11 节的实测数据印证了这一点：经典格式取 $N = 1600$ ，长时间段步长依次取 10、5、2、1、0.5 秒，烘干时间分别是

步长 Δt / 秒	10	5	2	1	0.5
烘干时间 / 秒	205553.80	205561.35	205565.88	205567.39	205568.10
与谱解之差 / 秒	-20.72	-13.17	-8.64	-7.13	-6.42

步长从 1 秒减到 0.5 秒，差距从 7.13 秒变成 6.42 秒、减少 0.71 秒，大约是从 2 秒减到 1 秒时减少量 1.51 秒的一半；步长每减半、减少量也减半，这是一阶收敛的特征，所以粗步长下的值不能用来做外推。也要说明：在这个文件里改用后向欧拉不会改变整体收敛阶，因为冻结系数已经把精度压到一阶，Rannacher 启动在这里只起压掉起始振荡的作用。

这一节要记住的 这个格式有三层设计：体积测度保证守恒，theta 法保证无条件稳定，Rannacher 启动压掉起始振荡；但物性冻结让相容性误差变成 Δt 量级，整体只剩一阶精度。

22.7 find_tdry: 线性求交

文件第 157 行到第 207 行，函数 `find_tdry`:

```
def find_tdry(N, prob, dt_plan, shrink=False, hi=300000.0, crit=0.15):
    """从 t=0 单次前向推进，返回全场最大含水率首次下穿判据 crit 的时刻 [s].

    N、prob、dt_plan、shrink 同 simulate; hi 为搜索上限 3.0e5 [s]，到上限仍未
    下穿则返回 nan; crit 为烘干判据 0.15 [kg/kg]（干基含水率）。判据取 max_r C
    而不是平均值：轴心处水分最后才干，最大值达标即整个截面达标。
    """
    h = 1.0 / N
    xi = np.arange(N + 1) * h
    xif = (np.arange(N) + 0.5) * h
    # 体积测度同 simulate: 节点中心格式下由  $\int_{xi-1/2}^{xi+1/2} xi \, dxi$  得
    到。
    vol = np.empty(N + 1)
    vol[0] = h * h / 8.0
    vol[1:N] = xi[1:N] * h
    vol[N] = h / 2.0 - h * h / 8.0

    C = np.full(N + 1, cm.C0)
    T = np.full(N + 1, cm.T0)
```

```

t = 0.0
step = 0
Cprev, tprev = None, None
while t < hi - 1e-9:
    dt = dt_plan[-1][1]
    for (t_lim, d) in dt_plan:
        if t < t_lim:
            dt = d
            break
    # 时间离散口径与 simulate 不同：半径取步末（与新解同点），环境取步中点
    # 值。
    R = float(cm.R_of(t + dt)) if shrink else cm.R0 # 取步末半径
    Cair = float(cm.C_air(t + 0.5 * dt)) # 环境取步中点值
    Tair = float(cm.T_air(t + 0.5 * dt))
    # 前 4 步 Rannacher 启动（与 simulate 的默认 n_ram=4 同口径）
    th = 1.0 if step < 4 else 0.5
    rho, cp, k, D = cm.props(prob, C, T)
    Lc, cc = _operator(N, h, xi, xif, vol, D, np.ones(N + 1), R, cm.hm,
    Cair)
    Lt, ct = _operator(N, h, xi, xif, vol, k, rho * cp, R, cm.h, Tair)
    Cnew = _theta_step(Lc, cc, C, dt, th)
    T = _theta_step(Lt, ct, T, dt, th)
    # 记下本步起点 (tprev, Cprev)：判据下穿时靠这两点做插值求交。
    cprev = float(np.max(C))
    C = Cnew
    tprev, Cprev = t, cprev
    t += dt
    step += 1
    cnew = float(np.max(C))
    if cnew < crit:
        # Cmax 在 [tprev, t] 上线性插值求交
        if Cprev is None or Cprev == cnew:
            return t
        return tprev + (crit - Cprev) / (cnew - Cprev) * (t - tprev)
return float("nan")

```

第 157 行的 `hi` 是搜索的最长时刻 300000 秒、`crit` 是判据 0.15。第 164 行到第 171 行与 `simulate` 的前几行相同，重新算一遍网格与测度；这个函数没有复用 `simulate`，而是自己写了一遍推进循环，因为它的每一步都要检查判据，而 `simulate` 只在指定时

刻输出。第 173 行到第 176 行是初值与计数。第 177 行多出两个变量：**Cprev** 存上一步的最大含水率、**tprev** 存上一步的时刻，这两个数是线性求交要用的。

这个函数找的是全场最大含水率首次跌破判据的时刻。有两个设计要点。

第一，判据取全场最大值 $\text{np.max}(\mathbf{C})$ ，而不是某个固定位置的含水率。理论上极值一定出现在轴心，但代码不预设这一点——取全场最大值只会让答案稍保守，而取错位置会静默给出错误答案。

第二，找到下穿的那一步之后，用线性插值求交，而不是直接返回步末时刻。现在推导第 206 行的公式。手上有两个点 $(t_{\text{prev}}, C_{\text{prev}})$ 与 (t, C_{new}) ，其中 $C_{\text{prev}} > 0.15 > C_{\text{new}}$ 。线性插值是用一条直线穿过两个已知点，再在直线上取中间位置的值，过这两点的直线方程是

$$\begin{aligned} C(t_*) &= C_{\text{prev}} + \frac{C_{\text{new}} - C_{\text{prev}}}{t - t_{\text{prev}}} (t_* - t_{\text{prev}}) \\ \implies \text{crit} - C_{\text{prev}} &= \frac{C_{\text{new}} - C_{\text{prev}}}{t - t_{\text{prev}}} (t_* - t_{\text{prev}}) \\ \implies t_* &= t_{\text{prev}} + \frac{\text{crit} - C_{\text{prev}}}{C_{\text{new}} - C_{\text{prev}}} (t - t_{\text{prev}}), \end{aligned} \quad (223)$$

最后一步是把分式除过去再把 t_{prev} 移到右边，这正是第 206 行写的式子，也是第 203 行注释说的线性插值求交。检查它是否落在这一步之内：分子 $\text{crit} - C_{\text{prev}}$ 与分母 $C_{\text{new}} - C_{\text{prev}}$ 都是负数，相除得正；触发下穿说明新值已经越过判据到了另一侧，所以分母的绝对值大于分子、商严格小于 1， t_* 落在 t_{prev} 与 t 之间。第 204 行处理退化情形：若这一步最大值没有变化，分母是零会出错，直接返回步末时刻 t ；另一个条件 **Cprev is None** 是防御性写法，按执行顺序到这里时它不会成立。第 207 行在跑满 **hi** 秒仍未下穿时返回 **nan**，调用者可以据此发现问题。

为什么要这样做：为什么不直接取第一个低于阈值的点

因为采样间隔是百秒量级，直接取点会把达标时刻系统性地推迟若干步。上述插值把误差从「半步长」降到「求解器精度」。主求解器那边用的是事件机制加稠密输出求根，两条路径的实现完全不同，但给出的结果一致，这本身就是一次互证。

22.8 时间离散口径的三处不同

simulate 与 **find_tdry** 里的时间离散口径并不完全一样，三处差别都在文件注释里写明了。

第 73 行取半径用的是步首时刻 t ，第 185 行取的是步末时刻 $t + \Delta t$ ；第 74 行和第 75 行的环境值取步首值，第 186 行和第 187 行取的是步中点值 $t + \Delta t/2$ 。环境函数在 14400 秒以前是上升曲线，用中点值相当于对这一段做中点求积、比用左端点更接近积分平均。第 78 行的启动步数取参数 **n_ram**（默认 4），第 189 行写死为 4，两者默认值一致。

这三种取法都不改变收敛阶，因为冻结系数已经把阶压到一阶了。但要紧的是：正因为口径不同，拿 `simulate` 与 `find_tdry` 的单步结果直接对比是不对的，两者相差 $O(\Delta t)$ ，必须各自做时间步外推。`fv_theta.py` 第 39 行到第 42 行的 docstring 专门声明了这一点，`fv_uniform.py` 第 58 行和第 59 行的注释里又声明了一次。

这一节要记住的 `fv_theta.py` 的价值有两层：一是作为与谱方法完全不同的第二族离散，用来交叉验证结果；二是用它把「固定步长加冻结系数只有一阶精度」这件事量化出来，从而说明为什么不能拿它的粗步长结果当答案。

23 对照求解器之二：均匀网格守恒型有限体积 `fv_uniform.py`

23.1 与主求解器的三处刻意差别

交叉验证要排除两边同时犯同一个错误的可能，所以程序之间差别越大越好。这套代码里有三族离散，把它们并排放在一起看最清楚：

	主求解器 <code>spectral.py</code>	<code>fv_theta.py</code>	<code>fv_uniform.py</code>
坐标	$u = \xi^2$	$\xi = r/R$	$\xi = r/R$
空间离散	切比雪夫谱微分	节点中心有限体积	均匀网格守恒型有限体积
时间格式	自适应变阶 BDF	定步长 θ 法	等步长后向欧拉
非线性处理	隐式迭代自动吸收	物性冻结在步首	Picard 迭代 3 次

四个维度全都不一样，这正是交叉验证需要的。如果两套方法只是同一份代码换了个名字，它们同时犯同一个错误的概率就很高，互证的价值也就没了。

坐标不同意味着矩阵元素完全不同；离散不同意味着误差来源不同，谱方法是全局逼近、误差随节点数指数下降，有限体积是局部守恒、误差按二阶下降；时间方法不同意味着步长序列与误差控制机制没有共同点。四个维度都不同而三边给出同一个答案，说明它们解的是同一个偏微分方程，各自的离散误差都已压到目标精度以下。

23.2 网格、测度与输出索引

文件第 25 行到第 53 行，函数 `solve_fv` 的开头：

```
def solve_fv(N, dt, t_end, prob, t_report, shrink=False, n_picard=3):
    """积分到 t_end [s] 并采样，返回 (xi, C_tab, T_tab)。
    ...
```

```

"""
h = 1.0 / N
xi = np.arange(N + 1) * h
xif = (np.arange(N) + 0.5) * h          # 内部面 xi_{i+1/2}, i=0..N-1

# 节点中心格式的体积测度（已约去 2*pi）：节点 i 代表环形控制体
# int_{xi_{i-1/2}}^{xi_{i+1/2}} xi dxi, 故紧贴轴心与表面的半控制体各为 h
^2/8。
vol = np.empty(N + 1)
vol[0] = h * h / 8.0
vol[1:N] = xi[1:N] * h
vol[N] = h / 2.0 - h * h / 8.0

C = np.full(N + 1, cm.C0)
T = np.full(N + 1, cm.T0)

nstep = int(round(t_end / dt))
trep = np.asarray(t_report, dtype=float)
# 报告时刻按"步号 = round(t/dt)"定位，落在半步上的时刻取不到值，其行保持 nan
。
idx = {int(round(t / dt)): k for k, t in enumerate(trep)}
Ctab = np.full((len(trep), N + 1), np.nan)
Ttab = np.full((len(trep), N + 1), np.nan)

```

清单里第一个省略号代表 docstring 的其余各行，它们写明了三个返回值的形状与含义。

第 25 行用等步长 `dt`、用 `t_report` 指定要记录的若干时刻、用 `n_picard` 指定非线性迭代次数，没有 `theta`，因为时间格式固定是后向欧拉。第 34 行到第 43 行生成网格与测度，与 `fv_theta.py` 里的写法逐字相同：同样的均匀 ξ 网格与同样的三个测度公式；也就是说两个对照文件共用同一套空间离散，差别在时间推进、非线性处理和线性求解器上。第 45 行和第 46 行用 `np.full` 把含水率与温度初始化成题目给的初值，对应式 (54)。

测度部分与 `fv_theta.py` 完全相同，因为几何一样、网格一样。接下来是输出时刻的定位：第 48 行算总步数，`round` 是四舍五入，因为 `t_end / dt` 可能因浮点误差得到 7199.999999，直接取整会少一步。第 49 行把报告时刻转成浮点数组。第 51 行建立字典，键是 `int(round(t / dt))`（该时刻对应第几步）、值是它在 `t_report` 里的位置。

`idx` 是一本「字典」，把「步号」映射到「第几个报告时刻」。因为时间步是等长的，

报告时刻只要落在整数步上就能取到值；落在半步上的取不到，对应那一行就保持初始的 `nan`。第 52 行和第 53 行建立两个表格、初始值填 `nan`；`nan` 是 not a number 的缩写、表示无效数值，某个报告时刻若因步数不够没走到，那一行就保持 `nan`，不会被当成零。

为什么要这样做：为什么用字典而不是循环查找

每一时间步结束时只需查一次字典就能知道「这一步要不要记快照」，时间复杂度是常数，而不是每步都去遍历报告时刻表。代价是报告时刻必须是步长的整数倍，`run_all.py` 传的 1200 秒与 1800 秒配 1、0.5、0.25 秒都是整数倍。

23.3 后向欧拉离散：逐个节点写出方程

在 ξ 坐标下两个方程可以合并写成 $\kappa \frac{\partial \phi}{\partial t} = \frac{1}{\xi R^2} \frac{\partial}{\partial \xi} \left(\xi \text{coef} \frac{\partial \phi}{\partial \xi} \right)$ ，水分取 $\phi = C$ 、 $\text{coef} = D$ 、 $\kappa = 1$ ，温度取 $\phi = T$ 、 $\text{coef} = k$ 、 $\kappa = \rho c_p$ ，与 `fv_theta.py` 用的是同一个方程。后向欧拉是用新时刻的值近似右端项的一阶时间格式，把式 (214) 的时间导数换成后向差商并乘 Δt ：

$$\phi_i^{n+1} - \phi_i^n = \frac{\Delta t}{R^2 \kappa_i \text{vol}_i} \left(g_{i+1/2}^{n+1} - g_{i-1/2}^{n+1} \right). \quad (224)$$

圆心节点 $i = 0$ 。这个控制体只有外侧一个面，在 $\xi_{1/2} = h/2$ 处，内侧没有面，因为 g 在 $\xi = 0$ 处等于零。面通量是 $g_{1/2} = \xi_{1/2} \frac{\text{coef}_0 + \text{coef}_1}{2} \frac{\phi_1 - \phi_0}{h} = \frac{h}{2} \frac{\text{coef}_0 + \text{coef}_1}{2h} (\phi_1 - \phi_0)$ ，两个 h 约掉，得到的正是代码里的 `gf = xif[0] * 0.5 * (D[0] + D[1]) / h`。代入式 (224) 并把未知量归到左边、记 $\text{coef} = \Delta t gf / (R^2 \kappa_0 \text{vol}_0)$ ：

$$(1 + \text{coef}) \phi_0^{n+1} - \text{coef} \phi_1^{n+1} = \phi_0^n, \quad (225)$$

主对角是 $1 + \text{coef}$ 、上对角是 $-\text{coef}$ 、右端是旧值，对应第 80 行算 `gf`、第 81 行算 `coef`、第 82 行 `diag += coef`、第 83 行 `A[0, i] = -coef`，`b` 保持为旧值。

内部节点 $i = 1, \dots, N-1$ 。两侧的面在 $\xi_{i-1/2}$ 与 $\xi_{i+1/2}$ 处，系数是 $gf_p = \xi_{i+1/2} \frac{\text{coef}_i + \text{coef}_{i+1}}{2h}$ 、 $gf_m = \xi_{i-1/2} \frac{\text{coef}_{i-1} + \text{coef}_i}{2h}$ 。代入式 (224)，并记 $cp_i = \Delta t gf_p / (R^2 \kappa_i \text{vol}_i)$ 、 $cm_i = \Delta t gf_m / (R^2 \kappa_i \text{vol}_i)$ ，把未知量归到左边：

$$-cp_i \phi_{i+1}^{n+1} + (1 + cp_i + cm_i) \phi_i^{n+1} - cm_i \phi_{i-1}^{n+1} = \phi_i^n, \quad (226)$$

对应第 91 行到第 98 行：`gf_p`、`gf_m` 是两个面系数，`cp_`、`cm_` 是两个系数，`diag += cp_ + cm_` 是主对角，`A[0, i] = -cp_` 是上对角，`A[2, i] = -cm_` 是下对角，`b` 保持为旧值。

表面节点 $i = N$ 。外侧面的通量由式 (216) 给出，内侧面的系数是 $gf = \xi_{N-1/2} \frac{\text{coef}_{N-1} + \text{coef}_N}{2h}$ 。代入式 (224)，把已知的环境值移到右边，并记两个系数

$$\text{coef}_i = \frac{\Delta t gf}{R^2 \kappa_N \text{vol}_N}, \quad \text{coef}_b = \frac{\Delta t R \beta}{R^2 \kappa_N \text{vol}_N}, \quad (227)$$

则表面节点的方程是

$$-coef_i \phi_{N-1}^{n+1} + (1 + coef_i + coef_b) \phi_N^{n+1} = \phi_N^n + coef_b \phi_{amb}, \quad (228)$$

对应第 84 行到第 90 行: `A[2, i] = -coef_i` 是下对角, `b[i] += coef_b * Cair` 是右端增加的那一项; 温度的写法完全相同, 区别只有四处: 面系数用 `k`、除法用 $\kappa = \rho_i c_{p,i}$ 、边界系数用 `cm.h` 而不是 `cm.hm`、环境值用 `Tair`。

23.4 Picard 迭代为什么能处理非线性

非线性指方程里含有未知量的非线性函数: 本题的扩散系数、导热系数、密度和比热容都依赖 ϕ 和 T , 所以离散方程组的系数里也含有未知量, 不能直接解。**Picard 迭代**是求解不动点的方法: 先给未知量一个猜测值, 用它算出所有系数, 把方程当线性方程解一遍得到新值, 再拿新值算系数、再解一遍, 如此重复, 写成 $\phi^{(m+1)} = \Phi(\phi^{(m)})$, 其中 Φ 表示先算系数再解线性方程这一整步操作。如果 Φ 是**收缩映射**, 也就是把任意两个猜测值之间的距离缩小, 反复做下去就会收敛到唯一的不动点; 本题的收缩性来自一步之内含水率与温度的变化只有 Δt 量级, 所以物性在迭代中变化很小, Φ 缩小差距的比例也是 Δt 量级。代码第 69 行把新值初始化成旧值, 第 70 行做 `n_picard` 次循环, 第 71 行在每轮开始时用当前猜测值算物性, 接着第 72 行到第 100 行组装并求解水分的三对角方程组、第 101 行到第 128 行组装并求解温度的。

为什么要这样做: 为什么用 Picard 迭代而不是别的办法

更快的办法是牛顿法, 但它要计算**雅可比矩阵**, 也就是把每个方程对每个未知量求一阶偏导数排成的大矩阵。本题未知量有两千多个 ($N = 400$ 时是 2×401 个), 雅可比矩阵的规模与组装量都远大于三对角矩阵, 还要额外推导偏导数表达式; Picard 迭代不需要雅可比矩阵, 复用的是同一套三对角组装代码, 只是把系数换一换。程序里的迭代次数固定 3 次、不判收敛: 收缩比是 Δt 量级, 第三轮以后系数的改变已经远小于后向欧拉自身的截断误差。复跑报告第 6 节可作旁证: $N = 400$ 、 $\Delta t = 0.25$ 秒时表面含水率是 1.51094433, 与谱配置法的 1.51091810 相差 2.6×10^{-5} , 而这个差别随步长按一阶缩小 ($N = 200$ 时从 $\Delta t = 1$ 秒的 1.51102304 变到 $\Delta t = 0.5$ 秒的 1.51097924, 变化 4.4×10^{-5} ; $N = 400$ 时从 $\Delta t = 0.5$ 秒的 1.51096623 变到 $\Delta t = 0.25$ 秒的 1.51094433, 变化 2.2×10^{-5} , 正好减半), 说明剩下的差别来自时间离散而不是迭代次数不够。

23.5 追赶法的完整推导

第 100 行和第 128 行调用自己写的求解函数 `_thomas`, 文件第 138 行到第 158 行:

```
def _thomas(A, b):
```

"""追赶法 (Thomas 算法) 解三对角系统 $A x = b$, 返回解向量 x 。

A 为 $(3, n)$ 的紧凑带: $A[0]$ = 上对角 (偏移 +1)、 $A[1]$ = 主对角、 $A[2]$ = 下对角

(偏移 -1)。不选主元: 本模块的矩阵由隐式扩散加正的对角项构成, 主对角严格大于两个非对角元之和 (严格对角占优), 消元过程中不会出现零主元。

"""

```
n = len(b)
cp = np.zeros(n)
dp = np.zeros(n)
cp[0] = A[0, 0] / A[1, 0]
dp[0] = b[0] / A[1, 0]
for i in range(1, n):
    m = A[1, i] - A[2, i] * cp[i - 1]
    cp[i] = A[0, i] / m
    dp[i] = (b[i] - A[2, i] * dp[i - 1]) / m
x = np.zeros(n)
x[-1] = dp[-1]
for i in range(n - 2, -1, -1):
    x[i] = dp[i] - cp[i] * x[i + 1]
return x
```

调用处第 100 行是 $C_{\text{new}} = \text{_thomas}(A, b)$ 、第 128 行是 $T_{\text{new}} = \text{_thomas}(A, b)$, 两处只换了场, 矩阵与右端都是刚组装好的。

追赶法 (也叫 Thomas 算法) 是解三对角方程组的直接方法, 分两趟: 第一趟从上往下消元, 第二趟从下往上回代。设未知量是 $x_0, \dots, x_{n-1}$, 第 i 个方程是

$$c_i x_{i-1} + b_i x_i + a_i x_{i+1} = d_i, \quad (229)$$

其中 $c_0 = 0$ (第一个方程没有 x_{-1})、 $a_{n-1} = 0$ (最后一个方程没有 x_n); 对应到代码: a_i 存在 $A[0, i]$ 、 b_i 存在 $A[1, i]$ 、 c_i 存在 $A[2, i]$ 、 d_i 就是 $b[i]$ 。

第一趟, 前向消元。目标是每行都化成 $x_i = dp_i - cp_i x_{i+1}$ 。先取 $i = 0$ 并用 $c_0 = 0$, 得 $b_0 x_0 + a_0 x_1 = d_0$, 两边除以 b_0 后记 $cp_0 = \frac{a_0}{b_0}$ 、 $dp_0 = \frac{d_0}{b_0}$, 就得到 $x_0 = dp_0 - cp_0 x_1$, 这就是代码第 148 行和第 149 行。再看能不能递推下去: 取 $i = 1$ 得 $c_1 x_0 + b_1 x_1 + a_1 x_2 = d_1$, 把 $x_0 = dp_0 - cp_0 x_1$ 代进去并展开,

$$\begin{aligned} c_1 (dp_0 - cp_0 x_1) + b_1 x_1 + a_1 x_2 &= d_1 \\ \implies (b_1 - c_1 cp_0) x_1 + a_1 x_2 &= d_1 - c_1 dp_0 \implies cp_1 = \frac{a_1}{m}, \quad dp_1 = \frac{d_1 - c_1 dp_0}{m}, \end{aligned} \quad (230)$$

其中 $m = b_1 - c_1cp_0$ ，这就得到 $x_1 = dp_1 - cp_1x_2$ ，与代码第 150 行到第 153 行一致 ($A[2, i] * cp[i - 1]$ 就是 c_1cp_0)。把同样的做法重复下去：假设已有 $x_{i-1} = dp_{i-1} - cp_{i-1}x_i$ ，代入第 i 个方程并展开，

$$c_i(dp_{i-1} - cp_{i-1}x_i) + b_ix_i + a_ix_{i+1} = d_i \implies (b_i - c_i cp_{i-1})x_i + a_ix_{i+1} = d_i - c_i dp_{i-1}, \quad (231)$$

于是前向消元的递推式是

$$cp_i = \frac{a_i}{b_i - c_i cp_{i-1}}, \quad dp_i = \frac{d_i - c_i dp_{i-1}}{b_i - c_i cp_{i-1}}, \quad (232)$$

对应代码第 150 行到第 153 行的循环。**第二趟，后向回代。**最后一行没有 x_n ，即 $a_{n-1} = 0$ ，由式 (232) 得 $cp_{n-1} = 0$ ，所以 $x_{n-1} = dp_{n-1}$ ，这就是代码第 155 行的 $x[-1] = dp[-1]$ 。有了最后一个未知量，其余逐个往回收：由 $x_i = dp_i - cp_ix_{i+1}$ 先算 x_{n-2} 、再算 x_{n-3} ，一直算到 x_0 ，这就是代码第 156 行和第 157 行的倒序循环。

为什么要这样做：为什么不用高斯消元

高斯消元解一般的 n 阶方程组要约 $n^3/3$ 次乘法，本题 $N = 400$ 时 $n = 401$ 、约两千万次；追赶法利用稀疏结构，每趟只做 n 次乘加、总共约 $8n$ 次也就是几千次。每走一个时间步都要解两次方程组，这个差别直接决定程序能不能跑完。

一个 3 乘 3 的手算例子。上面几步都是符号推导，看不出数具体怎么走。下面取一个能手算完的小例子，把前向消元与后向回代完整走一遍，每一步都写出中间结果，可以拿计算器逐行核对。

为什么要这样做：为什么取这个方程组

取 $n = 3$ ，方程组的形状与程序里遇到的一模一样：主对角是正的、两个次对角是负的、两个角上的元素是零。

$$\begin{cases} 2x_0 - x_1 = 2.5, \\ -x_0 + 2x_1 - x_2 = 0, \\ -x_1 + 2x_2 = 2.5, \end{cases} \quad (233)$$

$$a = (-1, -1, 0), \quad b = (2, 2, 2), \quad c = (0, -1, -1), \quad d = (2.5, 0, 2.5).$$

两条对角线的端点为什么是零，在这个小例子里看得最清楚： $a_2 = 0$ 是因为第三个方程里没有 x_3 ， $c_0 = 0$ 是因为第一个方程里没有 x_{-1} 。这两个零不是凑出来的，是三对角结构本身的标志，程序里也是靠它们让第一行与最后一行各少一个非对角元。

右端为什么取 2.5、0、2.5。把 $x_0 = x_1 = x_2 = 2.5$ 代进式 (233) 的三个方程：第一行 $2 \times 2.5 - 2.5 = 5 - 2.5 = 2.5$ ，第二行 $-2.5 + 2 \times 2.5 - 2.5 = -2.5 + 5 - 2.5 = 0$ ，第三

行 $-2.5 + 2 \times 2.5 = -2.5 + 5 = 2.5$ ，恰好就是右端的那三个数。所以我们事先就知道精确解是 $x = (2.5, 2.5, 2.5)$ ——算完可以直接对答案，不必再另想办法验证。这是设计这个例子的全部用意：把「算对没有」变成一个可以一眼看出来的问题。

第一趟：前向消元。

第 1 步 初始化：把第一个方程写成 $x_0 = dp_0 - cp_0x_1$ （代码第 148 行和第 149 行）

第一个方程是 $2x_0 - x_1 = 2.5$ ，里面有两个未知量，先解出 x_0 。两边同除以 $b_0 = 2$ ：

$$x_0 = \frac{2.5}{2} + \frac{1}{2}x_1 = 1.25 + 0.5x_1. \quad (234)$$

（写成加法是因为移项： $-x_1$ 移到右边变成 $+x_1$ ，再除以 2。）把它对照代码用的形式 $x_0 = dp_0 - cp_0x_1$ ，逐项对上：

$$dp_0 = \frac{d_0}{b_0} = \frac{2.5}{2} = 1.25, \quad cp_0 = \frac{a_0}{b_0} = \frac{-1}{2} = -0.5. \quad (235)$$

注意 cp_0 是负数。这不影响后面的任何推理，但它提醒我们： cp_i 可以为负，所以后面证明里必须用绝对值，不能直接说「 cp_i 小于 1」。

这一步的分母是 $b_0 = 2$ ，前向消元里第一次除法的除数就是它。

第 2 步 用第一个关系式消去 x_0 （代码第 150 行到第 153 行， $i = 1$ ）

第二个方程是 $-x_0 + 2x_1 - x_2 = 0$ ，里面还带着 x_0 ，要把它消掉。把上一步得到的 $x_0 = 1.25 + 0.5x_1$ 整个代进去：

$$-(1.25 + 0.5x_1) + 2x_1 - x_2 = 0 \implies -1.25 - 0.5x_1 + 2x_1 - x_2 = 0 \implies 1.5x_1 - x_2 = 1.25. \quad (236)$$

x_0 不见了，第二个方程现在只含 x_1 与 x_2 ——这正是前向消元要达到的效果：每消一次，就少一个未知量。

接着把 $1.5x_1 - x_2 = 1.25$ 也整理成 $x_1 = dp_1 - cp_1x_2$ 的形式。移项得 $1.5x_1 = 1.25 + x_2$ ，两边除以 1.5：

$$x_1 = \frac{1.25}{1.5} + \frac{1}{1.5}x_2 = \frac{5}{6} + \frac{2}{3}x_2. \quad (237)$$

对照 $x_1 = dp_1 - cp_1x_2$ ，得 $dp_1 = \frac{1.25}{1.5} = \frac{5}{6} \approx 0.833333$ 、 $cp_1 = -\frac{1}{1.5} = -\frac{2}{3} \approx -0.666667$ 。

用递推公式算一遍，结果应当相同。式 (232) 给出的是 $cp_i = \frac{a_i}{b_i - c_i cp_{i-1}}$ 、 $dp_i = \frac{d_i - c_i dp_{i-1}}{b_i - c_i cp_{i-1}}$ 。先算分母：

$$m = b_1 - c_1 cp_0 = 2 - (-1) \times (-0.5) = 2 - 0.5 = 1.5. \quad (238)$$

这里的符号要小心： $c_1 = -1$ 、 $cp_0 = -0.5$ ，两个负数相乘得 $+0.5$ ，所以是 2 减 0.5。然后

$$cp_1 = \frac{a_1}{m} = \frac{-1}{1.5} = -\frac{2}{3}, \quad dp_1 = \frac{d_1 - c_1 dp_0}{m} = \frac{0 - (-1) \times 1.25}{1.5} = \frac{1.25}{1.5} = \frac{5}{6}. \quad (239)$$

两种算法给出同一个数，说明「代数代入」与「递推公式」确实是同一件事，只是递推公式把同样的事情写成了可以循环的形式。

第 3 步 用第二个关系式消去 x_1 ($i = 2$)

第三个方程是 $-x_1 + 2x_2 = 2.5$ 。把 $x_1 = \frac{5}{6} + \frac{2}{3}x_2$ 代进去：

$$-\left(\frac{5}{6} + \frac{2}{3}x_2\right) + 2x_2 = 2.5 \implies -\frac{5}{6} + \left(2 - \frac{2}{3}\right)x_2 = 2.5 \implies -\frac{5}{6} + \frac{4}{3}x_2 = 2.5. \quad (240)$$

然后移项得 $\frac{4}{3}x_2 = 2.5 + \frac{5}{6}$ 。把右边的两个数通分：

$$2.5 + \frac{5}{6} = \frac{15}{6} + \frac{5}{6} = \frac{20}{6} = \frac{10}{3}, \quad (241)$$

所以

$$dp_2 = \frac{d_2 - c_2 dp_1}{m} = \frac{\frac{10}{3}}{\frac{4}{3}} = \frac{10}{3} \times \frac{3}{4} = 2.5. \quad (242)$$

这里的分母同样可以对照着看： $m = b_2 - c_2 cp_1 = 2 - (-1) \times \left(-\frac{2}{3}\right) = 2 - \frac{2}{3} = \frac{4}{3}$ 。它与上面代数代入得到的系数 $\left(2 - \frac{2}{3}\right)$ 完全一致。 cp_1 是负数这件事在这里起了作用： $-c_2 cp_1$ 里的两个负号相消，所以是 2 减正数 $\frac{2}{3}$ ，而不是 2 加 $\frac{2}{3}$ 。

最后一行没有上对角元素 ($a_2 = 0$)，于是

$$cp_2 = \frac{a_2}{m} = \frac{0}{4/3} = 0. \quad (243)$$

这个零是预料之中的，而且很重要：正因为 $cp_2 = 0$ ，回代才能从 $x_2 = dp_2$ 起步。如果最后一步的 cp_2 不是零，回代就没有起点。

前向消元的结果汇总如下。

i	分母 $m = b_i - c_i cp_{i-1}$	cp_i	dp_i
0	——（无需消元）	-0.5	1.25
1	1.5	$-\frac{2}{3} \approx -0.666667$	$\frac{5}{6} \approx 0.833333$
2	$\frac{4}{3} \approx 1.333333$	0	2.5

第二趟：后向回代。

第 4 步 从最后一个未知量往回逐个解出（代码第 155 行到第 157 行）

回代的起点是最后一个方程。因为 $cp_2 = 0$ ，第三个关系式已经写成 $x_2 = dp_2$ ，于是

$$x_2 = dp_2 = 2.5. \quad (244)$$

有了 x_2 ，代回第二个关系式 $x_1 = dp_1 - cp_1 x_2$ ：

$$x_1 = \frac{5}{6} - \left(-\frac{2}{3}\right) \times 2.5 = \frac{5}{6} + \frac{2}{3} \times 2.5 = \frac{5}{6} + \frac{5}{3} = \frac{5}{6} + \frac{10}{6} = \frac{15}{6} = 2.5. \quad (245)$$

注意中间那一步： cp_1 是负的，所以「减 $cp_1 x_2$ 」变成「加」。这是回代时最容易写错符号的地方，手算时可以特意停一下检查。

有了 x_1 ，再代回第一个关系式 $x_0 = dp_0 - cp_0 x_1$ ：

$$x_0 = 1.25 - (-0.5) \times 2.5 = 1.25 + 1.25 = 2.5. \quad (246)$$

同样地， cp_0 也是负的，两个负号相消。到这里三个未知量都求出来了。

核对。三个未知量都是 2.5，与事先知道的精确解一致。再代回原方程组逐行核对一遍：

$$\begin{aligned} 2x_0 - x_1 &= 5 - 2.5 = 2.5 && \text{等于第一行右端,} \\ -x_0 + 2x_1 - x_2 &= -2.5 + 5 - 2.5 = 0 && \text{等于第二行右端,} \\ -x_1 + 2x_2 &= -2.5 + 5 = 2.5 && \text{等于第三行右端.} \end{aligned} \quad (247)$$

三行都成立，追赶法的结果正确。

这一节要记住的 这个例子可以拿计算器逐步复核。要留意三点：一是 cp_0 与 cp_1 都是负数，代回时是「减负数等于加」，符号不要弄错；二是 cp_2 恰好为零，这是三对角结构的必然结果，也是回代能起步的原因；三是 3×3 的方程组手算两遍不过一分钟，但程序里 $N = 400$ 时未知量有 401 个，靠的就是把上面这套动作重复 401 次。

23.6 追赶法为什么不需要选主元

选主元是什么，一般为什么要做。高斯消元在消元过程中要做一连串除法，除数是当时的主对角元素。如果某个除数很小、甚至接近零，那么前面累积的一点点舍入误差会被这个除法放大很多倍，最后算出来的解可能与真实解相差很远。避免的办法叫**选主元**：在消到某一列时，先在该列里挑一个绝对值最大的元素，把它所在的行换到当前行，再继续消元。代价是每一步都要比较与交换，程序会复杂一些。

那本题为什么可以不做这件事？因为本题的矩阵有一个很强的性质，它保证除数永远不可能接近零。下面把这个性质说清楚，并且把它到底「保证了多少」证明出来。

矩阵的具体形状。程序里水分场的矩阵是这样的（温度场结构完全相同，只是系数换成 k 与 ρc_p ）：

位置	元素
主对角	$1 + cp_i + cm_i$
上对角	$-cp_i$
下对角	$-cm_i$

其中的两个系数是

$$cp_i = \frac{\Delta t gf_p}{R^2 \kappa_i vol_i}, \quad cm_i = \frac{\Delta t gf_m}{R^2 \kappa_i vol_i}. \quad (248)$$

分子上的 gf_p 、 gf_m 是面系数，等于 $\xi_{i\pm 1/2}$ 乘上两侧物性的算术平均再除以 h 。扩散系数、导热系数、半径、体积测度都是正数，时间步长也是正数，所以分母的三个因子 R^2 、 κ_i 、 vol_i 全为正，于是 $cp_i > 0$ 、 $cm_i > 0$ ，而且 gf_p 、 gf_m 也大于零。

这样每一行的三个元素就是：一个正的主对角，加上两个负的次对角。这个符号结构是后面一切结论的出发点。

严格对角占优。定义。一个矩阵叫**严格对角占优**，是指每一行主对角元素的绝对值，都严格大于该行其余元素绝对值之和。

本题每一行「其余元素绝对值之和」是

$$|-cp_i| + |-cm_i| = cp_i + cm_i, \quad (249)$$

用主对角减去它：

$$(1 + cp_i + cm_i) - (cp_i + cm_i) = 1 > 0. \quad (250)$$

差恰好等于 1。这个结果值得多看一眼：它不依赖 i 、不依赖时间步长 Δt 、不依赖节点数 N 、也不依赖物性的大小，是一个恒等于 1 的常数。也就是说，本题的矩阵不但严格对角占优，而且占优的余量有一个与所有参数无关的正下界。这一点后面会直接用到。

顺带说明那个 1 是哪里来的：它来自隐式格式左边 ϕ_i^{n+1} 的系数，也就是「这一步的值由上一步的值加上增量决定」里那个恒为 1 的部分。扩散项无论多强，都只是在这个 1 上面加正数，不会把占优性破坏掉。

要证的命题。用式 (232) 做前向消元时，对所有的 i 都有 $|cp_i| < 1$ ，因而每一步的分母 $m = b_i - c_i cp_{i-1}$ 都不为零。

先说明这个命题为什么正是我们需要的。程序里每一步除法用的除数就是 m 。若 $|cp_i| < 1$ 恒成立，就等于说除数永远比它要消掉的那个系数大，于是既不会出现除以零，也不会出现「除以一个很小的数」把误差放大。同时，回代的递推式 $x_i = dp_i - cp_i x_{i+1}$ 里， x_{i+1} 的误差传到 x_i 上要乘一个因子 $|cp_i| < 1$ ，误差往回传的时候是缩小的而不是放大的，这正是「不需要选主元」的真正含义。

证明。用数学归纳法。记原矩阵的主对角为 b_i 、上对角为 a_i 、下对角为 c_i ，它们满足严格对角占优

$$|b_i| > |a_i| + |c_i|. \quad (251)$$

本题里 $b_i = 1 + cp_i + cm_i > 0$ 、 $a_i = -cp_i < 0$ 、 $c_i = -cm_i < 0$ ，但下面的推理只用式 (251)，对一般情形也成立。

第一步：起点。最后一个方程没有上对角元，即 $a_{n-1} = 0$ ，这一点先记下；起点从第 0 行开始。由 $c_0 = 0$ ，第一个方程只有两项： $b_0 x_0 + a_0 x_1 = d_0$ 。两边除以 b_0 得 $cp_0 = a_0/b_0$ 。在式 (251) 里取 $i = 0$ 并注意 $c_0 = 0$ ：

$$|b_0| > |a_0| + |c_0| = |a_0|. \quad (252)$$

于是

$$|cp_0| = \frac{|a_0|}{|b_0|} < 1. \quad (253)$$

起点成立。用一句话说清楚这一步的意思：第一个方程的除数 b_0 比它要消掉的那个系数 a_0 大，所以第一次除法是安全的。

第二步：归纳假设。假设第 $i-1$ 步已经满足

$$|cp_{i-1}| \leq 1. \quad (254)$$

这里用 ≤ 1 而不是 < 1 是有意的：归纳步骤里只需要这个弱一些的条件，用弱条件证明更干净，而结论仍然可以加强到严格小于。

第三步：估计第 i 步的分母。第 i 步的分母是

$$m = b_i - c_i c p_{i-1}. \quad (255)$$

要证 $|m| > |a_i|$ ，这样才有 $|c p_i| = |a_i|/|m| < 1$ 。分三小步做。

(i) 用绝对值不等式拆开。对任意两个数 u 、 v 都有 $|u - v| \geq |u| - |v|$ 。取 $u = b_i$ 、 $v = c_i c p_{i-1}$ ，得

$$|m| = |b_i - c_i c p_{i-1}| \geq |b_i| - |c_i c p_{i-1}|. \quad (256)$$

(ii) 把乘积的绝对值拆成绝对值的乘积。因为 $|uv| = |u| |v|$ ，所以 $|c_i c p_{i-1}| = |c_i| |c p_{i-1}|$ ，代入上式：

$$|m| \geq |b_i| - |c_i| |c p_{i-1}|. \quad (257)$$

(iii) 用归纳假设把减去的量放大。由 $|c p_{i-1}| \leq 1$ 得 $|c_i| |c p_{i-1}| \leq |c_i|$ 。这一步的方向要看清：我们把被减去的那个量变大了，所以差只会变小，不等式仍然成立（估计下界时的标准手法）。于是

$$|m| \geq |b_i| - |c_i| |c p_{i-1}| \geq |b_i| - |c_i|. \quad (258)$$

第四步：用严格对角占优收口。由式 (251) 有 $|b_i| > |a_i| + |c_i|$ ，即

$$|b_i| - |c_i| > |a_i|. \quad (259)$$

把第三步与这一步接起来：

$$|m| \geq |b_i| - |c_i| > |a_i|. \quad (260)$$

于是

$$|c p_i| = \frac{|a_i|}{|m|} < 1. \quad (261)$$

归纳步骤完成。由起点与归纳步骤，对所有 i 都有 $|c p_i| < 1$ ，命题得证。□

拿手算例子对照。前面那个 3×3 方程组里，三个 $c p$ 分别是 $|c p_0| = 0.5 < 1$ 、 $|c p_1| = \frac{2}{3} \approx 0.667 < 1$ 、 $|c p_2| = 0 < 1$ ，三个数都落在单位区间内，与命题的结论一致；而且每一步真正算出来的分母 1.5 与 $\frac{4}{3}$ 都大于该步的分子 $|-1|$ ，除法确实没有放大任何东西。

这里要把话说准：这个手算例子的中间一行是 $|b_1| = 2$ 、 $|a_1| + |c_1| = 1 + 1 = 2$ ，只满足弱对角占优，不满足式 (251) 的严格不等号。所以在这一行上，式 (259) 给出的下界恰好是 $|m| \geq 2 - 1 = 1$ ，只能保证 $|m| \geq |a_1|$ ，套不进严格小于的结论。真正算出来的 $m = 1.5$ 比这个下界大，结论 $|c p_1| = \frac{2}{3} < 1$ 仍然成立。这正好说明式 (258) 是一个最坏情况的估计：它保证的是「无论物性、步长、节点数怎么变都不会出事」，而不是每一步的真实值都恰好贴着这个界。程序里的矩阵则每一行都满足严格不等号，因为主对角上那个 1 让式 (250) 的余量恒为 1。

为什么要这样做：严格对角占优在这里到底起了什么作用

它保证的是占优性在消元过程中不会丢失。对一般的矩阵，消元几次之后主对角可能被磨得很小，所以必须选主元；而严格对角占优的矩阵每消一次，剩下的子矩阵仍然严格对角占优（这本身也是一个小一号的归纳法），所以上面的归纳假设可以一直用下去，不会到某一步失效。本题因为占优余量恒等于 1（式 (250)），这个性质尤其干净：与节点数、步长、物性都无关，所以结论在整条时间推进过程中一直有效。

23.7 顺带看清误差是怎么来回传的

命题证完之后，还可以把「误差不放大」这件事说得更具体。回代递推式是

$$x_i = dp_i - cp_i x_{i+1}. \quad (262)$$

设 x_{i+1} 上带着一个误差 ε （来自更早的浮点舍入），把它代进去， x_i 上就多出 $cp_i \varepsilon$ 这一项。取绝对值：

$$|cp_i \varepsilon| = |cp_i| |\varepsilon| < |\varepsilon|. \quad (263)$$

也就是说，误差每来回传一步就乘一个小于 1 的因子，越往前越小，不会越滚越大。这正是「不用选主元」的实践含义：不是说不存在舍入误差，而是说这些误差在回代过程中被自然地压下去了。

前面的手算例子也是这样，三个 cp 值给出的衰减倍数是具体的： $cp_2 = 0$ 意味着 x_2 上的误差完全不会传到 x_1 ，也就是衰减到 0 倍； $|cp_1| = \frac{2}{3}$ 意味着传到 x_1 时只剩三分之二； $|cp_0| = 0.5$ 意味着再传到 x_0 时只剩一半。从最后一个未知量一路传到第一个未知量，误差被乘上

$$|cp_{n-2}| \cdot |cp_{n-3}| \cdots |cp_0| < \frac{2}{3} \times 0.5 = \frac{1}{3}, \quad (264)$$

也就是说， x_2 上的半点误差传到 x_0 时已经不到六分之一。程序里 $n = 401$ ，这个连乘只会更小。

这一节要记住的 严格对角占优 $\Rightarrow$ 前向消元的分母始终大于分子 $\Rightarrow |cp_i| < 1 \Rightarrow$ 既没有零主元、误差也不会被放大 $\Rightarrow$ 不需要选主元。本题的占优余量恒等于 1，所以这条结论与节点数、步长、物性都无关。

23.8 采样与报告

文件第 161 行到第 166 行：

```
def sample_xi(xi, vals, x_target):
    """把节点值 vals 线性插值到目标位置 x_target（无量纲坐标 i/N）。
```

```
x_target 可为标量或数组，返回同形状结果；超出 xi 范围时取端点值（np.interp
默认）。
"""
return np.interp(np.asarray(x_target, float), xi, vals)
```

`np.interp` 是 `numpy` 的线性插值函数，三个参数依次是目标位置、已知点的横坐标、已知点的纵坐标，在已知点之间用直线连接、在之外取端点值，并且是在 ξ 坐标上插值。`run_all.py` 用它取表面处的值，传进去的目标位置是 $0.02/0.02 = 1$ ，正好落在最后一个节点上，取到的就是表面节点本身的值。

把节点上的值线性插值到目标位置，有限体积用线性插值就够了，因为格式本身只有二阶精度，用更高阶的插值没有意义。这与主求解器那边用重心插值形成对比——那边是谱精度，插值也必须匹配。

这一节要记住的 `fv_uniform.py` 的关键词是「从零自己写」：自己组装三对角矩阵、自己写 Picard 迭代、自己写追赶法。它与 `fv_theta.py` 共用有限体积的思想，但时间格式与非线性处理完全不同，因此两者与主求解器构成三族互证。

24 结果文件：make_results.py 逐段解释

24.1 题目要求的表格与文件格式

题目在问题 1 里要求：按表 1 和表 2 的格式分别给出 100、300、600、900、1200、1500、1800 秒，以及到药材中心距离 0、0.5、1、1.5、2 厘米处的结果，并将 1800 秒内每隔 1 秒、距离每隔 0.1 厘米的完整结果保存到 `result1.xlsx`，所有结果保留四位小数，下同。问题 2 要求 3 小时内每隔 1 秒、距离每隔 0.1 厘米的结果存进 `result2.xlsx`；问题 3 要求水分浓度每隔 60 秒、距离每隔 0.1 厘米的结果存进 `result3.xlsx`；问题 4 要求每隔 60 秒、距离每隔 0.1 厘米的结果存进 `result4.xlsx`。题目附录 3 说明了编排方式：附件 3 是结果文件夹、包括这四个文件；`result1.xlsx` 在温度与水分浓度两个工作表中分别保存，A 列为时间（单位秒）、第 1 行为到药材中心的距离（单位厘米）；`result2.xlsx` 的编排相同；`result3.xlsx` 与 `result4.xlsx` 只保存水分浓度，编排方式也相同。要注意两件事：论文里的表 1 到表 6 每张只列 5 个位置（0、0.5、1、1.5、2 厘米，表 6 的最后一列换成药材表面），而文件里要列全 21 个距离；表 5 和表 6 的最后一行是烘干结束时间，所以文件最后一行的时间不是等间隔网格上的点。

文件第 44 行到第 47 行是这张表格的四个常量：

```
RCOLS = np.round(np.arange(0.0, 2.0001, 0.1), 10) # 0.0 .. 2.0 cm
```

```
RC = RCOLS * 1e-2 # m
CRIT = 0.15
N = 64
```

第 44 行生成 0.0 到 2.0 厘米、步长 0.1 厘米的 21 个距离，这就是题目要求的表格列。`np.arange(0.0, 2.0001, 0.1)` 从 0 起每次加 0.1、到不超过 2.0001 为止；终点写 2.0001 而不是 2.0，是因为浮点累加可能让最后一个值变成 1.999999999999998，写 2.0 会把它排除掉、少一列。外面的 `np.round(..., 10)` 保留 10 位小数、用来消掉浮点表示误差，否则 $0.1 + 0.1 + 0.1$ 会得到 0.30000000000000004，表头就不好看了。第 45 行把厘米换算成米，因为求解器里的长度单位是米；表头用厘米、计算用米，中间差 100 倍，这两行最容易搞错。第 46 行是烘干判据阈值，第 47 行是生产网格的节点数，与论文所有表格同一口径。

文件第 40 行到第 42 行算输出目录：

```
OUTDIR = os.path.abspath(os.path.join(os.path.dirname(os.path.abspath(
    __file__)),
                                     "..", "result"))
os.makedirs(OUTDIR, exist_ok=True)
```

`__file__` 是当前文件自己的路径，取两次 `os.path.dirname` 得到包的根目录，再拼上 `result` 两个字；这样算出的路径与在哪里运行命令无关，整个文件夹搬走也能正确输出。`os.makedirs(OUTDIR, exist_ok=True)` 创建目录，第二个参数表示已存在时不报错。

文件开头还有一道防护，也就是第 27 行到第 30 行：

```
if __name__ != "__main__":
    raise RuntimeError(
        "make_results 的生成逻辑在模块顶层，import 会直接覆盖 result/*.xlsx。"
        "请用 `python make_results.py` 运行；确实需要 import 时，请先把生成逻辑移入 main()。")
```

因为这个脚本会直接覆盖已经交付的结果文件，所以拦一道，防止别人误 `import` 把结果冲掉。

24.2 写表函数逐行说明

文件第 50 行到第 73 行，函数 `write_sheet` 的全部内容：

```
def write_sheet(ws, times, values, header_last=None):
    """把一个（时刻 x 半径）的数值矩阵写成一张工作表。
```

```

第 1 行是表头: A1 为模板规定的交叉标题, B1 起依次是 21 个距离 [cm];
A 列是各报告时刻 [s]; 表体是 values[i, j], 四舍五入到 4 位小数。
values 为 None 或非有限值时该单元格留空 (问题 4 里已收缩掉的半径即如此处理);
header_last 不为 None 时, 另用它在最后一列加一列表头与数值, 如 result4 的"
药材表面"。
"""
ws.cell(row=1, column=1, value="时间\\到药材中心的距离")
for j, rc in enumerate(RCOLS):
    ws.cell(row=1, column=2 + j, value=float(rc))
if header_last is not None:
    ws.cell(row=1, column=2 + len(RCOLS), value=header_last)
for i in range(len(times)):
    ws.cell(row=2 + i, column=1, value=float(times[i]))
    for j in range(len(RCOLS)):
        v = values[i, j]
        if v is not None and np.isfinite(v):
            ws.cell(row=2 + i, column=2 + j, value=round(float(v), 4))
if header_last is not None:
    for i in range(len(times)):
        v = values[i, len(RCOLS)]
        if v is not None and np.isfinite(v):
            ws.cell(row=2 + i, column=2 + len(RCOLS), value=round(float
(v), 4))

```

这个函数把一个「时刻乘半径」的数值矩阵写成一张工作表。第 50 行的四个参数是工作表对象、时间数组、数值数组和可选的最末一列表头, 行号与列号都从 1 开始数。

第 58 行写第一行第一格的文字: 括号里是两个反斜杠, 在 Python 里转义成一个反斜杠字符, 所以单元格里存的是时间、一个反斜杠、到药材中心的距离, 这一个单元格同时充当行标题与列标题的分隔。第 59 行和第 60 行写距离表头, `enumerate` 同时给出下标与元素, 第 j 个距离写到第 $2+j$ 列, 所以第一个距离在 B 列、A 列留给时间; `float(rc)` 把数值转成浮点数再写, 单元格里是数字而不是文字, 表格软件可以直接计算。第 61 行和第 62 行处理 `result4` 多出来的那一列: 传了 `header_last` 就写到第 $2+21=23$ 列, `result1` 到 `result3` 不传, 走不进这个分支, 表头只有 22 列。

第 63 行到第 68 行写数据区: 外层循环走行, 第 i 个时刻写到第 $2+i$ 行 (第 1 行是表头), 第 64 行写时间列, 第 65 行到第 68 行写这一行的 21 个数值。第 67 行是留空的关键: 只有当值不是空、也不是 `nan` 时才写, `np.isfinite` 判断一个数是不是有限

数，`nan` 和无穷大都会返回假；条件不成立时这一格什么都不写。新工作簿里没被赋值过的单元格本来就是空的，所以留空的实现方式就是根本不碰它，而不是写一个空字符串进去，两者有区别：空字符串会让单元格看起来非空。第 68 行把数值四舍五入到 4 位小数再写，这是题目要求的精度。`round` 是 Python 内置函数，第二个参数是保留的小数位数；它是在二进制浮点数上做的四舍五入，如果某个数正好落在第 5 位小数的中点上、而它在二进制里表示不出来，结果可能比十进制四舍五入少一个最低位；本题的解都是带十几位有效数字的浮点数，恰好落在中点上的情形没有出现，程序也没有为这种情形做特殊处理。写进去的是四舍五入后的数本身而不是显示格式，所以读回来也是四位小数。第 69 行到第 73 行写最后一列，逻辑与数据区相同，只是列号换成 $2 + 21 = 23$ 。

为什么要这样做：问题 4 里收缩掉的半径是怎么留空的

就是靠第 67 行那个判断。调用方在 `result4` 里对已经收缩掉的半径填的是 `nan`，`nan` 不是有限数，会被自动跳过。这样「物理上不存在的位置」在表里就是一个真正的空格，而不是被填上某个近似的数。

24.3 采样函数与坐标换算

文件第 76 行到第 88 行：

```
def sample(s, sol, u_targets, which):
    """把解采样成矩阵，返回形状（报告时刻数，采样点数）。

    s          求解器实例（用它的 interp_u 做插值）；
    sol         solve_ivp 的解对象，sol.y 的各列依次对应各报告时刻；
    u_targets   目标归一化半径  $u = (r/R)^2$ （无量纲，故与半径用什么单位无关）；
    which       "T" 取温度 [degC]，取其它值则取干基含水率 [kg/kg]。
    """
    out = np.empty((sol.y.shape[1], len(u_targets)))
    for i in range(sol.y.shape[1]):
        C, T = s.interp_u(sol.y[:, i], u_targets)
        out[i] = T if which == "T" else C
    return out
```

遍历每一个报告时刻，把该时刻的解插值到目标坐标上。第 84 行建立空数组：`sol.y` 是积分器返回的解矩阵，形状是（状态维数，输出时刻数），状态维数是 $2(N+1)$ ，所以 `sol.y.shape[1]` 就是输出时刻个数，新数组形状是（时刻数，目标点个数）。第 85 行到第 87 行逐时刻插值：`s.interp_u` 是第三部分讲过的重心插值，它按 u 变量插值并一次返回含水率与温度两组值；第 87 行按参数 `which` 决定存哪一组。

注意目标坐标是 u 而不是 r ，调用处是这么算的：

```
U = (RC / cm.R0) ** 2
```

插值必须在求解坐标系里做，所以先把物理距离换算成 $u = (r/R_0)^2$ 。which == "T" 取温度，否则取含水率，两个场共用同一个采样函数。

为什么要这样做：为什么不在 r 坐标上插值

因为谱解本身是 u 的多项式。在 u 上插值才是谱精度的；若先在 r 上均匀取点再插值，等于对多项式做了非最优采样，精度会掉。

24.4 result1 与 result2

文件第 94 行到第 105 行，生成 **result1.xlsx**：

```
print("result1 ...")
s = HerbSolver(N, prob=1)
tt = np.arange(1.0, 1801.0, 1.0)
at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
sol = s.solve(1800.0, t_eval=tt, rtol=1e-11, atol=at)
U = (RC / cm.R0) ** 2
wb = openpyxl.Workbook()
ws = wb.active; ws.title = "温度"
write_sheet(ws, tt, sample(s, sol, U, "T"))
ws2 = wb.create_sheet("水分浓度")
write_sheet(ws2, tt, sample(s, sol, U, "C"))
wb.save(os.path.join(OUTDIR, "result1.xlsx"))
```

第 94 行在屏幕上打印进度，因为这一步要算几分钟。第 95 行建立问题 1 的求解器， N 就是上面那个 64，物性用 **prob=1** 的常数物性。第 96 行构造时间列：从 1 秒到 1800 秒、间隔 1 秒，共 1800 个时刻、单位是秒；论文的表 1 只列 7 个时刻，而文件里每 1 秒给一个值，两者长度差很多。第 97 行构造容差向量：**np.concatenate** 把两段拼起来，前一段长度 $N + 1$ 对应含水率、绝对容差 10^{-13} ，后一段同样长度对应温度、绝对容差 10^{-11} ；顺序必须与状态向量的排列一致，也就是含水率在前、温度在后。第 98 行调用求解器，**rtol** 是相对容差、**t_eval** 指定只在这些时刻输出。第 99 行算出目标 u 数组：**RC** 是米制距离、**cm.R0** 是 0.02 米，相除得 0 到 1、平方后仍是 0 到 1；两端正好落在节点上，重心插值在 $u = 0$ 和 $u = 1$ 处直接取节点值。第 100 行建立新工作簿，新工作簿默认带一个工作表、用 **wb.active** 拿到。第 101 行把默认工作表改名为温度，分号把两条语句写在一行、作用与分两行相同；工作表名必须是题目附录 3 里写的温度与水分浓度，

因为阅卷时要按名字找。第 102 行把温度写进这个工作表，`sample` 的第四个参数是 "T"。第 103 行用 `wb.create_sheet` 追加第二个工作表并命名为水分浓度；第 104 行把含水率写进去、参数是 "C"。第 105 行保存文件，落在 < 包根 > /result/result1.xlsx。两张表共用同一个解对象 `sol`，只是取不同的物理量。

`result2` 的结构完全相同，也就是文件第 107 行到第 119 行，只把 `prob` 换成 2、时间延长到 10800 秒；第 112 行的容差放宽到 10^{-12} 与 10^{-10} 、比 `result1` 宽一个数量级；第 116 行和第 118 行复用上一段算出的 `U`，因为问题 2 也不收缩。一个文件里放两个工作表而不是两个文件，是因为题目附录 3 就是这么规定的。

24.5 烘干时间与事件函数

文件第 122 行到第 138 行：

```
def dry(s, shrink=False, rtol=1e-11):
    """求烘干时长 t_dry [s]，事件口径与 run_all.dry_time 相同。

    事件函数取状态中水分那段的最大值减 CRIT：含水率沿半径由中心向外递减，最大值在轴心
    u=0（表面先干、中心最后达标），故它穿过 0 的时刻就是"处处不高于 0.15 kg/kg"的首达
    时刻；terminal 配 direction=-1 表示只在下降方向触发。rtol 是积分的相对容差。
    shrink 在本函数体内并不使用：是否含收缩由构造求解器 s 时决定，保留它只是让调用处
    一眼看出这一路是问题 4。
    """
    at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])

    def ev(t, y):
        return np.max(y[:s.Np]) - CRIT
    ev.terminal = True
    ev.direction = -1
    e = s.solve(300000.0, rtol=rtol, atol=at, events=ev)
    return float(e.t_events[0][0])
```

这个函数求烘干时长，判据就是第二部分的式 (141)。

第 122 行的 `s` 是求解器对象、`rtol` 是相对容差；`shrink` 在函数体内没有被用到，因为是否收缩由求解器对象自己记着（问题四建立对象时传了 `shrink=True`），传这个

参数只是让调用处读起来清楚。

第 133 行到第 135 行定义**事件函数**：求解器在每个时间步计算它的值，当值穿过零时记录下这个时刻；本函数返回全场最大值与判据之差 $e(t) = \max_{0 \leq j \leq N} C_j(t) - 0.15$ ，还没干时 $e > 0$ 、干了时 $e < 0$ ，所以零点是烘干时刻。第 135 行 `ev.terminal = True` 表示事件发生后停止积分，不必白算到 300000 秒；第 136 行 `ev.direction = -1` 表示只在函数值下降穿过零时触发，干燥过程中最大值整体是下降的，加了这个条件以后触发更明确，而且初值是 2.55、远高于阈值 0.15，本来不会误触发，把方向写明确是一个好习惯。

第 137 行求解，时间上限给 300000 秒。这个数是这样定的：问题三的实际答案是 205574 秒，问题四更快，所以 300000 秒一定够；而事件是终止事件，达标即停，不会真的算那么久。好处是不用事先知道答案，代价是万一事件没触发程序要跑满 300000 秒才停。第 138 行取出事件时刻：`t_events` 是列表、每个事件对应一个数组，可能有多个触发点，`[0][0]` 取第一个事件的第一个时刻。

24.6 result3 与 result4

文件第 144 行到第 154 行，生成 `result3.xlsx`：

```
print("result3 ...")
s3 = HerbSolver(N, prob=3)
td3 = dry(s3)
tt3 = np.arange(60.0, np.floor(td3 / 60.0) * 60.0 + 1.0, 60.0)
tt3 = np.append(tt3, td3)
at3 = np.concatenate([np.full(s3.Np, 1e-13), np.full(s3.Np, 1e-11)])
sol3 = s3.solve(td3, t_eval=list(tt3), rtol=1e-11, atol=at3)
U3 = (RC / cm.R0) ** 2
wb = openpyxl.Workbook(); ws = wb.active; ws.title = "Sheet1"
write_sheet(ws, tt3, sample(s3, sol3, U3, "C"))
wb.save(os.path.join(OUTDIR, "result3.xlsx"))
```

第 146 行先求一次烘干时间，因为要先知道表格写到哪里。第 147 行构造时间列需要拆开看：`np.floor(td3 / 60.0) * 60.0` 是找出不超过烘干时间的、60 的最大整数倍，`np.floor` 是向下取整函数。以交付的结果文件回读值为例，烘干时间是 205574.5185452786 秒，除以 60 得 3426.24、向下取整是 3426、乘回 60 得 205560；于是 `np.arange` 从 60 起每次加 60 直到不超过 205561，得到 60, 120, ..., 205560 共 3426 个数；终点后加 1.0 是为了让 205560 被包含进来，否则浮点误差可能把它排除掉。第 148 行用 `np.append` 把精确的烘干时间接到时间列末尾：题目表 5 的最后一行是烘干结束时间、不是 60 秒的整数倍，所以最后一行的时间是一个带很多位小数的数。第 149 行构造容差向量、与

result1 相同。第 150 行求解，终止时刻就是烘干时间，**t_eval** 用 **list(tt3)** 转成列表，最后一个元素正好等于终止时刻。第 151 行算目标 u 数组，与第 99 行表达式相同、只是换了变量名 **U3**。第 152 行建立工作簿并把工作表命名为 **Sheet1**，题目没有规定这两个文件的工作表名，所以直接用默认名。第 153 行只写含水率、参数是“C”；没有传 **header_last**，表头是 22 列。

文件第 159 行到第 176 行，生成 **result4.xlsx**：

```
print("result4 ...")
s4 = HerbSolver(N, prob=4, shrink=True)
td4 = dry(s4, shrink=True)
tt4 = np.arange(60.0, np.floor(td4 / 60.0) * 60.0 + 1.0, 60.0)
tt4 = np.append(tt4, td4)
at4 = np.concatenate([np.full(s4.Np, 1e-13), np.full(s4.Np, 1e-11)])
sol4 = s4.solve(td4, t_eval=list(tt4), rtol=1e-11, atol=at4)
vals = np.full((len(tt4), len(RCOLS) + 1), np.nan)
for i, t in enumerate(tt4):
    Rt = s4.R_of(t)
    for j, r in enumerate(RC):
        if r <= Rt + 1e-12:          # 该物理位置仍然存在
            Cn, _ = s4.interp_u(sol4.y[:, i], (r / Rt) ** 2)
            vals[i, j] = Cn[0]
        vals[i, -1] = sol4.y[s4.N, i]    # 当前表面值
wb = openpyxl.Workbook(); ws = wb.active; ws.title = "Sheet1"
write_sheet(ws, tt4, vals, header_last="药材表面")
wb.save(os.path.join(OUTDIR, "result4.xlsx"))
```

第 160 行建立问题 4 的求解器、多传一个 **shrink=True**，这样求解器每一步按附件 2 的半径历史取当前半径；第 161 行求烘干时间；第 162 行和第 163 行构造时间列，做法与 **result3** 完全相同。以交付的结果文件回读，问题 4 的烘干时间是 182778.2814815244 秒，等于 3046×60 加 18.28 秒，所以均匀部分到 182760 秒共 3046 个数，追加烘干时间以后是 3047 行。第 166 行建立结果数组，形状是（时刻数，22）、初始值全是 **nan**；比距离列的 21 多一列、多出来的一列放表面值；先全部填 **nan**，后面把能算的格子算出来，剩下的 **nan** 在写表时被跳过、成为空格。

这是一个必须逐时刻做的换算。求解器工作在物质坐标 $u = \xi^2$ 上，而表格的列名是当前物理距离 r 。两者的关系是 $r = \xi R(t)$ ，所以对每个时刻、每个目标半径，要先算

$$u_j = \left(\frac{r_j}{R(t_i)} \right)^2 \quad (265)$$

再去插值，这正是第二部分第九章的式 (179)：分子是固定的物理距离，分母是当前半径、

随时间变化。

第 167 行按时刻循环，第 168 行取这一时刻的半径，`s4.R_of` 在 `shrink` 为真时调用附件 2 的插值函数。第 169 行到第 172 行按距离循环、逐个判断这个物理位置还在不在。第 170 行的条件是 r 不超过当前半径，多出来的 10^{-12} 米是容差、用来处理 r 与 $R(t)$ 相等时的浮点误差：典型情形是 $t = 0$ 附近 $r = 2.0$ 厘米正好等于初始半径， $r/R(t)$ 因为舍入可能算成 1.0000000000000002 ，插值会跑到区间外面去，加上容差就把它留在区间内。第 171 行做坐标换算与插值，换算关系就是式 (265)、分母是当前半径 `Rt`；`s4.interp_u` 返回两个值、这里只要含水率，所以第二个用下划线接住；插值结果是长度为 1 的数组，所以第 172 行取 `Cn[0]`。第 173 行单独算表面值，`sol4.y[s4.N, i]` 直接取第 N 个节点也就是 $u = 1$ 处的含水率、不需要插值，不论半径怎么变，表面永远是 $u = 1$ 那个节点。这一列对应题目表 6 的药材表面。

为什么要这样做：物理位置消失以后为什么必须留空，不能填外推值

因为那个位置已经不在药材里面了：半径缩到 1.374 厘米时， $r = 1.5$ 厘米处是空气，没有含水率这个量，填任何一个数都是凭空造出来的。更严重的是，如果按固定 u 去取值，取到的是另一个材料点的含水率，数值看起来完全正常却对应错误的位置，这种错误无法从表格本身发现；所以判断条件写在插值之前，不满足条件就跳过，让这一格保持 `nan`。用已有文件回读的数字说明留空的实际范围： $r = 1.5$ 厘米这一列有值的最晚一行是 $t = 13140$ 秒、从 $t = 13200$ 秒起为空，这与附件 2 的数据一致：半径在 $t = 12600$ 秒时是 1.510 厘米、在 $t = 14400$ 秒时是 1.477 厘米，按分段线性插值在 $t = 13145.5$ 秒降到 1.5 厘米，所以 13140 秒这一行还有值、13200 秒这一行就没有了。其余各列最后一个有值行是： $r = 1.3$ 厘米在 $t = 30420$ 秒、 $r = 1.4$ 厘米在 $t = 19560$ 秒、 $r = 1.6$ 厘米在 $t = 8400$ 秒、 $r = 1.7$ 厘米在 $t = 5460$ 秒、 $r = 1.8$ 厘米在 $t = 3420$ 秒、 $r = 1.9$ 厘米在 $t = 1380$ 秒； $r = 2.0$ 厘米整列为空，因为从第一行 $t = 60$ 秒起半径就已经小于 2.0 厘米；而 $r = 0$ 到 $r = 1.2$ 厘米这几列在全表 3047 行上都有值，因为半径在整个烘干过程中始终不小于 1.2 厘米（复跑报告第 10 节记录烘干结束时刻 $R = 1.2000$ 厘米）。

为什么要这样做：输出坐标搞错的后果有多大

论文里举了一个具体例子：以 $t = 6$ 小时、 $r = 1.0$ 厘米为例。当时半径是 1.374 厘米，正确算法是 $u = (1.0/1.374)^2 = 0.5297$ 。如果误以为「物理距离等于初始坐标乘初始半径」，即用 $u = (1.0/2.0)^2 = 0.25$ ，那就把 $t = 0$ 时刻位于半径 1.0 厘米处的那个物质点，当成了现在的 1.0 厘米处，而这个点其实已经跑到了 0.687 厘米处。两种算法给出的含水率分别是 1.0215 与 1.3782，相差百分之三十五。这个对比在报告第 5.6 节里也写了。

24.7 收尾统计与回读校验

文件第 178 行到第 182 行：

```
print("result1: %d 行 x %d 列" % (len(tt), len(RCOLS) + 1))
print("result2: %d 行 x %d 列" % (len(tt2), len(RCOLS) + 1))
print("result3: %d 行, tdry=%.3f s" % (len(tt3), td3))
print("result4: %d 行, tdry=%.3f s" % (len(tt4), td4))
print("已写入:", OUTDIR)
```

跑完以后把每个文件的行数与烘干时刻打出来，方便跟论文里的数字对位。这是一个很小的习惯，但交付时很省事：不用打开 Excel 去数行数。打印的是数据行数、不含表头，表格文件里的实际行数要再加一。

用 `openpyxl` 把这四个文件读回来，得到的结果是：

文件	工作表	总行数	总列数	首末时刻 / 秒
result1.xlsx	温度、水分浓度	1801	22	1 与 1800
result2.xlsx	温度、水分浓度	10801	22	1 与 10800
result3.xlsx	Sheet1	3428	22	60 与 205574.5185452786
result4.xlsx	Sheet1	3048	23	60 与 182778.2814815244

总行数等于表头一行加数据行数，所以四个文件的数据行数分别是 1800、10800、3427、3047：**result1** 每 1 秒一行覆盖 1800 秒、**result2** 每 1 秒一行覆盖 3 小时、**result3** 与 **result4** 每 60 秒一行、最后再加烘干时间那一行。四个文件的表头第一格都是时间、一个反斜杠、到药材中心的距离，第 2 列到第 22 列是 0 到 2.0 厘米共 21 个距离，**result4** 的第 23 列是药材表面。这些数还与复跑报告里打印的表对得上：回读 **result2.xlsx** 的最后一行， $r = 0$ 处温度是 49.9051 摄氏度，与报告第 7 节表 3 中 3.0 小时那一行相同；同一行的水分浓度在 $r = 0$ 与 $r = 2.0$ 厘米处是 1.7673 与 1.0083，与表 4 中 3.0 小时那一行相同；回读 **result3.xlsx** 中 $t = 21600$ 秒那一行， $r = 0$ 到 $r = 2.0$ 厘米五个位置是 1.0150、0.9863、0.9003、0.7548、0.5342，与报告第 9 节表 5 中 6 小时那一行完全相同；回读 **result4.xlsx** 中 6 小时那一行，表面值是 0.4228，与报告第 10 节表 6 相同。

这一节要记住的 `make_results.py` 做四件事：按题目给的距离网格构造表头，用当前半径把物理距离换算成 u 再做插值，把位置不存在的地方留空，把数值四舍五入到 4 位小数写进 `xlsx`；四个文件的位置、工作表名与列名都按题目附录 3 编排。

25 插图体系：fig_core.py 与 make_figures.py

25.1 为什么要分成两个文件

fig_core.py 管三件事：设定中文字体与字号、提供唯一的出图入口、缓存四个问题的解。make_figures.py 只管一张一张画。

这样分工的好处是样式只写一遍。文件第 63 行到第 73 行就是那个唯一的入口：

```
def save(fig, name):
    """把画布 fig 存成 figure/<name>, 关闭画布并打印一行日志；返回写出的绝对路径。

    所有插图都收敛到这一个入口，是为了把分辨率和紧裁剪钉死在一处——逐图手写
    savefig 很容易漏掉 dpi。注意本函数会 close(fig)，调用方不得再复用该画布。
    """
    p = os.path.join(FIGDIR, name)
    fig.savefig(p, dpi=DPI)
    plt.close(fig)
    print(" [fig] %s" % name)
    return p
```

所有插图都必须经过这个函数。如果让每张图自己写 `savefig`，那么二十几张图里漏写分辨率参数是迟早的事，结果是同一篇论文里插图的像素密度不一致。注意这个函数会 `close(fig)`，调用方不能再复用那块画布。

第 27 行到第 32 行算好目录并在导入时就建好：

```
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, ".."))
FIGDIR = os.path.join(ROOT, "figure")
DATADIR = os.path.join(ROOT, "中间数据")
os.makedirs(FIGDIR, exist_ok=True)
os.makedirs(DATADIR, exist_ok=True)
```

FIGDIR 是出图目录、DATADIR 是中间数据目录，两个都相对交付包的根目录算出来，所以整个包搬走也能正确输出。

25.2 中文字体与出图分辨率

文件第 38 行到第 54 行是字体与全局样式：

```

for _fp in [r"C:\Windows\Fonts\msyh.ttc", r"C:\Windows\Fonts\simhei.ttf"]:
    if os.path.exists(_fp):
        fm.fontManager.addfont(_fp)
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu
    Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["font.size"] = 10.5
plt.rcParams["axes.titlesize"] = 11
plt.rcParams["axes.labelsize"] = 10.5
plt.rcParams["legend.fontsize"] = 9
plt.rcParams["xtick.labelsize"] = 9.5
plt.rcParams["ytick.labelsize"] = 9.5
plt.rcParams["figure.dpi"] = 200
plt.rcParams["savefig.dpi"] = 240
plt.rcParams["savefig.bbox"] = "tight"
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
plt.rcParams["grid.linewidth"] = 0.6

```

第 38 行到第 40 行两件事，都是踩过坑才知道的。

第一，中文字体必须手动注册。第 38 行给出字体文件的路径，前面那个 `r` 表示原始字符串、里面的反斜杠不当作转义字符，这样 Windows 路径可以直接写；`msyh.ttc` 是微软雅黑、`simhei.ttf` 是黑体。第 40 行把存在的字体文件注册到 matplotlib 的字体管理器里（`fm` 是第 25 行导入的模块）；不注册的话，即使系统里装了字体，matplotlib 自带的字体表里也没有中文字体，所有中文标签会渲染成一片方框。这里写成「两个候选、存在才注册」，是为了在只装了其中一个字体的机器上也能跑。

第 41 行把无衬线字体的首选设为微软雅黑，后面两个是备选；`plt.rcParams` 是全局配置字典、改一次对后面所有图都生效，`font.sans-serif` 是首选字体列表、按顺序找第一个能用的。

第二，负号也要单独处理。第 42 行把负号改用 ASCII 的连字符，因为中文字体里缺少 U+2212（真正的数学减号）这个字形，不改的话坐标轴上的负刻度会显示成方框。

第 43 行到第 48 行是字号：正文 10.5 磅、标题 11 磅、图例 9 磅、刻度 9.5 磅。第 49 行到第 51 行是分辨率与紧裁剪：`figure.dpi` 是屏幕上预览用的、`savefig.dpi` 是存盘用的、`savefig.bbox="tight"` 让存盘时自动裁掉四周空白。第 52 行到第 54 行把网格打开并统一网格的深浅与线宽。

文件第 60 行给出出图分辨率：

```
DPI = 240
```

这个数是算出来的，不是随手定的。文件第 56 行到第 59 行的注释里给了完整的算术：插图按版心宽度排版后会被缩到 6.30 英寸，缩放系数约 0.76；按 200 dpi 渲染，印到纸上的有效密度只剩约 260 ppi，240 dpi 才有 310 ppi 以上。这是一个「从成品倒推参数」的例子：先确定成品要多清楚，再反推中间该用什么参数。注意 dpi 只决定像素密度，不改变图内字号的磅值，所以提高 dpi 不会动到任何一张图的版面。

文件第 22 行和第 23 行把绘图后端切成 Agg：

```
import matplotlib
matplotlib.use("Agg")
```

后端是 matplotlib 里负责把图形真正画出来的那一层，不同的后端对应不同的输出方式：有的把图送到屏幕上弹出窗口，有的只把图画到内存缓冲区里再存成文件；Agg 是后一种，只生成像素图、不打开窗口。第 23 行必须写在 `matplotlib.pyplot` 被导入之前（本文件里 `pyplot` 是第 24 行导入的），因为 `pyplot` 一被导入就会选择并初始化后端，之后再换可能不生效或者报错。

指定 Agg 有两个原因：第一，这些脚本只做保存图片这一件事、从来不调用显示窗口的函数，用交互后端是多余的开销；第二，交互后端依赖图形界面，在没有桌面的机器上、在批处理脚本里、在远程终端启动 Python 时都可能报错或者卡住，Agg 不依赖图形界面。

25.3 解缓存与源码指纹

`make_figures.py` 不出图前先算解，而是读缓存。缓存文件是中间数据 `/solutions.npz`，键名与内容如下：

键	内容
<code>p1_t, p1_y</code>	问题一：0 ~ 1800 s，报告点间隔 10 s
<code>p2_t, p2_y</code>	问题二：0 ~ 10800 s，报告点间隔 60 s
<code>p3_t, p3_y, p3_tdry</code>	问题三：0 ~ t_{dry} 均匀 1400 点，外加烘干时刻本身
<code>p4_t, p4_y, p4_tdry</code>	问题四：含收缩，结构同问题三
<code>version</code>	缓存版本号字符串
<code>fingerprint</code>	求解相关源文件的 SHA-256 前 16 位

文件第 79 行是版本号：

```
CACHE_VERSION = "2026-09-13.N64.rtol1e-11"
```

注释里写明了纪律：任何改变解的内容的改动（物理模型、参数、离散、容差）都必须同时递增此号，否则旧的 `solutions.npz` 会被静默沿用，图上画的是旧解。

除了版本号，还有一个源码指纹，也就是文件第 84 行到第 100 行：

```
def _code_fingerprint():
    """对求解相关源文件（common.py、spectral.py、fig_core.py）整体取 SHA-256，
    返回前 16 位十六进制字符串。

    把它随解一起落盘，下次启动时对比即可发现"改了代码但缓存没重算"；只靠版本号
    字符串要靠人记得手工递增，漏改一次就会画出旧解。
    """
    h = hashlib.sha256()
    for name in ("common.py", "spectral.py", "fig_core.py"):
        p = os.path.join(HERE, name)
        if os.path.exists(p):
            with open(p, "rb") as fh:
                h.update(fh.read())
    return h.hexdigest()[:16]

CACHE_FINGERPRINT = _code_fingerprint()
```

它把三个求解相关文件的全部内容读进来算一个哈希值，随解一起写进缓存文件。下次启动时对比哈希值，只要改动了物理模型、参数或离散方式，缓存就会自动失效重算。

为什么要这样做：为什么版本号之外还要指纹

因为版本号是字符串，靠人记得手工递增。改代码时忘了改版本号，旧的解缓存就会被静默沿用，图上画的是旧解，而正文引用的是新数——这种错误极难发现。指纹是自动算的，改一个字符都会变。它让「改了代码却忘了改版本号」这件事在下次启动时立刻暴露出来。

文件第 103 行到第 140 行是缓存的判定逻辑：

```
def build_cache(force=False):
    ...
    cache = os.path.join(DATADIR, "solutions.npz")
    if os.environ.get("A_FORCE_CACHE") == "1":
        force = True
    if os.path.exists(cache) and not force:
        try:
            z = np.load(cache)
```

```

        ver = str(z["version"]) if "version" in z else "<无版本号>"
        fp = str(z["fingerprint"]) if "fingerprint" in z else "<无指纹>"
    """
    except Exception as exc:
        # 缓存损坏：重算而不是崩溃

        print("[cache] 缓存无法读取 (%s)，改为重算" % exc)
    else:
        if ver == CACHE_VERSION and fp == CACHE_FINGERPRINT:
            print("[cache] 命中: %s (版本 %s)" % (cache, ver))
            return

```

三种情形分开处理：版本号与指纹都对，直接用缓存返回；读不出来（文件损坏），打印原因并重算，而不是让整个出图流程崩掉；读得出来但对不上，把「缓存版本对当前版本」「缓存指纹对当前指纹」两行都打出来，再重算。环境变量 `A_FORCE_CACHE=1` 可以强制重算。第 141 行往后是真正求解四个问题的部分：问题一与问题二按固定间隔出点，问题三与问题四先由事件求出 t_{dry} ，再按 1400 个均匀点回放，这样画出来的干燥曲线是光滑的。

为什么要这样做：为什么要缓存而不是每次现算

因为四问解加起来要几分钟，而画一张图只改动版面、不改数据。把解与它的「身份证明」（版本号加指纹）一起落盘，既省时间，又不至于让人画出一张基于旧解的图。注意缓存里连 `p3_tdry` 都存了，因为好几张图的标注要用到烘干时刻，如果每次重算这个时刻，图上标的数就可能与别处对不上。

25.4 画图前的强制对账

文件第 36 行到第 38 行是导入：

```

from fig_core import (save, load_cache, build_cache, FIGDIR, DATADIR, DPI)
import fig_core as FC
import verify_data as VD

```

`make_figures.py` 会 import `verify_data`，但只用它的读取与校验函数。文件第 927 行和第 928 行是画第 24、25 张收敛性图之前的对账：

```

vd = VD.load()
VD.check_consistency(vd, tdry_cached=td3)

```

先拿验证数据文件里的烘干时刻与刚算出的解对账，不一致就直接报错、不画图。

`VD.load()` 里还有一道版本校验，也就是 `verify_data.py` 第 164 行到第 168 行：

```

if data.get("version") != DATA_VERSION:
    raise RuntimeError(
        "验证数据文件版本不符（文件 %r / 期望 %r）：%s\n"
        " 请重新生成： python verify_data.py --recompute"
        % (data.get("version"), DATA_VERSION, DATA_PATH))

```

报错信息里直接给出重新生成的命令。这样即使过了几个月，看到报错的人也知道该做什么。同理，数据文件根本不存在时 `load()` 也会抛错并给出那条命令——不画图也比画一张来源不明的图好。

25.5 25 张图画的是什么

`make_figures.py` 的 `main()` 函数按论文出现顺序分成七组。每一段代码上方的注释都写明了：这幅图画的是哪个物理量、横纵轴是什么、数据取自哪个键、各子图分别是什么。

组	图	内容
一	1 ~ 4	附件 1 的环境序列与一阶惯性拟合、拟合残差诊断、附件 2 的半径历史
二	5 ~ 6	无量纲分析、坐标变换的节点分布示意
三	7 ~ 11	问题一的剖面与云图、表面通量核对、Bessel 系数衰减、谱收敛验证
四	12 ~ 14	问题二的剖面与云图、表面传质通量的降速特征
五	15 ~ 17	问题三的干燥曲线、终点放大、速率衰减拟合
六	18 ~ 20	问题四的剖面与收缩、受控对照的效应分解
七	21 ~ 25	三族互证、收敛性、积分器稳定性、残差统计、技术路线图

其中有一张图是刻意不用的。`fig20_p4_vs_p3.png` 把问题三与问题四的烘干时刻直接相减，但这两个问题同时改变了物性关系与几何，差值 -11.1% 是二者共同作用的净结果，把它当作「收缩效应」属于错误归因。论文改用同一物性下只改半径的受控对照，所以这张备用图不进入正文。代码里保留了它，注释也写明了原因。

受控对照图不在这个文件里。第六组的 `fig20_p4_controlled.png` 由下一个脚本 `make_controlled_comparison.py` 单独生成。另外这个文件在末尾还会多画一张 `fig26_model_schematic.png`（物理模型与坐标系示意图，无数据、纯示意），它排在

第七组之后、不参与上面的分组；这张图用到的 0.60 是附件 2 实测的收缩终值比例（半径由 2.00 厘米降到 1.20 厘米），不是拟合或外推出来的。所以 `figure/` 目录里一共 27 个 PNG：这个脚本画的 26 张、受控对照的 1 张，其中未被正文引用的只有备用图 `fig20_p4_vs_p3.png`。

全篇共用一套颜色语义。文件第 43 行到第 53 行定义了一张语义色表：同一种颜色在任何一页上永远指同一种东西，温度是暖红、水分浓度是冷蓝、判据阈值线是深靛点线、阶段分界与环境设定值是赭黄虚线、实测散点是中灰。这份表只作用于 `make_figures.py`；`make_controlled_comparison.py` 独立运行、不 `import` 本模块，那里另有一份同值的副本，改动时必须同步，否则同一含义在两个脚本出的图里会是两种颜色。

第 56 行到第 65 行的 `_glow` 函数是这套样式的具体手法：先铺一层同色、宽而淡的底线，再压一条实线，返回实线的句柄。目的是让主结论曲线在一堆陪衬线里第一眼就跳出来，而不必把线加粗到失真；底层用透明度而不是预混浅色，是为了在深浅两种底色上都保持同一个色相。要注意两层线都进图例时会出现重复条目，所以主线的标签只挂在实线上。

25.6 采样细节：域外的点必须留空

文件第 83 行到第 107 行，函数 `_samples`：

```
def _samples(s, sol, t_idx=None, rcols=RCOLS):
    """把解按  $u=(r/R)^2$  插值到 rcols 给出的物理半径上，返回 (C, T) 两个 (nt, nr) 数组。

    C 为干基含水率 [kg/kg]、T 为温度 [degC]；nt 为取样时刻数、nr = len(rcols)。

    t_idx 给出要取的时刻下标（默认全部）；s 需提供 R_of、interp_u 与 Np 属性，形状取自传入的 sol (sol.y 的第 1 维是时刻、第 2 维是状态)。

    问题 4 的半径随收缩变小，固定的物理距离 r_j 可能在某个时刻已落到药材之外 ( $r_j > R(t_i)$ )。该位置此时不存在——按论文 S7.4 与 make_results.py 的约定填 NaN，而不能把  $u_t=(r_j/R(t))^2 > 1$  送去插值：那是求解域之外的做外推，浮点下还会因重心插值分母抵消到 0 而返回 inf。

    """
    idx = range(sol.y.shape[1]) if t_idx is None else t_idx
    idx = list(idx)
    C = np.full((len(idx), len(rcols)), np.nan)
```

```

T = np.full((len(idx), len(rcols)), np.nan)
for k, i in enumerate(idx):
    Rt = s.R_of(sol.t[i])
    inside = rcols <= Rt + 1e-12
    if not np.any(inside):
        continue
    Cn, Tn = s.interp_u(sol.y[:, i], (rcols[inside] / Rt) ** 2)
    C[k, inside] = Cn
    T[k, inside] = Tn
return C, T

```

`inside` 是一个布尔数组，标记哪些目标半径在当时的药材之内。只有这些位置送去插值，其余保持 `nan`：`C` 和 `T` 在第 97 行和第 98 行就全部填成了 `nan`，第 105 行和第 106 行用的是布尔下标赋值，只覆盖 `inside` 为真的那些列，其余列不动。

注释里写明了两个不能外推的理由：一是物理上那个位置不存在；二是浮点下重心插值的分母会抵消到零、返回无穷大。这与 `make_results.py` 里 `result4` 的处理是同一个约定，两处保持一致，图和表才不会打架。

第 100 行取该时刻的半径用的是 `s.R_of(sol.t[i])`，第 104 行的换算与式 (265) 完全相同。第 102 行那个「一个都不在里面」的提前跳出只是为了省时间，因为 `np.any` 为假时后面的插值什么也不会改。

25.7 求交而不是取点

文件第 68 行到第 80 行，函数 `_cross_time`：

```

def _cross_time(t, v, crit):
    """求 v 由大变小、首次跌破 crit 的时刻 [s]，返回 float；始终大于 crit 则返回 nan。

    用相邻两点线性求交，而不是直接取第一个低于阈值的采样点：采样间隔在问题 3 里是百秒量级，直接取点会把达标时刻系统性地推迟若干步，图上的标注与正文就对不上了。
    """
    idx = np.where(np.asarray(v) <= crit)[0]
    if idx.size == 0:
        return np.nan
    k = int(idx[0])
    if k == 0:
        return float(t[0])

```

```
return float(t[k - 1] + (v[k - 1] - crit) / (v[k - 1] - v[k]) * (t[k] - t[k - 1]))
```

这就是 `fv_theta.py` 里用过的线性求交，公式与式 (223) 相同，只是搬到了绘图脚本里。图上要标注的「截面平均达标」这类数字，如果直接取第一个低于阈值的采样点，标注出来的时刻会与正文对不上。第 74 行到第 76 行处理退化情形：一直没跌破就返回 `nan`；第一个点就已经低于阈值，说明阈值在起点之前就被穿过了，返回第一个时刻。

这一节要记住的 插图体系的两个原则：样式与数据来源各只有一个入口（`save()` 与解缓存），图上的每一个数字都由代码现场求出、不写死。

26 受控对照：make_controlled_comparison.py

26.1 这个脚本要解决什么问题

从问题三到问题四，物性关系与几何同时改变了：物性从附录 3 换成附录 4，半径从固定变成实测收缩。两者直接相减只能得到「净差异」，无法归因到任何单独一项。

这个脚本补算一个受控算例：物性仍取附录 4，但半径固定不收缩。这样一来就有了三个算例：

算例	物性关系	半径	$t_{\text{dry}} / \text{s}$
A	附录 3	固定 R_0	205575.5
B	附录 4	固定 R_0	464120.2
C	附录 4	实测收缩	182778.3

于是总差异被拆成两段纯效应：A 到 B 只换物性、半径都固定，是纯物性效应 +125.8%；B 到 C 只加收缩、物性都用附录 4，是纯收缩效应 -60.6%；A 到 C 是两问的净差异 -11.1%。两者方向相反，这正是不能直接相减的原因。

这两个百分数就写在文件里，也就是第 112 行：

```
effects = np.array([125.8, -60.6, -11.1])
```

26.2 受控算例是怎么算的

文件第 38 行到第 64 行：

```

def fixed_radius_time(n=64):
    """补算"固定半径 + 附录 4 物性"这个受控算例，返回烘干时刻 t_dry [s]。

    只把 HerbSolver 的 shrink 置 False（半径恒为 R_0 = 2 cm），物性仍取 prob
    =4 的附录 4
    关系，于是它与问题 3 的差别就只剩物性一项。事件定义、容差与积分器（BDF）都与
    其余
    脚本一致，故算出的差值不是求解设置造成的。
    """

    solver = HerbSolver(n, prob=4, shrink=False)
    # atol 按状态分块：前 Np 个是水分浓度 C [kg/kg]（量级 1），后 Np 个是温度 T
    [degC]
    # （量级 3e2）。两块量级不同，同一个绝对容差对两者并不等价。
    atol = np.concatenate(
        [np.full(solver.Np, 1.0e-13), np.full(solver.Np, 1.0e-11)]
    )

    def event(t, y):
        """事件函数：y 前半段是水分浓度场 [kg/kg]，取其最大值（干燥最慢的中心）
        减判据。"""
        return np.max(y[: solver.Np]) - CRIT

    event.terminal = True          # 命中阈值立即停止积分，不必白算到 600000 s
    event.direction = -1          # 只认由大变小穿越，避免初值恰好等于阈值时误判
    # 600000 s（约 167 h）是足够宽的上界，事件必定在此之前触发
    sol = solver.solve(
        600000.0, rtol=1.0e-10, atol=atol, events=event, method="BDF"
    )
    if not sol.t_events[0].size:
        raise RuntimeError("Drying threshold was not reached.")
    return float(sol.t_events[0][0])

```

只改了一个开关：把 shrink 置为 False。物性仍是 prob=4，容差与积分器都与其
余脚本一致。

注释里专门写明了这一点：「事件定义、容差与积分器（BDF）都与其余脚本一致，
故算出的差值不是求解设置造成的」。这就是受控实验的全部要点——只改一个变量。

第 45 行的 HerbSolver(n, prob=4, shrink=False) 与问题四只差一个 shrink；

第 48 行到第 50 行的绝对容差按状态分块给，与第三部分讲过的理由相同（水分量级 1、温度量级 300）。第 52 行到第 54 行的事件函数与 `make_results.py` 里的 `ev` 完全同形。第 56 行和第 57 行让积分在命中阈值时立刻停止，第 59 行到第 61 行求解，积分上限放宽到 600000 秒，因为这个算例比问题三慢得多（表里的 B 行是 464120 秒）；第 62 行和第 63 行是一个必要的检查：万一事件没触发，宁可抛出错误，也不要返回一个空数组里的元素。

为什么要这样做：为什么必须补算这一个算例，而不是直接相减

因为直接相减得到的是一个复合的差异，它把「物性变差让干燥变慢」与「半径变小让路径变短、干燥变快」两种相反的效应混在一起。补算一个只改物性的算例，就把这两件事分开了：A 到 B 只有物性在变，B 到 C 只有几何在变。这也是论文里把那 11.1% 的净差异拆成 +125.8% 与 -60.6% 两段的依据。

26.3 两档网格的自查

文件第 155 行到第 169 行：

```
def main():
    """跑两档网格的固定半径算例、画出受控对照图，并把三个数值打到屏幕上。

    两档网格只用来检查空间离散够不够细：rel_gap 应远小于要归因的那几个百分点，
    出图用 N=64 的结果。
    """
    td48 = fixed_radius_time(48)
    td64 = fixed_radius_time(64)
    rel_gap = abs(td64 - td48) / td64
    out = Path(__file__).resolve().parents[1] / "figure" / "
fig20_p4_controlled.png"
    draw(out, td64)
    print(f"N=48 fixed-R Appendix-4: {td48:.3f} s")
    print(f"N=64 fixed-R Appendix-4: {td64:.3f} s")
    print(f"relative grid gap: {rel_gap:.3e}")
    print(f"figure: {out}")
```

跑两档网格，算出相对差 `rel_gap`，用来说明空间离散够不够细：相对差应当远小于要归因的那几个百分点。出图用 $N = 64$ 的结果，与其余脚本的生产网格一致。

第 164 行用 `Path(__file__).resolve().parents[1]` 从本文件往上退一级得到交付包根目录，再拼上 `figure` 与文件名，写出的是受控对照图 `fig20_p4_controlled.png`；

这个脚本不经过 `fig_core.save()`，所以它自己在 `draw()` 里设好了 240 dpi 与中文字体。

有一处可以说明的地方。三个算例里，A 与 C 两行的时长是写成模块常量的，也就是文件第 34 行和第 35 行：

```
TD_PROBLEM3 = 205575.5      # s    问题 3: 附录 3 物性、半径固定不收缩
TD_PROBLEM4_SHRINK = 182778.3  # s    问题 4: 附录 4 物性、含实测收缩 R(t)
```

只有中间的 B 行是现场算的。这意味着如果以后改了模型参数，重跑这个脚本时 B 行会变，而 A、C 两行不会变，图上就会出现「三段效应不自洽」。更稳妥的做法是把 A、C 也改成从解缓存（中间数据/`solutions.npz` 里的 `p3_tdry` 与 `p4_tdry`）里读，和 B 行一样来自现场计算，三者才在同一口径上相减。这一点如实记在这里，是因为图上那三根柱子的高度直接取决于这三个数。

这一节要记住的 受控对照的价值在于「只改一个变量」：同一个求解器、同一套容差、同一个积分器，只把半径开关关掉，于是算出来的差值只能归因到物性这一项。

27 整条产出链

把这一部分与第三部分合起来看，从零到交付的完整顺序如下。下面的命令都在 `code/` 目录下用 `python` 运行：

步	命令	作用
一	<code>verify_data.py --recompute</code>	四类扫描, 生成 <code>verification.json</code> (约 22 分钟)
二	<code>run_all.py</code>	全部结果与验证, 写出复跑报告 (文件头注释记实测约 6.5 分钟)
三	<code>make_results.py</code>	生成 <code>result1</code> 到 <code>result4.xlsx</code>
四	<code>make_controlled_comparison.py</code>	受控算例与效应分解图
五	<code>make_figures.py</code>	生成其余 25 张插图

第一步，运行 `python verify_data.py --recompute`。这一步做四类扫描：谱解网格扫描、定步长格式的时间步扫描、经典有限体积的空间扫描、积分器与容差的组合，把结果写进中间数据/`verification.json`。它最耗时，但它产出的数据文件是第五步画图的前提：如果跳过它，`make_figures.py` 在画收敛性与积分器稳定性那两张图时会直

接报错并提示重新生成，而不是画出一张没有数据来源的图。不带 `--recompute` 参数时它只做校验，是秒级的，`run_all.py` 与 `make_figures.py` 内部就是这么调它的。

第二步，运行 `python run_all.py`。这一步做第四部分讲的全部验证：烘房环境标定、与闭式解比较、网格加密、两种对照格式互证、时间步长敏感性，分 12 节写进复跑报告 `.txt`。它不产生任何 `xlsx` 文件与图片；报告用的是相对路径，所以写在运行命令所在的目录里，在 `code/` 目录下运行就落在 `code/` 目录里。报告里还有一行对账结果：验证数据文件里 $N = 64$ 项的烘干时刻必须与本次算出的生产值相符，不符就把原因打印出来，让人一眼看出是数据过期还是这次算错了。

第三步，运行 `python make_results.py`。这一步重新求解问题 1 到问题 4 并写出四个结果文件，落在 `< 包根 >/result/` 目录下，同时在屏幕上打印四个文件的行数与两个烘干时间。这一步是最费时的，因为问题 3 要积到 205574 秒、问题 4 要积到 182778 秒。

第四步，运行 `python make_controlled_comparison.py`。这一步跑两档网格的固定半径算例，画出受控对照图 `figure/fig20_p4_controlled.png`，并把 $N = 48$ 、 $N = 64$ 两个烘干时刻与相对网格差打在屏幕上。它不依赖前面的产出，自己调用求解器。

第五步，运行 `python make_figures.py`。这一步先调 `build_cache()` 取得四问解（缓存命中就直接用，否则重算并写中间数据 `/solutions.npz`），再按七组顺序逐张画图，落在 `< 包根 >/figure/` 目录下，全部 240 dpi。画收敛性与积分器稳定性那两张图之前，它会先用 `verify_data` 做一次强制对账。

五步之间的依赖关系有三条。

第一条是顺序上的：验证必须放在出表出图之前，只有验证通过，后面算出的表和图才有意义；如果先出表再验证，一旦程序里有错，表格里的数字是全错的、而错误要到最后一刻才发现。第二步与第三步的顺序可以互换，但第一步必须在第五步之前。

第二条是数据上的：`make_figures.py` 的收敛性数据来自第一步的 `verification.json`，受控对照图的数据由第四步自己现算，`make_results.py` 与 `make_figures.py` 各自独立求解。三条出口（`xlsx`、解缓存、文本报告）共用同一份解的定义，所以论文里的数、结果文件里的数、图上的数只有一个来源。

第三条是路径上的：附件位置由 `common.py` 按自己的文件位置向上搜索，输出目录由各个脚本按自己的文件位置推算，所以整个文件夹搬到别处仍然能跑；需要留意的是 `run_all.py` 写复跑报告 `.txt` 用的是相对路径，文件会落在运行命令所在的那个目录、而不是代码目录。所有脚本都以模块名导入 `common` 与 `spectral`，而 Python 会把脚本所在目录放进模块搜索路径，所以只要按路径运行脚本、导入都能成功。

这一节要记住的 这一部分涉及的六个文件，外加第三部分讲过的 `verify_data.py`，可

以按「它会不会改变结果」分成两组。`fv_theta.py`、`fv_uniform.py`、`make_controlled_comparison`与 `verify_data.py` 是验证组,它们只负责提供旁证;`make_results.py`、`fig_core.py`、`make_figures.py` 是产出组,它们决定最终交出去的表和图长什么样。两组都建立在第三部分那份唯一的解之上。

Part VI

问题一闭式解的完整推导

28 这一部分要解决什么

问题一的温度场可以不离散、不用网格、不用时间步进,直接用一条公式算出来。这条公式叫贝塞尔-杜阿梅尔展开式。第三部分第十三章只用十几行提到它,说公式是从别处推来的,而那个别处并不存在。这一部分把推导从头写全,一步都不跳。

三件事使得问题一可以用公式解:

第一,问题一的密度、比热容、导热系数都是常数,不随水分浓度和温度变化。这一点由附录 2 给出。

第二,水分扩散系数只依赖水分浓度,不依赖温度。于是温度方程里没有出现水分浓度,温度方程自成一体,可以先单独解出温度,再解水分。两个场是解耦的。

第三,附件 1 的烘房温度可以写成一条指数曲线

$$T_{\text{air}}(t) = T_{\text{set}} - Ae^{-t/\tau_T}, \quad A = T_{\text{set}} - 28. \quad (266)$$

指数形式的驱动项对应的响应也是指数形式,这就给了解析求解的机会。

这一节要记住的 问题一的温度场有闭式解,原因是常物性、场解耦、环境是指数形式。推导分五步:齐次化、准静态分解、解常微分方程、解特征值问题、定展开系数。

29 第一步:把边界条件化成齐次

先把问题一的方程与条件抄全。方程是

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rk \frac{\partial T}{\partial r} \right), \quad 0 < r < R_0, \quad (267)$$

边界条件是

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad -k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h (T(R_0, t) - T_{\text{air}}(t)), \quad (268)$$

初始条件是

$$T(r, 0) = 28. \quad (269)$$

所谓齐次边界条件，是指边界条件里不含已知的外界函数，右端只出现未知函数本身。式 (268) 右边有 $T_{\text{air}}(t)$ ，所以是非齐次的。非齐次边界条件不能直接做分离变量，必须先把外界函数挪走。

第 1 步 定义新的未知函数 w

令

$$w(r, t) = T(r, t) - T_{\text{air}}(t). \quad (270)$$

它的物理含义是药材温度比烘房温度低多少。

第 2 步 把新函数代进方程

先算时间导数。因为 $T = w + T_{\text{air}}$ ，而 T_{air} 只依赖 t ，不依赖 r ，所以

$$\frac{\partial T}{\partial t} = \frac{\partial w}{\partial t} + \frac{dT_{\text{air}}}{dt}. \quad (271)$$

由式 (266) 求导，第一步先写出指数函数的导数公式 $\frac{d}{dt}e^{-t/\tau_T} = -\frac{1}{\tau_T}e^{-t/\tau_T}$ ，于是

$$\frac{dT_{\text{air}}}{dt} = -(T_{\text{set}} - 28) \left(-\frac{1}{\tau_T} \right) e^{-t/\tau_T} = \frac{A}{\tau_T} e^{-t/\tau_T}. \quad (272)$$

这里用了 $A = T_{\text{set}} - 28$ 。两个负号相乘得正号，所以烘房温度是上升的，变化率为正。

再算空间导数。 T_{air} 与 r 无关，所以

$$\frac{\partial T}{\partial r} = \frac{\partial w}{\partial r}, \quad \frac{\partial^2 T}{\partial r^2} = \frac{\partial^2 w}{\partial r^2}. \quad (273)$$

把这两条代入式 (267)，左边变成 $\rho c_p \left(\frac{\partial w}{\partial t} + \frac{A}{\tau_T} e^{-t/\tau_T} \right)$ ，右边变成 $\frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial w}{\partial r} \right)$ 。
两边同除以 ρc_p ，并利用 $\alpha = k/(\rho c_p)$ ，得到

$$\frac{\partial w}{\partial t} + \frac{A}{\tau_T} e^{-t/\tau_T} = \alpha \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right). \quad (274)$$

把驱动项移到右边：

$$\boxed{\frac{\partial w}{\partial t} = \alpha \nabla^2 w - \frac{A}{\tau_T} e^{-t/\tau_T}} \quad (275)$$

其中 ∇^2 是圆柱坐标下的径向拉普拉斯算子，它的展开式就是 $\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial}{\partial r} \right)$ 。

第 3 步 把边界条件与初始条件也换掉

表面条件里 T 换成 $w + T_{\text{air}}$ ：

$$-k \frac{\partial w}{\partial r} \Big|_{r=R_0} = h (w(R_0, t) + T_{\text{air}}(t) - T_{\text{air}}(t)) = h w(R_0, t). \quad (276)$$

T_{air} 一加一减消掉了，剩下的条件是

$$-k \frac{\partial w}{\partial r} \Big|_{r=R_0} = h w(R_0, t). \quad (277)$$

这是一个齐次条件，右端只有未知函数。这一步就是齐次化的全部内容。

中心条件不变，因为 T_{air} 与 r 无关：

$$\frac{\partial w}{\partial r} \Big|_{r=0} = 0. \quad (278)$$

初始条件：

$$w(r, 0) = T(r, 0) - T_{\text{air}}(0) = 28 - (T_{\text{set}} - A). \quad (279)$$

把 $A = T_{\text{set}} - 28$ 代进去：

$$w(r, 0) = 28 - T_{\text{set}} + T_{\text{set}} - 28 = 0. \quad (280)$$

也就是说， w 的初始条件是零。这一点后面要用。

这一节要记住的 换成 $w = T - T_{\text{air}}$ 以后，边界条件变成齐次的，初始条件变成零，代价是方程右边多了一个已知的驱动项。这个代价是划算的，因为齐次条件才能做分离变量。

30 第二步：把解拆成两部分

方程 (275) 右边的驱动项是 $-\frac{A}{\tau_T}e^{-t/\tau_T}$ ，它随时间按指数衰减。被指数驱动的系统，其响应通常也是同一频率的指数。所以尝试把解写成

$$w(r, t) = W(r)e^{-t/\tau_T} + \hat{w}(r, t). \quad (281)$$

第一项表示跟着外界一起走的准静态部分，第二项表示从零初值出发的瞬态衰减部分。

第 4 步 把拆分代进方程，逐项对齐

先算左边的时间导数：

$$\frac{\partial w}{\partial t} = W(r) \left(-\frac{1}{\tau_T} \right) e^{-t/\tau_T} + \frac{\partial \hat{w}}{\partial t} = -\frac{W}{\tau_T} e^{-t/\tau_T} + \frac{\partial \hat{w}}{\partial t}. \quad (282)$$

再算右边的拉普拉斯项。因为 ∇^2 只对 r 求导，而 e^{-t/τ_T} 与 r 无关，可以提出来：

$$\alpha \nabla^2 w = \alpha \nabla^2 W e^{-t/\tau_T} + \alpha \nabla^2 \hat{w}. \quad (283)$$

把两边的结果都代进式 (275)：

$$-\frac{W}{\tau_T} e^{-t/\tau_T} + \frac{\partial \hat{w}}{\partial t} = \alpha \nabla^2 W e^{-t/\tau_T} + \alpha \nabla^2 \hat{w} - \frac{A}{\tau_T} e^{-t/\tau_T}. \quad (284)$$

把含 $\hat{w}$ 的项留在左边，其余的移到右边：

$$\frac{\partial \hat{w}}{\partial t} = \alpha \nabla^2 \hat{w} + e^{-t/\tau_T} \left[\alpha \nabla^2 W + \frac{W}{\tau_T} - \frac{A}{\tau_T} \right]. \quad (285)$$

第 5 步 令中括号为零

现在提出一个要求：让方括号里的表达式恒等于零。

$$\alpha \nabla^2 W + \frac{W}{\tau_T} - \frac{A}{\tau_T} = 0. \quad (286)$$

这样一来，式 (285) 就退化为

$$\frac{\partial \hat{w}}{\partial t} = \alpha \nabla^2 \hat{w}. \quad (287)$$

这是一个齐次热传导方程，右边没有驱动项。

为什么可以这样要求。因为 e^{-t/τ_T} 是 t 的函数，如果方括号里的表达式不为零，那么随着 t 变化，这个表达式会被一个指数因子反复缩放，无法与左边的时间导数在任意时刻都相等。唯一能让等式对所有 t 成立的办法，就是让它恒等于零。这种把含时间的因子与含空间的因子分开、各自独立的论证，是分离变量法的标准做法。

第 6 步 边界条件怎样分配给两部分

表面条件 (277) 对式 (281) 也要成立：

$$-k \frac{\partial}{\partial r} (W e^{-t/\tau_T} + \hat{w}) \Big|_{r=R_0} = h (W(R_0) e^{-t/\tau_T} + \hat{w}(R_0, t)). \quad (288)$$

把含 W 的项与含 $\hat{w}$ 的项分开写：

$$e^{-t/\tau_T} [-kW'(R_0) - hW(R_0)] + [-k\hat{w}'(R_0, t) - h\hat{w}(R_0, t)] = 0. \quad (289)$$

第一项含因子 e^{-t/τ_T} ，第二项不含。两者对任意时刻都要抵消，只能各自为零。于是

$$-kW'(R_0) = hW(R_0), \quad -k \frac{\partial \hat{w}}{\partial r} \Big|_{r=R_0} = h\hat{w}(R_0, t). \quad (290)$$

两部分各自满足同一个齐次表面条件。中心条件同理，两部分都满足 $\partial_r(\cdot)|_{r=0} = 0$ 。

第 7 步 初始条件怎样分配

式 (280) 给出 $w(r, 0) = 0$, 代入式 (281):

$$W(r)e^0 + \hat{w}(r, 0) = 0 \implies \hat{w}(r, 0) = -W(r). \quad (291)$$

W 还没有解出来, 但它的形式已经确定, 所以 $\hat{w}$ 的初始条件是一个已知函数。

31 第三步: 解出准静态部分 W

31.1 把方程化成标准形式

式 (286) 两边同除以 α :

$$\nabla^2 W + \frac{1}{\alpha\tau_T} W = \frac{A}{\alpha\tau_T}. \quad (292)$$

记

$$\kappa^2 = \frac{1}{\alpha\tau_T}, \quad \kappa = \frac{1}{\sqrt{\alpha\tau_T}}. \quad (293)$$

再乘上 κ^2 , 右边变成 $\kappa^2 A$, 于是

$$\nabla^2 W + \kappa^2 W = \kappa^2 A. \quad (294)$$

31.2 把拉普拉斯算子写开

圆柱径向拉普拉斯算子作用在 W 上是

$$\nabla^2 W = \frac{1}{r} \frac{d}{dr} \left(r \frac{dW}{dr} \right) = \frac{d^2 W}{dr^2} + \frac{1}{r} \frac{dW}{dr}. \quad (295)$$

第二步的等号是把乘积的导数展开得到的: $\frac{1}{r} (W' + rW'') = W'' + \frac{W'}{r}$ 。

所以式 (294) 写成

$$\frac{d^2 W}{dr^2} + \frac{1}{r} \frac{dW}{dr} + \kappa^2 W = \kappa^2 A. \quad (296)$$

31.3 认识贝塞尔方程

式 (296) 的左边去掉右端项以后, 形状是

$$\frac{d^2 y}{dx^2} + \frac{1}{x} \frac{dy}{dx} + y = 0. \quad (297)$$

这条方程叫零阶贝塞尔方程, 它的两个线性无关解叫第一类与第二类贝塞尔函数, 记作 $J_0(x)$ 与 $Y_0(x)$ 。

这里不需要知道贝塞尔函数的复杂定义, 只要知道四件事, 后面每一步都用得到:

第一, $J_0(0) = 1$, 且 J_0 在 $x > 0$ 上振荡衰减, 有无穷多个零点。

第二, $Y_0(x)$ 在 $x \rightarrow 0$ 时趋于负无穷, 是发散的。

第三, 导数关系 $\frac{d}{dx} J_0(x) = -J_1(x)$, 其中 J_1 是第一类一阶贝塞尔函数。这条关系在下面代边界条件时要用。

第四, 把自变量换成 κr 时要用链式法则。设 $x = \kappa r$, 则 $\frac{d}{dr} = \kappa \frac{d}{dx}$, 所以 $J_0(\kappa r)$ 对 r 的导数是 $-\kappa J_1(\kappa r)$ 。

31.4 写出通解, 去掉发散的那一项

方程 (296) 是非齐次的, 右端是常数 $\kappa^2 A$ 。常数函数 $W = A$ 代入左边得 $\kappa^2 A$, 正好等于右端, 所以 $W = A$ 是它的一个特解。

非齐次方程的通解等于一个特解加上齐次方程的通解, 于是

$$W(r) = A + B J_0(\kappa r) + D Y_0(\kappa r). \quad (298)$$

现在用 $r = 0$ 处的条件去掉 Y_0 。当 $r \rightarrow 0$ 时, $Y_0(\kappa r) \rightarrow -\infty$ 。但药材中心的温度是有限的, W 在中心必须取有限值。要让它有限, 只能取 $D = 0$ 。于是

$$W(r) = A + B J_0(\kappa r). \quad (299)$$

31.5 用表面条件定出 B

先求导。由上面第三条性质, 以及自变量是 κr :

$$W'(r) = B \cdot (-\kappa J_1(\kappa r)) = -B \kappa J_1(\kappa r). \quad (300)$$

在 $r = R_0$ 处:

$$W'(R_0) = -B\kappa J_1(\kappa R_0), \quad W(R_0) = A + BJ_0(\kappa R_0). \quad (301)$$

代入表面条件 $-kW'(R_0) = hW(R_0)$:

$$-k(-B\kappa J_1(\kappa R_0)) = h(A + BJ_0(\kappa R_0)). \quad (302)$$

左边两个负号相乘得正:

$$kB\kappa J_1(\kappa R_0) = hA + hBJ_0(\kappa R_0). \quad (303)$$

把含 B 的项移到同一边:

$$B(k\kappa J_1(\kappa R_0) - hJ_0(\kappa R_0)) = hA. \quad (304)$$

两边同除以括号里的量, 得到

$$B = \frac{hA}{k\kappa J_1(\kappa R_0) - hJ_0(\kappa R_0)} \quad (305)$$

31.6 代入数值

问题一的常数是 $\rho = 820$ 、 $c_p = 2600$ 、 $k = 0.36$ 、 $h = 25$ 、 $R_0 = 0.02$ 、 $\tau_T = 1877.5$ 、 $T_{\text{set}} = 50.212$ 。逐个算:

$$\alpha = \frac{k}{\rho c_p} = \frac{0.36}{820 \times 2600} = 1.688555 \times 10^{-7} \text{ m}^2/\text{s}, \quad (306)$$

$$\text{Bi} = \frac{hR_0}{k} = \frac{25 \times 0.02}{0.36} = 1.388889, \quad (307)$$

$$\kappa = \frac{1}{\sqrt{\alpha\tau_T}} = \frac{1}{\sqrt{1.688555 \times 10^{-7} \times 1877.5}} = 56.163269 \text{ m}^{-1}, \quad (308)$$

$$\kappa R_0 = 56.163269 \times 0.02 = 1.123265, \quad A = 50.212 - 28 = 22.212000. \quad (309)$$

代进式 (305) 得

$$B = \frac{25 \times 22.212}{0.36 \times 56.163269 \times J_1(1.123265) - 25 \times J_0(1.123265)} = -68.909425. \quad (310)$$

于是准静态部分在中心和表面的值是

$$W(0) = A + BJ_0(0) = 22.212 - 68.909 = -46.697 \text{ }^\circ\text{C}, \quad (311)$$

$$W(R_0) = 22.212 - 68.909 \times J_0(1.123265) = -26.616 \text{ }^\circ\text{C}. \quad (312)$$

两个值都是负数，含义是药材比烘房冷。中心比表面更冷，这符合升温阶段的物理图像。

这一节要记住的 W 的方程是一个常微分方程，解是 $A + BJ_0(\kappa r)$ 。两个待定常数由两条条件定出：中心有界去掉 Y_0 ，表面 Robin 条件定出 B 。

32 第四步：解瞬态部分 $\hat{w}$ 的特征值问题

32.1 分离变量

$\hat{w}$ 满足三个条件：齐次方程 (287)、齐次表面条件、齐次中心条件。设解可以写成

$$\hat{w}(r, t) = \phi(r)G(t), \quad (313)$$

其中 ϕ 只依赖 r ， G 只依赖 t 。代入式 (287)：

$$\phi(r)G'(t) = \alpha G(t)\nabla^2\phi(r). \quad (314)$$

两边同除以 $\alpha\phi(r)G(t)$ ：

$$\frac{G'(t)}{\alpha G(t)} = \frac{\nabla^2\phi(r)}{\phi(r)}. \quad (315)$$

左边只依赖 t ，右边只依赖 r 。一个只依赖 t 的量恒等于一个只依赖 r 的量，唯一的可能是两者都等于同一个常数。把这个常数记作 $-\gamma$ ，负号是为了后面方便：

$$\frac{G'}{\alpha G} = \frac{\nabla^2\phi}{\phi} = -\gamma. \quad (316)$$

32.2 解时间部分

由 $\frac{G'}{\alpha G} = -\gamma$ 得 $G' = -\alpha\gamma G$ 。这是最简单的常微分方程，解是

$$G(t) = e^{-\alpha\gamma t}. \quad (317)$$

只要 $\gamma > 0$, G 随时间衰减, 这就是瞬态部分的衰减。后面会看到 γ 确实是正的。

32.3 解空间部分

由 $\frac{\nabla^2 \phi}{\phi} = -\gamma$ 得

$$\nabla^2 \phi + \gamma \phi = 0. \quad (318)$$

把拉普拉斯算子写开:

$$\frac{d^2 \phi}{dr^2} + \frac{1}{r} \frac{d\phi}{dr} + \gamma \phi = 0. \quad (319)$$

这个方程与前面式 (297) 形状相同, 只是常数从 1 换成了 γ 。为了让常数变成 1, 令

$$\gamma = \frac{\alpha \beta^2}{R_0^2}, \quad \text{也就是} \quad \beta = R_0 \sqrt{\frac{\gamma}{\alpha}}. \quad (320)$$

再令 $x = \frac{\beta r}{R_0}$ 。可以验证, 这样换元以后方程变成零阶贝塞尔方程, 于是解是

$$\phi(r) = J_0\left(\frac{\beta r}{R_0}\right). \quad (321)$$

第二类贝塞尔函数 Y_0 又一次因为中心发散被去掉。

为什么可以验证。把 $x = \frac{\beta r}{R_0}$ 代进去, 先用链式法则求两种导数的换算:

$$\frac{d}{dr} = \frac{\beta}{R_0} \frac{d}{dx}, \quad \frac{d^2}{dr^2} = \frac{\beta^2}{R_0^2} \frac{d^2}{dx^2}. \quad (322)$$

更直接的办法是把 $\phi(r) = J_0(\beta r/R_0)$ 代回方程左边逐项计算。已知 $y(x) = J_0(x)$ 满足 $y'' + \frac{1}{x}y' + y = 0$ 。对 ϕ 用上面的换算式得 $\phi'(r) = \frac{\beta}{R_0}y'(x)$, $\phi''(r) = \frac{\beta^2}{R_0^2}y''(x)$ 。于是

$$\phi'' + \frac{1}{r}\phi' + \gamma\phi = \frac{\beta^2}{R_0^2}y'' + \frac{1}{r}\frac{\beta}{R_0}y' + \gamma y. \quad (323)$$

注意这里的 $\frac{1}{r}$ 也要用新变量表示。由 $x = \frac{\beta r}{R_0}$ 反解出 $r = \frac{R_0 x}{\beta}$, 于是

$$\frac{1}{r} = \frac{\beta}{R_0 x}. \quad (324)$$

代进去：

$$= \frac{\beta^2}{R_0^2} y'' + \frac{\beta}{R_0 x} \cdot \frac{\beta}{R_0} y' + \gamma y = \frac{\beta^2}{R_0^2} \left(y'' + \frac{1}{x} y' \right) + \gamma y. \quad (325)$$

由贝塞尔方程 $y'' + \frac{1}{x} y' = -y$ ，上式

$$= -\frac{\beta^2}{R_0^2} y + \gamma y = \left(\gamma - \frac{\beta^2}{R_0^2} \right) y. \quad (326)$$

要让左边恒等于零，必须 $\gamma = \frac{\beta^2}{R_0^2}$ 。这里 β 是一个纯数，没有量纲，所以 γ 的量纲是长度的负二次方，这与拉普拉斯算子 $\nabla^2 \phi$ 的量纲一致，两边量纲相符。

32.4 用表面条件定出 β

把 $\phi(r) = J_0(\beta r/R_0)$ 代入齐次表面条件 $-k\phi'(R_0) = h\phi(R_0)$ 。

先求导。用链式法则与 $\frac{d}{dx} J_0 = -J_1$ ：

$$\phi'(r) = \frac{\beta}{R_0} \cdot \left(-J_1 \left(\frac{\beta r}{R_0} \right) \right) = -\frac{\beta}{R_0} J_1 \left(\frac{\beta r}{R_0} \right). \quad (327)$$

在 $r = R_0$ 处 $\frac{\beta r}{R_0} = \beta$ ，所以

$$\phi'(R_0) = -\frac{\beta}{R_0} J_1(\beta), \quad \phi(R_0) = J_0(\beta). \quad (328)$$

代入条件：

$$-k \left(-\frac{\beta}{R_0} J_1(\beta) \right) = h J_0(\beta) \implies \frac{k\beta}{R_0} J_1(\beta) = h J_0(\beta). \quad (329)$$

两边同乘以 $\frac{R_0}{k}$ ：

$$\beta J_1(\beta) = \frac{hR_0}{k} J_0(\beta). \quad (330)$$

记 $\text{Bi} = \frac{hR_0}{k}$ ，最终得到

$$\boxed{\beta J_1(\beta) = \text{Bi} J_0(\beta)} \quad (331)$$

这个方程叫特征方程，它的正根 $\beta_1 < \beta_2 < \beta_3 < \dots$ 叫特征值。问题一的 $\text{Bi} = 1.388889$ ，前四个根是

$$\beta_1 = 1.4184725188, \quad \beta_2 = 4.166469601, \quad \beta_3 = 7.2084784644, \quad \beta_4 = 10.3082731535. \quad (332)$$

特征方程没有解析解，只能用数值方法求根。代码的做法在第八节讲。

对应的衰减率由式 (320) 给出：

$$\gamma_n = \frac{\alpha \beta_n^2}{R_0^2}. \quad (333)$$

这一节要记住的 瞬态部分由特征函数 $J_0(\beta_n r/R_0)$ 与衰减因子 $e^{-\gamma_n t}$ 组成。特征值 β_n 由表面条件决定，衰减率 γ_n 由特征值决定。

33 第五步：正交性与展开系数

33.1 把 $\hat{w}$ 写成级数

瞬态部分的初始条件是 $\hat{w}(r, 0) = -W(r)$ ，它是一个已知函数。把已知函数用特征函数展开：

$$\hat{w}(r, t) = \sum_{n \geq 1} b_n J_0\left(\frac{\beta_n r}{R_0}\right) e^{-\gamma_n t}. \quad (334)$$

在 $t = 0$ 时：

$$\sum_{n \geq 1} b_n J_0\left(\frac{\beta_n r}{R_0}\right) = -W(r). \quad (335)$$

要定出 b_n ，需要知道特征函数之间的正交性。

33.2 内积与权

在圆柱问题里，两个函数的内积要带一个权 r ：

$$\langle f, g \rangle = \int_0^{R_0} r f(r) g(r) dr. \quad (336)$$

之所以带权 r ，是因为圆柱的每一圈面积与 r 成正比。第一部分推导控制方程时出现的那个 r 因子，现在又出现了，它决定了正交性成立的形式。

33.3 证明正交性

取两个不同的特征函数 ϕ_n 、 ϕ_m ，对应特征值 β_n 、 β_m 。两者都满足

$$\nabla^2 \phi_n = -\frac{\beta_n^2}{R_0^2} \phi_n, \quad \nabla^2 \phi_m = -\frac{\beta_m^2}{R_0^2} \phi_m. \quad (337)$$

用第一式乘 ϕ_m ，第二式乘 ϕ_n ，相减：

$$\phi_m \nabla^2 \phi_n - \phi_n \nabla^2 \phi_m = -\frac{\beta_n^2}{R_0^2} \phi_n \phi_m + \frac{\beta_m^2}{R_0^2} \phi_n \phi_m = \frac{\beta_m^2 - \beta_n^2}{R_0^2} \phi_n \phi_m. \quad (338)$$

两边乘以 r 并从 0 积到 R_0 ：

$$\int_0^{R_0} r (\phi_m \nabla^2 \phi_n - \phi_n \nabla^2 \phi_m) dr = \frac{\beta_m^2 - \beta_n^2}{R_0^2} \int_0^{R_0} r \phi_n \phi_m dr. \quad (339)$$

现在算左边的积分。圆柱坐标下的格林第二恒等式给出

$$\int_0^{R_0} r (\phi_m \nabla^2 \phi_n - \phi_n \nabla^2 \phi_m) dr = R_0 [\phi_m(R_0) \phi_n'(R_0) - \phi_n(R_0) \phi_m'(R_0)]. \quad (340)$$

这条恒等式就是把 $\frac{1}{r}(rf')'$ 形式的算子做两次分部积分得到的，边界项里出现的 R_0 来自权 r 在 $r = R_0$ 处的取值， $r = 0$ 处的边界项因为前面有因子 r 而自动消失。

现在看式 (340) 的右端。两个特征函数都满足同一个齐次表面条件：

$$-k\phi_n'(R_0) = h\phi_n(R_0), \quad -k\phi_m'(R_0) = h\phi_m(R_0). \quad (341)$$

两式都除以 $-k$ ，得 $\phi_n'(R_0) = -\frac{h}{k}\phi_n(R_0)$ ，对 m 同理。代入右端：

$$\phi_m(R_0) \left(-\frac{h}{k}\phi_n(R_0) \right) - \phi_n(R_0) \left(-\frac{h}{k}\phi_m(R_0) \right) = 0. \quad (342)$$

两项完全一样，一正一负，抵消为零。所以式 (340) 的左端为零，代回式 (339) 得

$$\frac{\beta_m^2 - \beta_n^2}{R_0^2} \langle \phi_n, \phi_m \rangle = 0. \quad (343)$$

当 $m \neq n$ 时 $\beta_m \neq \beta_n$ ，所以 $\beta_m^2 - \beta_n^2 \neq 0$ ，只能是

$$\boxed{\langle \phi_n, \phi_m \rangle = 0, \quad m \neq n} \quad (344)$$

这就是带权 r 的正交性。

33.4 模长

$n = m$ 时的内积叫模长的平方。由贝塞尔函数的积分公式可以算出

$$\langle \phi_n, \phi_n \rangle = \int_0^{R_0} r J_0^2\left(\frac{\beta_n r}{R_0}\right) dr = \frac{R_0^2}{2} [J_0^2(\beta_n) + J_1^2(\beta_n)]. \quad (345)$$

33.5 展开系数

对式 (335) 两边同时与 ϕ_m 做内积：

$$\left\langle \sum_{n \geq 1} b_n \phi_n, \phi_m \right\rangle = \langle -W, \phi_m \rangle. \quad (346)$$

内积是线性的，求和与内积可以交换顺序：

$$\sum_{n \geq 1} b_n \langle \phi_n, \phi_m \rangle = \langle -W, \phi_m \rangle. \quad (347)$$

由正交性 (344)，右边的求和里只有 $n = m$ 那一项不为零：

$$b_m \langle \phi_m, \phi_m \rangle = -\langle W, \phi_m \rangle \implies b_m = -\frac{\langle W, \phi_m \rangle}{\langle \phi_m, \phi_m \rangle}. \quad (348)$$

还差 $\langle W, \phi_m \rangle$ 。这个内积可以用格林恒等式绕过去，不必直接积分。

用与前面相同的方法，取 W 与 ϕ_n 。 W 不满足特征方程，它满足的是式 (294)，也就是

$$\nabla^2 W = \kappa^2 A - \kappa^2 W = \kappa^2 (A - W). \quad (349)$$

再用一次格林第二恒等式：

$$\int_0^{R_0} r (\phi_n \nabla^2 W - W \nabla^2 \phi_n) dr = R_0 [\phi_n(R_0) W'(R_0) - W(R_0) \phi_n'(R_0)]. \quad (350)$$

右端的做法与前面完全一样： W 与 ϕ_n 都满足同一个齐次表面条件 $-kW'(R_0) = hW(R_0)$ 与 $-k\phi_n'(R_0) = h\phi_n(R_0)$ ，所以右端也是零。

于是左端为零。把两个拉普拉斯项代进去：

$$\int_0^{R_0} r \left[\phi_n \kappa^2 (A - W) - W \left(-\frac{\beta_n^2}{R_0^2} \phi_n \right) \right] dr = 0. \quad (351)$$

拆成三个内积：

$$\kappa^2 A \langle 1, \phi_n \rangle - \kappa^2 \langle W, \phi_n \rangle + \frac{\beta_n^2}{R_0^2} \langle W, \phi_n \rangle = 0. \quad (352)$$

把后两项合并：

$$\left(\frac{\beta_n^2}{R_0^2} - \kappa^2 \right) \langle W, \phi_n \rangle = -\kappa^2 A \langle 1, \phi_n \rangle. \quad (353)$$

两边同除以括号里的量：

$$\langle W, \phi_n \rangle = -\frac{\kappa^2 A}{\frac{\beta_n^2}{R_0^2} - \kappa^2} \langle 1, \phi_n \rangle. \quad (354)$$

再算 $\langle 1, \phi_n \rangle$ ，它是一次可以直接做的积分：

$$\langle 1, \phi_n \rangle = \int_0^{R_0} r J_0 \left(\frac{\beta_n r}{R_0} \right) dr = \frac{R_0^2}{\beta_n} J_1(\beta_n). \quad (355)$$

这一步用到了 $\int_0^z x J_0(x) dx = z J_1(z)$ 。验证方法是对 $z J_1(z)$ 求导：由乘积法则与 $\frac{d}{dz} (z J_1(z)) = z J_0(z)$ ，确实得到被积函数。

把式 (355) 与式 (345) 一起代入式 (348)：

$$b_n = -\frac{1}{\frac{R_0^2}{2} [J_0^2 + J_1^2]} \cdot \left(-\frac{\kappa^2 A}{\frac{\beta_n^2}{R_0^2} - \kappa^2} \right) \cdot \frac{R_0^2}{\beta_n} J_1(\beta_n), \quad (356)$$

其中 J_0 、 J_1 的自变量都是 β_n 。两个负号相乘得正， R_0^2 与分母里的 R_0^2 抵消，2 移到分子：

$$b_n = \frac{2\kappa^2 A J_1(\beta_n)}{\frac{\beta_n [J_0^2(\beta_n) + J_1^2(\beta_n)]}{\frac{\beta_n^2}{R_0^2} - \kappa^2}}. \quad (357)$$

记归一化系数

$$N_n = \frac{2J_1(\beta_n)}{\beta_n [J_0^2(\beta_n) + J_1^2(\beta_n)]}, \quad (358)$$

则

$$b_n = \frac{A\kappa^2 N_n}{\frac{\beta_n^2}{R_0^2} - \kappa^2} \quad (359)$$

前四个系数是

$$b_1 = 47.2789118, \quad b_2 = -0.6580347, \quad b_3 = 0.0967137, \quad b_4 = -0.0276945. \quad (360)$$

这一节要记住的 正交性来自两个特征函数共享同一个齐次边界条件，证明的关键是格林恒等式的边界项恰好为零。展开系数不需要直接做内积，用格林恒等式可以化成只含 $\langle 1, \phi_n \rangle$ 的表达式，而后者有一次现成的积分公式。

34 第六步：把三部分拼起来，并做三个自检

34.1 最终的公式

把准静态部分、瞬态部分与烘房温度本身合起来：

$$T(r, t) = T_{\text{air}}(t) + W(r)e^{-t/\tau_T} + \sum_{n \geq 1} b_n J_0\left(\frac{\beta_n r}{R_0}\right) e^{-\gamma_n t} \quad (361)$$

其中

$$T_{\text{air}}(t) = T_{\text{set}} - Ae^{-t/\tau_T}, \quad W(r) = A + BJ_0(\kappa r), \quad (362)$$

$$\kappa = \frac{1}{\sqrt{\alpha\tau_T}}, \quad B = \frac{hA}{k\kappa J_1(\kappa R_0) - hJ_0(\kappa R_0)}, \quad (363)$$

$$\beta_n \text{ 由 } \beta J_1(\beta) = \text{Bi} J_0(\beta) \text{ 给出,} \quad \gamma_n = \frac{\alpha\beta_n^2}{R_0^2}, \quad b_n = \frac{A\kappa^2 N_n}{\frac{\beta_n^2}{R_0^2} - \kappa^2}. \quad (364)$$

34.2 自检一：初始时刻

在 $t = 0$ 时，式 (361) 变成

$$T(r, 0) = T_{\text{air}}(0) + W(r) + \sum_{n \geq 1} b_n J_0\left(\frac{\beta_n r}{R_0}\right). \quad (365)$$

由式 (266), $T_{\text{air}}(0) = T_{\text{set}} - A = 28$ 。而由式 (335), 求和那一项恰好等于 $-W(r)$ 。两者相加为零, 所以 $T(r, 0) = 28$, 与初始条件 (269) 一致。

这个自检说明展开系数确实定对了。如果 b_n 的正负号弄错, 这一步就会得到 $28 + 2W(r)$, 立刻暴露错误。

34.3 自检二：无穷时刻

当 $t \rightarrow \infty$ 时, $e^{-t/\tau_T} \rightarrow 0$, $e^{-\gamma_n t} \rightarrow 0$, 所以 $T \rightarrow T_{\text{set}} = 50.212$, 与烘房温度一致。物理上说得通: 放得足够久, 药材温度会趋近烘房温度。

34.4 自检三：表面热流

在 $r = R_0$ 处把式 (361) 对 r 求导, 再乘以 $-k$, 得到的就是表面导热量。由 W 与 ϕ_n 各自满足的齐次表面条件, 加上 T_{air} 与 r 无关, 可以逐项验证

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h(T(R_0, t) - T_{\text{air}}(t)). \quad (366)$$

即内部导出的热量等于表面对流带走的热量, 边界条件满足。

34.5 数值结果

取 $t = 1800$ 秒, 代入式 (361) 得

$$T(0, 1800 \text{ s}) = 34.0424989153 \text{ }^\circ\text{C}, \quad T(R_0, 1800 \text{ s}) = 37.1989219992 \text{ }^\circ\text{C}. \quad (367)$$

保留四位小数就是 34.0425 与 37.1989, 正是题目要求的问题一的两个关键答案。

35 第七步：级数收敛有多快

35.1 衰减因子决定的收敛速度

级数第 n 项含因子 $e^{-\gamma_n t}$ 。在 $t = 1800$ 秒, 四个衰减因子乘时间的值是

$$\gamma_1 t = 1.529, \quad \gamma_2 t = 13.191, \quad \gamma_3 t = 39.483, \quad \gamma_4 t = 80.742. \quad (368)$$

把它们换成指数的值：

$$e^{-1.529} = 0.217, \quad e^{-13.191} = 1.9 \times 10^{-6}, \quad e^{-39.483} = 7.1 \times 10^{-18}, \quad e^{-80.742} = 1.2 \times 10^{-35}. \quad (369)$$

再看系数的大小： $b_1 = 47.3$ ， $b_2 = -0.66$ ， $b_3 = 0.097$ 。第二项的贡献量级是 $0.66 \times 1.9 \times 10^{-6} \approx 1.2 \times 10^{-6}$ ，第三项是 $0.097 \times 7.1 \times 10^{-18} \approx 7 \times 10^{-19}$ ，已经低于双精度浮点数的分辨率。

35.2 取几项就够

直接算一下取不同项数的结果：

项数	$T(0, 1800)$	$T(R_0, 1800)$	变化
1	34.0425001446	37.1989215308	基准
2	34.0424989153	37.1989219992	第 10 位小数变化
3	34.0424989153	37.1989219992	不变
4	34.0424989153	37.1989219992	不变
8	34.0424989153	37.1989219992	不变

从第二项开始，数值在双精度下完全不变。所以这个级数实际上只需要两项。代码里求了两百个特征值，那是为了做自检和画图，真正的计算量比它小得多。

这一节要记住的 闭式解只需两项就到机器精度，这是它相对任何数值格式的最大优势：数值格式需要几十到几千个网格点，而这里只要两行公式。

36 第八步：这些公式在代码里长什么样

代码文件是 `analytic_p1.py`。它只做两件事：在导入时把与时间无关的量一次算好，然后在函数里按公式求和。

36.1 导入时算好的量

```

RHO, CP, K = 820.0, 2600.0, 0.36
R = cm.R0
ALPHA = K / (RHO * CP)
BI = cm.h * R / K
A = cm.Tset - cm.T0
KAPPA = np.sqrt(1.0 / (ALPHA * cm.tauT))
NROOT = 200
BET = cm.bessel_roots(BI, NROOT)
GAM = ALPHA * BET ** 2 / R ** 2
_B = cm.h * A / (K * KAPPA * j1(KAPPA * R) - cm.h * j0(KAPPA * R))
_NN = 2 * j1(BET) / (BET * (j0(BET) ** 2 + j1(BET) ** 2))
BN = A * KAPPA ** 2 * _NN / (BET ** 2 / R ** 2 - KAPPA ** 2)

```

逐行对照前面的推导：

第 1 行是问题一的三个常数物性，来自附录 2。

第 2 行取半径。`cm.R0` 是 0.02 米。

第 3 行算热扩散率 $\alpha = k/(\rho c_p)$ ，对应式 (293) 上面那一段的数值。

第 4 行算毕渥数 $Bi = hR_0/k$ ，它出现在特征方程 (331) 里。

第 5 行算 $A = T_{\text{set}} - 28$ ，对应式 (266)。

第 6 行算 $\kappa = 1/\sqrt{\alpha\tau_T}$ ，对应式 (293)。

第 7 行指定求 200 个特征值。

第 8 行调用 `cm.bessel_roots` 求特征方程的正根，得到数组 `BET`，它的前四个元素就是式 (332)。

第 9 行算衰减率数组 $\gamma_n = \alpha\beta_n^2/R_0^2$ ，对应式 (333)。

第 10 行算式 (305) 给出的 B 。`j0` 与 `j1` 是 `scipy` 提供的第一类贝塞尔函数。

第 11 行算归一化系数 N_n ，对应式 (358)。注意代码里用的是数组运算，`BET` 是长度 200 的数组，所以这一行同时算出全部 N_n 。

第 12 行算展开系数 b_n ，对应式 (359)。同样是一次算完全部。

36.2 函数里的求和

```

def T_exact(r, t, Tset=None, tauT=None, nterm=NROOT):
    Ts = cm.Tset if Tset is None else Tset
    tau = cm.tauT if tauT is None else tauT
    a = Ts - cm.T0

```

```

kap = np.sqrt(1.0 / (ALPHA * tau))
B = cm.h * a / (K * kap * j1(kap * R) - cm.h * j0(kap * R))
bn = a * kap ** 2 * _NN / (BET ** 2 / R ** 2 - kap ** 2)
r = np.atleast_1d(np.asarray(r, dtype=float))
val = np.full(r.shape, Ts - (Ts - cm.T0) * np.exp(-t / tau))
val += (a + B * j0(kap * r)) * np.exp(-t / tau)
for n in range(nterm):
    val += bn[n] * j0(BET[n] * r / R) * np.exp(-GAM[n] * t)
return val

```

第 2 行与第 3 行允许调用者临时换一组环境标定值。参数为 **None** 时用默认值，给定了就用给定值。这个设计让第十节的敏感性检验成为可能：只要换一个 **Tset** 就能重算整条公式。

第 4 到第 7 行把依赖环境标定值的四个量在函数内部重算一遍，而不是直接使用模块级的那一份，否则换了 **Tset** 以后公式会自相矛盾。第 4 行算 $a = T_{\text{set}} - 28$ ，对应式 (266) 下面那句话；第 5 行算 κ ，对应式 (293)；第 6 行算 B ，对应式 (305)；第 7 行算 b_n ，对应式 (359)。

第 8 行把输入的 r 转成数组，即使传入一个标量也能继续做数组运算。

第 9 行是整个公式的第一部分：烘房温度 $T_{\text{set}} - (T_{\text{set}} - 28)e^{-t/\tau}$ ，对应式 (266)。**np.full(r.shape, ...)** 表示生成一个与 r 形状相同、每个元素都等于这个值的数组。

第 10 行加上第二部分 $W(r)e^{-t/\tau}$ ，其中 $W(r) = a + BJ_0(\kappa r)$ 写在括号里。

第 11 行与第 12 行是循环累加第三部分，每一项是 $b_n J_0(\beta_n r / R_0) e^{-\gamma_n t}$ 。第 11 行的 **range(nterm)** 决定累加多少项。

第 13 行返回结果。返回的是数组，即使输入是标量。

36.3 特征方程怎么求根

特征方程 (331) 没有解析解，只能数值求根。代码在 **common.py** 里的 **bessel_roots** 函数：

```

def bessel_roots(Bi, n, beta_min=0.0):
    from scipy.special import jnp_zeros
    z = jnp_zeros(1, n + 2)
    br = [0.0] + list(z)
    roots = []
    f = lambda b: b * j1(b) - Bi * j0(b)

```

```

lo = 1e-12
hi = z[0] - 1e-12
if f(lo) * f(hi) < 0:
    roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
for i in range(len(z) - 1):
    lo, hi = z[i] + 1e-12, z[i + 1] - 1e-12
    if f(lo) * f(hi) < 0:
        roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
    if len(roots) >= n:
        break
return np.array(roots[:n])

```

这个函数做的事是：先把实轴分成一段一段小区间，保证每个小区间里恰好有一个根，然后在每个区间里用布伦特法把根精确找出来。这种方法叫括号法。

第 3 行取 J_1 的前 $n + 2$ 个正零点，记作数组 z 。为什么用 J_1 的零点来分段，理由是这样的：特征函数 $\phi(r) = J_0(\beta r/R_0)$ 在 $(0, R_0)$ 内变号 k 次，对应于 β 落在 J_0 的第 k 个零点之前。而由贝塞尔函数的交错性质， J_0 的零点与 J_1 的零点互相穿插。用 J_1 的零点把区间切开，每个小区间里函数 $f(\beta) = \beta J_1(\beta) - \text{Bi}J_0(\beta)$ 的符号改变一次，正好给出一个根。

第 5 行定义待求根的函数 f ，它就是把特征方程 (331) 移到一边。

第 6 到第 9 行处理第一个根。第一个根落在 $(0, z_1)$ 之内。区间左端点不取 0 而取 1×10^{-12} ，是因为 $\beta = 0$ 处 $J_1(0) = 0$ 同时 $J_0(0) = 1$ ，函数值恰好是 $-\text{Bi}$ ，虽然不算奇异，但留出一点余量更稳妥。右端点也缩进 10^{-12} ，避免正好落在 J_1 的零点上，那里函数值虽然有限，但会让符号判断退化。

第 10 到第 15 行对其余的区间逐个处理。每个区间的两个端点都向内缩进 10^{-12} ，然后检查两端函数值是否异号。异号说明区间内有根，调用 **brentq** 求出来。

第 14 行检查是否已经找够 n 个根，找够就跳出循环。

第 16 行取前 n 个根返回。

布伦特法是一种把二分法与割线法结合的求根方法，在函数连续且两端异号的前提下一定收敛，而且收敛速度接近二阶。它的容差参数 **xtol** 与 **rtol** 分别控制自变量的绝对与相对精度，这里都取到了双精度极限附近。

为什么不能直接用牛顿法，因为那需要推导 f 的导数，而布伦特法不需要导数，写起来更短，也更不容易出错。

36.4 自检

`analytic_p1.py` 的 `_selftest` 函数在程序直接运行时打印一串自检量，它们全部出现在代码包的复跑报告里。下面列出其中四项：

自检量	数值
热扩散率 α	$1.688555 \times 10^{-7} \text{ m}^2/\text{s}$
毕渥数 Bi	1.388889
κ 与 κR_0	56.163269 m^{-1} 与 1.123265
特征方程残差 $\max_n \beta_n J_1(\beta_n) - \text{Bi} J_0(\beta_n) $	1.8×10^{-12}

自检量	数值
前四个特征值 β_1 到 β_4	1.4184725188, 4.166469601, 7.2084784644, 10.3082731535
前四个展开系数 b_1 到 b_4	47.27891, -0.658035, 0.096714, -0.027695
$\gamma_n t$ 在 $t = 1800$ 秒的值	1.529, 13.191, 39.483, 80.742
初始时刻全场极差	3.5×10^{-8} 摄氏度

特征方程残差说明特征值求得很准：把求出的 β_n 代回特征方程，两边的差不超过 1.8×10^{-12} 。

初始时刻全场极差这一项需要解释。在 $t = 0$ 时，式 (335) 要求级数之和恰好等于负的 $W(r)$ ，所以这一项衡量的就是展开系数定得准不准。程序用全部 200 项去算，得到的极差是 3.5×10^{-8} 摄氏度。它比特征方程残差大四个数量级，原因是 b_n 衰减得慢，量级只有 $\beta_n^{-5/2}$ ，被截掉的尾巴在高阶上仍有微小贡献。在 $t = 1800$ 秒处，这些高阶项还要再乘上 $e^{-\gamma_n t}$ ，被压得更低，所以对最终结果没有影响，这一点在第七节已经算过。

这四项合起来说明三件事：特征值对、展开系数对、级数截断的误差可以忽略。

37 第九步：用这个闭式解检验数值解

闭式解最重要的用途是检验数值解。把谱配置法的结果与式 (361) 逐点比较，得到下表。比较的是全场 201 个半径位置上的最大偏差。

空间节点参数 N	全场最大偏差 / K	与上一行的比值
4	7.586×10^{-6}	—
6	2.876×10^{-10}	2.6×10^4
8	1.087×10^{-10}	2.6
12	1.110×10^{-10}	1.0
16	1.129×10^{-10}	1.0
24	1.140×10^{-10}	1.0
32	1.144×10^{-10}	1.0
64	1.139×10^{-10}	1.0

读这张表要注意三点。

第一， N 从 4 加到 6，误差掉了四个数量级。这是谱收敛的典型特征：节点数稍微增加，精度就跳一个台阶，而不是像有限差分那样按固定的阶数慢慢下降。

第二，从 $N = 8$ 开始，误差停在 1.1×10^{-10} 不再下降。这不是方法失效，而是碰到了时间积分容差的平台。本次比较把时间容差固定为 10^{-12} ，时间离散引入的误差就在 10^{-10} 量级，空间误差再小也被它盖住。

第三，误差的水平 10^{-10} 开尔文意味着什么。药材温度是几十摄氏度，双精度浮点数大约有十六位有效数字，所以 10^{-10} 已经接近这台计算机能表示的极限。换句话说，数值解与精确解在双精度意义上没有差别。

这一节要记住的 闭式解的价值在于它是可以信赖的参照物。数值解与它吻合到 10^{-10} 开尔文，说明空间离散、边界处理、时间推进三个环节都没有写错。如果边界通量的符号写反，这个比较会立刻出现大偏差。

38 第十步：用这个闭式解做的敏感性检验

38.1 一个必须回答的问题

环境标定给出 $T_{\text{set}} = 50.212101$ 、 $\tau_T = 1877.4941$ 。程序里取的是三位有效数字 $T_{\text{set}} = 50.212$ 、 $\tau_T = 1877.5$ 。问题一要求保留四位小数，那么四位小数会不会被这个取舍影响。

用闭式解来回答，因为它算得快，换一组常数只要改两个输入。结果如下表，其中前两列是输入的标定值，后两列是 $t = 1800$ 秒的温度。

标定口径	$T(0, 1800) / ^\circ\text{C}$	$T(R_0, 1800) / ^\circ\text{C}$
全精度拟合, 50.212101 与 1877.4941	34.0425404	37.1989829
三位有效, 50.212 与 1877.5	34.0424989	37.1989220
设定值减 0.002 度, 50.2100	34.0419548	37.1980937
设定值加 0.002 度, 50.2140	34.0430430	37.1997503

38.2 怎么读这张表

第一, 全精度与三位有效数字两行之差是 4.2×10^{-5} 摄氏度与 6.1×10^{-5} 摄氏度, 都在第 5 位小数上, 四舍五入到四位小数以后两者相同。所以程序取三位有效数字是安全的。

第二, 把设定值改动 0.002 摄氏度, 中心温度就从 34.04195 变到 34.04304, 四舍五入以后是 34.0420 与 34.0430, 第 4 位小数被翻动了。这说明四位小数的最后一位, 其可靠性完全取决于标定常数的位数。

第三, 标定自身的标准误是 0.0303 摄氏度, 比上面那个 0.002 摄氏度大十五倍。也就是说, 拟合本身带来的不确定度远大于取舍带来的差别。四位小数是题目要求的输出格式, 它并不代表这四位数字都有同等的物理意义。

为什么要这样做: 这对写论文有什么影响

答: 不能声称四位小数是精确到四位小数的物理真值。正确的说法是: 在本文采用的标定口径下, 模型给出这四位数字; 标定常数的标准误为 0.0303 摄氏度, 对应结果在第 4 位小数上具有敏感性。把这一点写清楚, 比起含糊地说结果精确, 更站得住脚。

39 这一部分的小结

问题一的温度场推导已经完整了。回顾一下这条链:

第一步, 用 $w = T - T_{\text{air}}$ 把边界条件齐次化, 代价是方程右边多一个指数驱动项, 初始条件变成零。

第二步, 把解拆成准静态部分与瞬态部分, 要求方括号恒为零, 得到两个各自独立的问题。

第三步, 准静态部分是一个常微分方程, 解是 $A + BJ_0(\kappa r)$, 常数 B 由表面条件定出。

第四步, 瞬态部分是齐次热方程, 分离变量以后得到零阶贝塞尔函数, 特征值由特征方程 $\beta J_1(\beta) = \text{Bi}J_0(\beta)$ 决定。

第五步，用带权 r 的正交性把初始条件展开，系数 b_n 通过格林恒等式化为一个现成的积分，最终得到式 (359)。

第六步到第十步是检验：初始条件自检、稳态自检、边界热流自检、级数只需两项、与数值解吻合到 10^{-10} 开尔文、标定常数敏感性的量级。

这一节要记住的 问题一不需要任何数值离散。整条推导只用到了常微分方程、贝塞尔函数的两条性质、分部积分和正交展开。这就是把物理看透以后再动手算的价值：能算出闭式解的时候，不要急着上网格。